

Deep Research Plan for Slice Evaluation & Operational Test Architecture

A. Phase-by-Phase Design Specs

E1 – Data & Repository Integrity

E1-Requirements:

- Ensure all core repositories provide complete CRUD access to Slice's data models (Thesis, Observation, Trade). This includes adding missing methods for retrieval and listing as specified in the E1 plan.
- Introduce a **DataAccess** layer with strict method contracts (no dynamic attribute hacks) that consolidates repository operations and returns Pydantic data transfer objects (DTOs).
- Define Pydantic models for **PortfolioSnapshot** (with a list of positions and aggregated totals) and **PortfolioDepthSnapshot** (with breakdowns like concentration, factor exposures, and thesis exposures).
- Guarantee data integrity via round-trip tests: inserting and retrieving an entity yields the same content, and DataAccess methods return correctly typed objects.
- Use a test database or transaction to validate repository operations without side effects on real data.

E1-Research Summary:

The repository pattern decouples business logic from the database, improving testability and resilience . By defining clear interfaces (e.g., ThesisRepository.get_by_id), we avoid tight coupling to the ORM and can even swap persistence layers if needed . This abstraction allows injecting in-memory or mock repositories for unit tests . We will apply this pattern to make Slice's data layer more robust. Furthermore, using Pydantic models for returned data ensures validation of schema and types. Pydantic can enforce that data from SQLAlchemy (via ORM mode) matches expected fields . This adds a safety net against missing fields or type mismatches when moving data from the DB to the application. In testing persistence, it's generally accepted that true database operations are integration tests rather than pure unit tests . We'll embrace that by running repository tests against a test DB (e.g., a temporary SQLite or a PostgreSQL test schema) to verify end-to-end integrity. This aligns with best practices to use integration tests for persistence logic and reserve unit tests for pure logic components .

E1-Design:

- **Modules & Classes:**

Create or update src/slice/repositories/ modules:

- thesis_repo.py (ThesisRepository),
- observation_repo.py (ObservationRepository),
- trade_repo.py (TradeRepository).

Each will use SQLAlchemy or raw SQL (consistent with current codebase) and handle session management internally (if using SQLAlchemy sessions). We will implement the following methods:

```
class ThesisRepository:
    def insert(self, thesis: Thesis) -> Thesis: ...
    def get_by_id(self, thesis_id: str) -> Thesis | None: ...
    def list_all(self) -> list[Thesis]: ...
    def list_recent(self, limit: int) -> list[Thesis]: ...

class ObservationRepository:
    def insert(self, obs: Observation) -> Observation: ...
    def get_by_id(self, obs_id: str) -> Observation | None: ...
    def list_for_thesis(self, thesis_id: str) -> list[Observation]: ...
    def list_recent(self, limit: int) -> list[Observation]: ...

class TradeRepository:
    def insert(self, trade: Trade) -> Trade: ...
    def list_all(self) -> list[Trade]: ...
    def list_by_thesis(self, thesis_id: str) -> list[Trade]: ...
    # (Optionally, list_recent or list_by_date if needed)
```

- These methods correspond to the required data operations. For example, ThesisRepository.list_recent(limit) will execute a SQL query ordering by created date (or primary key) and return up to limit theses. Similarly, ObservationRepository.list_for_thesis(thesis_id) will query observations filtering by a thesis reference (likely using a join table or a thesis_refs field, depending on schema). All repository methods return domain model instances (the ORM or dataclass objects like Thesis), not Pydantic yet – conversion to Pydantic happens in the DataAccess layer.
- **DataAccess Layer:**

Update src/slice/intelligence/context/data_access.py (or create a new module if more appropriate, e.g., src/slice/data_access.py) to implement:

```
class DataAccess:
    def __init__(self, thesis_repo: ThesisRepository, obs_repo:
ObservationRepository,
```

```

        trade_repo: TradeRepository, scenario_repo:
ScenarioRepository | None = None):
    ...
    def get_thesis(self, thesis_id: str) -> Thesis | None: ...
    def get_all_theses(self) -> list[Thesis]: ...
    def get_observations_for_thesis(self, thesis_id: str) ->
list[Observation]: ...
    def get_recent_observations(self, limit: int = 10) ->
list[Observation]: ...
    def get_current_portfolio(self) -> PortfolioSnapshot: ...
    def get_portfolio_depth(self, theses: list[Thesis]) ->
PortfolioDepthSnapshot: ...
    # plus any other context-building methods needed by other phases

```

- The `DataAccess` methods will use the underlying repos. For example, `get_thesis(id)` uses `thesis_repo.get_by_id(id)`. For composite results:
 - `get_current_portfolio()` will call `trade_repo.list_all()` to retrieve all trades, then aggregate into a `PortfolioSnapshot` DTO.
 - `get_portfolio_depth(theses)` may compute metrics like concentration (e.g., largest asset or sector exposure as a percentage) and exposures per thesis. We will implement it in a basic way: e.g., concentration could be a dict of `{asset_name: portfolio_percent}` for top N assets, factor exposures might be left as empty or stub dict if no factor data, and `thesis_exposures` a dict mapping thesis ID to current portfolio value allocated to that thesis (we can compute this by summing values of trades associated with each thesis).
- **DTO Models (Pydantic):**

Define these in `src/slice/models/portfolio.py` (new file) or similar:

```

from pydantic import BaseModel

class Position(BaseModel):
    asset: str
    quantity: float
    value: float # current market value of this position

class PortfolioTotals(BaseModel):
    portfolio_value: float
    gross_exposure: float
    net_exposure: float

class PortfolioSnapshot(BaseModel):
    positions: list[Position]
    totals: PortfolioTotals

class PortfolioDepthSnapshot(BaseModel):
    concentration: dict[str, float] # e.g., {'AAPL': 0.15, 'MSFT':
0.10, ...} top positions or sectors by % of portfolio
    factors: dict[str, float] # e.g., {'Duration': 4.5, 'Beta': 1.2}
(stubbed or simple for now)
    thesis_exposures: dict[str, float] # e.g., {thesis_id: current_value}
showing how much each thesis contributes

```

- The PortfolioSnapshot.totals includes portfolio_value (the sum of all position values, which is the current portfolio equity value), gross_exposure (sum of absolute position values, so if short positions exist, they add positively to gross exposure), and net_exposure (sum of position values, which in a long-only world equals portfolio_value; if shorts exist, net_exposure = portfolio_value including shorts as negative values). We ensure portfolio_value is present as required.

These models use Pydantic to validate data types. We'll enable orm_mode = True in the Config of these models if we directly pass ORM objects (though here we likely construct them from basic Python dicts/lists) .

- **Data Integrity Behaviors:**

We will add validation in repository inserts (e.g., ensure no duplicate primary keys if using client-provided IDs, handle exceptions for DB constraints, etc.). If an insert fails (due to DB error), the repository should raise an exception (which tests can expect). We won't rely on hasattr or dynamic checks at runtime; instead, the DataAccess and DTO layers guarantee the presence of needed fields (e.g., PortfolioTotals.portfolio_value is always set). This eliminates the current brittle behavior where UI tried to access portfolio["total_value"] and failed; now UI will use snapshot.totals.portfolio_value reliably.

E1-Testing Plan:

- *Repository Unit/Integration Tests:* For each repository, write tests that use a temporary database. For instance, tests/test_thesis_repository.py will create a Thesis object (perhaps using a factory or just manually constructing with required fields), insert it via ThesisRepository.insert, then retrieve it with get_by_id and verify that the returned object matches the original (IDs and fields equal). Also test list methods: insert multiple entries, then list_all returns the full list, list_recent(n) returns the most recent n (by creation date or id) – we can verify the length and that it's sorted correctly. Similar tests for ObservationRepository and TradeRepository (e.g., insert a trade, list by thesis ensures only trades for that thesis are returned). We'll use an isolated test database or transaction rollback in fixtures to avoid persistence between tests.
- *DataAccess Tests:* Use dependency injection to provide DataAccess with real repositories pointed at the test DB. Test methods like:
 - get_thesis(thesis_id) returns a Thesis or None appropriately. Use a non-existent ID to ensure None is returned without error.
 - get_observations_for_thesis(thesis_id) after seeding some observations for a specific thesis: verify all returned observations indeed have that thesis_id in their thesis_refs.
 - get_current_portfolio() on an empty trade set should return a PortfolioSnapshot with positions=[] and totals.portfolio_value = 0 (the Phase 9 tests expect a shape even if empty). After inserting some trades (e.g., a BUY trade for 100 shares of

XYZ at \$10, and another BUY for 50 shares of ABC at \$20), we can simulate a price source (perhaps monkeypatch the PriceSource used inside DataAccess – see E2 design) to provide current prices, then call `get_current_portfolio()`. The result's positions list should contain XYZ and ABC with correct quantities and values, and `totals.portfolio_value` should equal $(100price_XYZ + 50price_ABC)$. Also verify `gross_exposure` (which for all longs = `portfolio_value`) and `net_exposure` (same if no shorts) are computed.

- `get_portfolio_depth(theses)` test: Using the trades above and providing the corresponding Thesis objects list, check that `thesis_exposures` includes those thesis IDs with values matching the sum of trades for each. If multiple trades map to one thesis, the exposure aggregates them. For concentration, if we have 2–3 assets, check that the dict's top entries correspond to those assets and percentages sum to ~1 (within rounding). Since this may be simplistic, we might just verify keys exist and values make sense (e.g., each value = `asset_value/portfolio_value`). If factors is stubbed (empty or fixed values), ensure it's a dict present (could just be empty in tests).
- All DTOs should validate properly. For example, ensure that `PortfolioSnapshot.totals.portfolio_value` is a float and equal to the sum of positions' values (we can compute it separately in the test to compare).
- *Negative cases:* Try inserting a Thesis with missing required field (should raise an error before DB if our model validation catches it, or an `IntegrityError` if DB constraints fail). Ensure the repository surfaces an exception and doesn't return a partial object. Also test that listing methods don't throw if no records (just return empty list). These tests ensure robust, predictable repo behavior.

E1-Risks & Dependencies:

- *Schema Knowledge:* We assume the database schema supports these operations (e.g., `ThesisRepository.insert` knows which table/columns to insert into). If the current code uses raw SQL, we must carefully write queries to match column names. Lack of detail on the DB schema is a risk; we mitigate by scanning existing model definitions (which likely include `__tablename__` or similar). We'll clarify any ambiguity (e.g., how thesis ID is generated – if using UUID or serial).
- *Concurrent Access:* The design doesn't explicitly handle concurrent writes or transactions. For evaluation, this is acceptable, but for production we'd consider adding transactions or locks in critical sections. We assume single-user or sequential operations during testing.
- *Missing Repository Methods:* The codebase summary noted some repos lack methods we need. We will implement them as above. There is a dependency on confirming what constitutes "recent" ordering – we will assume by creation timestamp or ID (which may be a UUID/time-based). If timestamps are not stored, we may need to define "recent" via another heuristic or add a timestamp field.

- *Data Volume:* The simple `list_all` operations may be fine for evaluation (small data). In a real scenario with many records, pagination would be needed, but that's out of scope. We'll note that as a future consideration but not required in this evaluation phase.
 - *Assumption:* We assume a *PriceSource* interface will be available by E2 for computing portfolio values (to get current prices of assets). In E1 tests, we might stub it or set prices manually since E1 focus is structural integrity. The `get_current_portfolio()` might use a `price_source` injected into `DataAccess` or a global price service; if not yet implemented in code, we will include it in the E2 design.
-

E2 – Thesis → Quant → Risk/Scenario Path

E2-Requirements:

- Develop a **ThesisEvaluationService** that can deterministically evaluate a thesis's investment expression (asset allocation) against historical price data, producing standardized performance metrics and risk analyses.
- Use the existing Thesis model's `expression_legs` (each with asset, direction, and `size_pct`) to define a simple **static allocation strategy**. Validate that legs sum to $\leq 100\%$ and are evaluable (e.g., assets have price data available).
- Implement a daily rebalanced backtest: assume the portfolio reallocates to the target weights each day using daily closing prices (or a chosen price field). This yields an **equity curve** (time series of portfolio value) and performance metrics such as **total return**, **CAGR** (annualized return), **volatility** (annualized std. dev. of returns), **Sharpe ratio**, and **max drawdown**.
- Integrate a rudimentary risk assessment: produce a few risk metrics (e.g., concentration or exposure to largest asset, and a simple VaR or drawdown risk measure).
- Implement a **scenario analysis** engine: define a set of hypothetical shock scenarios (e.g., "Equity Crash: -20% to equities, +5% to bonds" or "Rate Spike: +100bps to yields, impact on assets") and calculate the portfolio's P&L under each. Provide results as a list of {name, `pnl_abs`, `pnl_pct`} (absolute and percentage change in portfolio value under the scenario).
- Ensure that `ThesisEvaluationService` outputs all these results in a **ThesisEvaluationResult** DTO, ready for use by later phases (LLM critique, UI, etc.), independent of any LLM or UI logic.
- All calculations should be **deterministic and repeatable** given the same input data (no randomness), to facilitate testing.

E2-Research Summary:

Backtesting is a fundamental step in validating a strategy – it simulates an investment strategy on historical data to evaluate performance . We adopt a straightforward backtesting approach for static allocation strategies: since the thesis expression is essentially a target weight for each asset, rebalancing daily to those weights means the portfolio’s daily return is the weighted sum of each asset’s return that day. This simplifies computation and ensures a consistent exposure (analogous to a continuously rebalanced index). The performance metrics we produce are standard in portfolio analysis. For instance, **CAGR (Compound Annual Growth Rate)** translates the total return over the backtest period into an annual growth rate. **Volatility** is typically the standard deviation of daily returns annualized ($\sqrt{252}$ factor for trading days). **Sharpe ratio** measures risk-adjusted return: $\text{Sharpe} = \frac{R_{\text{annual}} - R_f}{\sigma_{\text{annual}}}$ (often assuming risk-free rate $R_f \approx 0$ for simplicity) . **Max drawdown** is the largest peak-to-trough drop in the equity curve – an important risk metric indicating worst-case loss from a high point. By including these metrics, we align with best practices for strategy evaluation, giving a holistic view of performance and risk.

For scenario analysis, it’s common to apply shocks to asset prices or risk factors and see the impact on portfolio value. We’ll implement a simple version: define scenario shock vectors and compute instantaneous P&L as $\text{weight} * \text{price change}$. This resonates with stress testing used in risk management (e.g., shifting all equities down X% and recalculating portfolio) . The risk metrics we output will be rudimentary due to limited data (e.g., if one asset comprises 80% of the portfolio weight, that’s a concentration risk to flag). Although a full VaR calculation is complex, we can approximate a basic one (e.g., historical 95% worst daily loss from the equity curve as a proxy for VaR).

Our design draws on standard quantitative finance patterns: using vectorized operations or libraries like Pandas to compute returns. We note that specialized libraries (Backtrader, VectorBT, etc.) exist for backtesting and could offer richer functionality , but given Slice’s scope, a custom lightweight backtest is sufficient and easier to integrate. We favor clarity and testability: by generating synthetic price data in tests, we can predict the expected outputs and verify that ThesisEvaluationService computes them correctly.

E2-Design:

- **Modules & Classes:**

Create `src/slice/evaluation/thesis_evaluation.py` with:

```
class ThesisEvaluationService:
    def __init__(self, price_source, risk_engine=None, scenario_engine=None):
        self.price_source = price_source          # interface to get historical
price data
```

```

self.risk_engine = risk_engine or SimpleRiskEngine()
self.scenario_engine = scenario_engine or SimpleScenarioEngine()

def evaluate_thesis(self, thesis: Thesis) -> ThesisEvaluationResult:
    # 1. Validate thesis expression
    # 2. Fetch historical prices for assets
    # 3. Run backtest to get equity curve
    # 4. Compute performance metrics
    # 5. Compute risk metrics via risk_engine
    # 6. Compute scenario impacts via scenario_engine
    # 7. Construct and return ThesisEvaluationResult

```

- The `price_source` could be an abstraction (e.g., with method `get_history(asset: str, start: date, end: date) -> pandas.Series`) that retrieves daily price data for each asset. In production, this might connect to a database of market data or an API; for now, we might use a simplified in-memory store or a stub in tests. The **risk_engine** and **scenario_engine** are pluggable components:
 - `SimpleRiskEngine` can be a simple class with method `compute_metrics(equity_curve: pd.Series, thesis: Thesis) -> dict` that returns a few risk metrics. For example, it can compute max drawdown (by scanning the equity curve), concentration (max weight from thesis legs, since weights are static), and maybe the 5th percentile daily return as a proxy for VaR.
 - `SimpleScenarioEngine` with method `apply_scenarios(thesis: Thesis) -> list[ScenarioImpact]` where `ScenarioImpact` is a DTO for scenario results. It will have a set of predefined scenarios (we can define a list of dicts for shocks), and it will calculate portfolio P&L = $\sum(\text{weight}_i * \text{shock}_i)$ for each scenario. More explicitly, if a scenario says asset X drops 10%, and the thesis has 20% long in X, then $\text{pnl_pct} = 20\% * (-10\%) = -2\%$ of portfolio; `pnl_abs` could assume a base portfolio value (like 1.0 or \$100) to give a dollar impact (e.g., -2 dollars on \$100). We will include scenario assumptions in the output (so the user knows what the scenario means).
- **Backtest Implementation:**

Inside `evaluate_thesis`:

1. **Validate Thesis:** Check that $\sum(\text{leg.size_pct for leg in thesis.expression_legs}) \leq 100$. Also ensure each `leg.asset` has data available in `price_source`. If not, we raise a `ValueError` or return an error result (for now, likely raise and let higher layer handle).
2. **Fetch Data:** Determine a reasonable historical period. Possibly use a default (e.g., last 1 year) or based on thesis start date if provided. Use `price_source.get_history(asset)` for each asset in legs. Ensure all series have the same dates (intersect or fill missing with forward-fill or zeros if needed). We will likely align by date index.
3. **Calculate Daily Portfolio Value:** Start with an initial notional (say \$100 or \$10000 – the absolute number won't affect percentage metrics). For each day:

- If rebalancing daily: The weight is constant each day. So portfolio return on day $t = \sum(\text{weight}_i * \text{return}_{i,t})$. We can compute this by first calculating each asset's daily returns from prices.
- Compute cumulative equity curve by iteratively applying daily returns to initial capital. Alternatively, simpler: We can directly compute the equity curve via vectorized dot-product of weights with normalized price series. Specifically, normalize all asset prices to 1.0 at start, multiply by weight, sum across assets each day to get portfolio normalized value. This yields a series starting at 1.0 and tracking the portfolio's growth. Multiply by initial capital for dollar curve.
- If any leg is short (direction = "short"), treat weight as negative for that asset. That means the portfolio's return contribution is $-w * \text{asset_return}$ for that leg. We must be careful: a short leg's daily P&L = $-(\text{weight} * \text{asset_change})$. This is effectively how we include short positions.
- The result is an equity_curve: e.g., list of EquityPoint(date=datetime, value=float). We can define EquityPoint as a Pydantic model or just use basic types in the DTO.

4. Performance Metrics: Using the equity curve:

- $\text{total_return} = (\text{final_value} / \text{initial_value}) - 1$. If initial was 1.0 (100%), final minus 1 gives total return in fraction.
- Determine period in years: e.g., if data covers N trading days, $\text{years} = N/252$ (assuming 252 trading days/year). Then $\text{CAGR} = (\text{final_value}/\text{initial_value})^{(1/\text{years})} - 1$ if N is sufficiently large, else we can approximate or even skip CAGR if period < 1 year (or still compute it).
- Calculate daily returns array (e.g., daily % change of the equity curve). Compute volatility = $\text{stdev}(\text{daily_returns}) * \text{sqrt}(252)$.
- $\text{sharpe} = (\text{avg_daily_return} / \text{stdev_daily_return}) * \text{sqrt}(252)$ (assuming 0% risk-free), which is equivalent to $\frac{\text{CAGR}}{\text{volatility}}$ if using the same period.
- max_drawdown: iterate over equity curve computing the running maximum and tracking the largest percentage drop:

$$\max_t \left(\frac{\text{max equity up to } t}{\text{equity}_t} - \frac{\text{max equity up to } t}{\text{max equity up to } t} \right)$$

We'll get a value in $[0,1]$. Represent it as a positive fraction (e.g., 0.2 for a 20% drawdown).

5. Risk Metrics: Use risk_engine to compute additional metrics. For example, concentration = max(leg.size_pct) (largest single leg percent) could be one output (since if one leg is 80%, that's a risk concentration). If we have multiple asset classes, we could sum weights by asset class and report that too. Another metric: we can compute 1-day 95% VaR by taking the 5th percentile of daily returns distribution * portfolio value (with a negative sign to indicate loss). Or simpler, report the worst daily return observed (which is effectively historical VaR at

100% confidence for that period). The RiskSnapshot in the codebase had fields like book_net and book_gross, which correspond to net and gross exposure (we already compute those in PortfolioSnapshot). We might integrate these: e.g., risk metrics dict could include gross_exposure and net_exposure (which for a single thesis's hypothetical portfolio will just be 100% and maybe 0% net if perfectly hedged, but likely net ~100% too if not using leverage).

- We'll produce a risk_metrics: dict in ThesisEvaluationResult, for example: {"max_drawdown": 0.25, "concentration": 0.5, "VaR_95": -0.02} (meaning 2% one-day loss at 95% confidence).

6. **Scenario Analysis:** Use scenario_engine to apply shocks. Define a few scenarios in SimpleScenarioEngine – for instance:

- "Equity -20%": For each leg, if asset is categorized as equity (we might guess by asset ticker or have a mapping in ScenarioEngine), apply -20% shock to that asset's price. Compute portfolio impact = sum(weight_i * shock_i). If we have no classification, perhaps define scenarios per asset or broad ones like "All assets -10%" to simulate general drop.
- "Rates +1%": If any assets are bonds or rate-sensitive, apply price changes (e.g., bonds -5% if rates up, stocks maybe -2%). This part may be heuristic. We could use a simple assumption: if asset name contains certain keywords, but since this is more about having any scenario output, it's acceptable if our scenarios are simplistic or even trivial (the key is that the pipeline supports scenarios).
- For each scenario, calculate pnl_pct and then pnl_abs = pnl_pct * portfolio_value (assuming some base portfolio value, say current value or initial value).
- Wrap each in a ScenarioImpact model with fields name, pnl_abs, pnl_pct. Pydantic model:

```
class ScenarioImpact(BaseModel):
    name: str
    pnl_abs: float
    pnl_pct: float
```

•

6.

If we have a Scenario model already (the codebase had Scenario with assumptions, expected_impact, etc.), we might repurpose it, but likely easier to use our own DTO for output so we can include the computed P&L.

7. **Construct Output DTO:** Define ThesisEvaluationResult Pydantic model with fields:

```
class ThesisEvaluationResult(BaseModel):
    total_return: float
    cagr: float
    volatility: float
    sharpe: float
```

```

max_drawdown: float
equity_curve: list[EquityPoint] # or perhaps list[dict] with date and
value
risk_metrics: dict[str, float]
scenarios: list[ScenarioImpact]

```

- 7. We will populate this and return it from `evaluate_thesis`. This DTO cleanly separates the quant results from any presentation or LLM logic. (If `EquityPoint` is defined, it might be like `BaseModel` with `date: date` and `value: float`, but in JSON it could also just be two parallel lists or dict of `date->value`. To keep it simple, a list of points is fine for now.)

- **Integration with existing code:**

The risk interface (phase 5 code) defined `RiskSnapshot` with some fields, but since it's stubby, we won't use it directly for evaluation results. Instead, we'll treat our `risk_metrics` dict as an ephemeral result. If later we want to persist or present risk, we might map it into a `RiskSnapshot`. For now, `ThesisEvaluationResult` contains what we need. We should ensure fields like `total_return`, `sharpe`, etc., are floats even if not available (e.g., if the backtest is too short, we still fill something, or 0). We avoid returning NaN or None in metrics to simplify consumers (fill 0.0 or a sentinel for impossible metrics but likely our backtest always yields something).

E2-Testing Plan:

- *Synthetic Price Data:* Create a dummy `price_source` for tests that returns predictable price series (e.g., using Pandas or simple lists). For example, two assets A and B:
 - Asset A prices: start at 100 and increase 1% each day for 10 days.
 - Asset B prices: start at 100 and decrease 0.5% each day for 10 days.

This way, we can manually calculate expected portfolio results for a given weight.

- *Basic Performance Test:* For a thesis with one leg 100% in asset A (long):
 - The evaluation should essentially mirror asset A's performance. Compute expected `total_return` (which should be roughly $0.01 * 10 \text{ days} = \sim 10.5\%$ cumulative for 10 days with compounding), volatility (should equal asset's volatility), etc., and compare to `ThesisEvaluationResult`. We expect `risk_metrics['concentration'] = 1.0` (100% in one asset), `max_drawdown = 0` since A was monotonically increasing in this synthetic case.
- *Multi-Asset Test:* Thesis with 50% A, 50% B long. Since A up, B down, the portfolio might end slightly up or flat depending on magnitudes:
 - We can calculate day-by-day: $\text{daily return} = 0.5 * (A_return) + 0.5 * (B_return)$. Verify the service does this properly. Check that `total_return` is between A's and B's individual returns. Also check drawdown: since B is dragging, there might be a drawdown when A's gains not enough to offset B's losses at some point.

- Ensure `equity_curve` list length = number of trading days (10 in our test). The final equity value in that curve should equal $\text{initial} * (1 + \text{total_return})$.
 - Check Sharpe: if the average daily return is small but volatility is also small in our controlled data, the Sharpe should be reasonable. We might not verify exact number in test (because that could be sensitive to float error), but we ensure it's computed and within a plausible range (e.g., Sharpe positive if portfolio gained). For deterministic tests, we might assert e.g. $\text{sharpe} > 0$ for a positive-return portfolio.
- *Short Position Test:* Thesis with a short leg. For example, 100% short Asset A (which was rising). Then portfolio should go down since shorting a rising asset loses money. We expect `total_return` to be negative, and perhaps a large drawdown because it's steadily losing. This checks that our negative weight logic works. Also test a combination like 50% long A, 50% short A (a weird thesis effectively market neutral on A): that should theoretically yield ~0 return (the long and short cancel out) minus any compounding issues. Evaluate that our code handles opposite directions in same asset (if our design allows duplicate asset legs; if not, we won't have that scenario).
- *Scenario Output Test:* We will configure the `SimpleScenarioEngine` in test to have a known scenario, e.g., "All -10%" which applies -10% to both A and B. If our portfolio was 50/50, the expected `pnl_pct` = -10% (since the whole portfolio drops 10%). We verify that one of the `ThesisEvaluationResult.scenarios` has `name="All -10%"` and `pnl_pct ≈ -0.10` ($\pm$ some tolerance if floats). If we also include a scenario "A -10% only" (shock only asset A), we can manually compute expected P&L: if 50% in A, that scenario's `pnl_pct` should be -5%. We verify that as well.
 - Also verify the scenario list length matches the number of defined scenarios, and that each entry's name matches a known scenario name.
- *Risk Metrics Test:* Using the known equity curve in tests, we can manually compute max drawdown and see if `risk_metrics['max_drawdown']` matches. For example, with 100% asset A (monotonic up), `max_drawdown` expected 0 – test that. If we use an asset that goes down then up, we know a drawdown. We can craft a price series e.g., goes 100 -> 90 -> 110 (so a 10% drawdown from 100 to 90). Test that the service reports ~0.1 (10%) drawdown.
 - If we include VaR in risk metrics: for a small data set, 95% VaR might end up equal to the worst daily loss. We can identify that from our synthetic returns and check if the reported VaR corresponds.
- *Error Handling:* Feed an invalid thesis (e.g., legs sum to 150%). The service should raise an error. In a test, catch the exception and assert that it is a `ValueError` with an explanatory message (like "Total allocation > 100%"). Similarly, if we ask for an asset with no price data (simulate `price_source` returning empty), the service might raise a runtime error; we can test that path as well.
- *Performance of the code is not critical in tests*, but we might include a test to ensure evaluation runs under a certain time for a reasonable data size (though in Python, 10 years of daily data ~2500 points, trivial). If needed, we can measure that `evaluate_thesis` executes quickly for our synthetic input.

E2-Risks & Dependencies:

- *Data Availability:* The evaluation is only as good as the price data. We assume `price_source` can provide historical prices for all assets in the thesis. If an asset is something like an index or macro indicator, we need that data. If the codebase already has data ingestion (Phase 2 of original plan), we should integrate with it. If not, we might have to restrict evaluation to assets we have (e.g., stocks, or use dummy data). This dependency could limit testing realistic theses.
- *Rebalancing assumption:* Daily rebalancing might not reflect how a human PM executes a thesis (they might buy and hold weights, not constantly trade). We choose it for simplicity and determinism. The trade-off is transaction costs and trading frictions are ignored. We note this is a *modeling assumption*. Since our focus is evaluation consistency, that's acceptable. In the future, we could allow different rebalance frequencies.
- *Short positions modeling:* As discussed, our backtest treats short as negative weight. This assumes we can short indefinitely with no borrow cost and use proceeds freely – a simplification. It can lead to edge cases (e.g., if an asset's price goes to 0, a short position's gain is capped, but our model would treat it straightforwardly). Given this is just an evaluation harness, we accept these limitations but mark that a more robust engine should handle shorts with margin and borrowing costs.
- *Time alignment:* If assets have different trading days or one is missing data on a day (holidays, etc.), the backtest must handle it (likely by forward-filling last price or ignoring that day's return for that asset). Our simple engine will probably forward-fill or drop days to align. We must document this assumption. In tests we use perfect alignment to avoid complexity.
- *Calculating Sharpe for short periods:* If the backtest period is very short (few days), annualized metrics can be misleading (e.g., one-week return annualized could be huge). We assume a decent history length. We might still output them because the user requested those metrics, but it's a known caveat.
- *Integration with other phases:* The `ThesisEvaluationResult` will be consumed by E3 (LLM critique) and E4 (orchestrator). It's important we define it clearly (fields and types) so that LLM prompts or UI can refer to them. We depend on consensus with those phases about the output format. For instance, LLM might expect a JSON with keys like `"total_return"` etc. We will ensure keys are intuitive and documented.
- *Future extension:* If a more advanced quant engine (like **vectorbt** or **backtesting.py**) is later introduced, our `ThesisEvaluationService` could be refactored to use it under the hood. Our design is such that `ThesisEvaluationResult` remains the contract, so the rest of the system wouldn't change. This modular approach means our tests for correctness now also serve as regression tests for any engine swap later (the results should remain consistent).

E3 – LLM Tools & Evaluation Harness

E3-Requirements:

- Wrap the large language model (LLM) interactions in well-defined “tool” functions that each produce **structured, schema-defined outputs** rather than free-form text. These tools include:
 1. `llm_review_thesis(thesis: Thesis, evaluation: ThesisEvaluationResult) -> ThesisReview`
 2. `llm_cross_theses(theses: list[Thesis]) -> CrossThesisReport`
 3. `llm_query_intuition(question: str, observations: list[Observation]) -> IntuitionAnswer`
 4. `llm_daily_summary(context: DailyContext) -> DailySummary`
- For each output, define a **Pydantic schema** (DTO) that the LLM **must conform to**. For example, `ThesisReview` might have fields like `critique: str`, `questions: list[str]`, `risk_flags: list[str]`, `insufficient_context: bool`.
- Enforce strict prompting rules to minimize hallucination: In the system prompt, instruct the LLM to **use only provided context** and to signal if information is missing (by setting a flag or saying “I don’t know”). If the LLM cannot fulfill the request with given data, it should indicate that via the structured output (e.g., `insufficient_context = true`) instead of fabricating.
- Implement a **contract test harness** for these LLM tools: we will use a fake or stub LLM (or a canned JSON response) to verify that:
 - The output is parseable into the correct Pydantic model (i.e., valid JSON matching the schema).
 - Any references in the output (like IDs of theses or observations, or asset tickers) are a subset of the input context provided. This checks that the LLM isn’t introducing entities we didn’t supply.
 - The `insufficient_context` flag is properly used when applicable (for example, if we ask `llm_query_intuition` a question but provide no relevant observations, the model should not hallucinate an answer, it should mark insufficient context).
- Capture metrics around LLM usage for observability: track each call’s latency (already in `SessionResponse`), usage count, and any errors (like JSON parsing failures). Provide a diagnostics endpoint (e.g., `GET /api/v1/diagnostics/llm`) that returns aggregate metrics such as total calls per tool, average latency, error rates, etc., so we can evaluate LLM performance over time.
- Ensure **no direct DB calls from LLM functions** – all needed context must be passed in. LLM tools are pure functions of input context and the model’s response, to avoid hidden dependencies and enable easier testing.

E3-Research Summary:

To maintain reliability when integrating LLMs, it’s critical to constrain their output format. Recent techniques like OpenAI’s function calling or JSON **structured outputs** guarantee that the model’s response adheres to a specified JSON schema. This eliminates many “formatting” failures and reduces the need for multiple prompt attempts. We will leverage this concept: either

by using OpenAI's structured output feature or by a strict prompt template plus post-validation. The prompt will instruct the model to output JSON only, and we'll use Pydantic to define and validate the schema, similar to how the Guardrails library allows binding an LLM to a Pydantic schema . As backup, any output from the model will be parsed and validated; if parsing fails, we treat it as an error and can even re-prompt or set `insufficient_context` as a fallback.

Hallucination control is addressed through **grounding** the model with context and explicit instructions. A known best practice is to say *"Only use information explicitly provided in the prompt or retrieved context. If the answer is unknown or not in the context, respond with 'I don't know' (or set a flag)."* . By including this in the system or developer prompt for our LLM tools, we reduce the chance of the model making up facts. This aligns with prompt engineering principles for hallucination mitigation . In retrieval-augmented generation (RAG), it's standard to encourage the model to abstain if it lacks info , which we implement via the `insufficient_context` boolean.

We also consider using external frameworks or libraries: **Guardrails AI** provides a way to enforce output schemas and even repair minor JSON errors . We may not bring in the full library, but we adopt the same principle: *post-process and validate*. The LLM output will be run through a JSON parser and Pydantic model. If there's a mismatch (e.g., extra fields, wrong types), that's a failing test case we catch. Modern LLMs under function calling are quite good at respecting schemas, but as a safety net, we implement checks and possibly a single retry with a more forceful prompt if parsing fails. However, our primary plan is one-shot output enforcement to keep things deterministic in tests.

We incorporate telemetry for LLM calls because monitoring these is crucial in production (both for cost and performance). Best practices in LLM observability suggest tracking latency percentiles and error rates . We'll record each call's `latency_ms` (given by SessionOrchestrator or manually timed) and increment counters for successes/failures. These metrics give us insight into whether our prompts are causing delays (e.g., if model is struggling to format JSON and timing out or requiring long outputs).

E3-Design:

- **DTO Schemas:** Define Pydantic models for each LLM output:

```
class ThesisReview(BaseModel):
    critique: str
    questions: list[str]
    risk_flags: list[str]
```

```

    insufficient_context: bool = False

class CrossThesisReport(BaseModel):
    overlaps: list[str]
    contradictions: list[str]
    gaps: list[str]
    insufficient_context: bool = False

class IntuitionAnswer(BaseModel):
    answer: str
    references: list[str]          # e.g., list of observation IDs or titles
used
    insufficient_context: bool = False

class DailySummary(BaseModel):
    key_narratives: list[str]
    risk_highlights: list[str]
    thesis_references: list[str]   # e.g., thesis IDs or titles mentioned
    insufficient_context: bool = False

```

- These schemas capture structured information. We include an `insufficient_context` boolean in each, defaulting to False. The LLM is expected to set it True (and possibly leave other fields blank or with a placeholder message) if it cannot find a basis to answer from the input. We explicitly list fields like questions or risk_flags for ThesisReview to ensure we gather specific outputs (the critique might be a narrative text, but risk_flags could be a list like ["High leverage", "Concentration in USD"]). CrossThesisReport focuses on comparing theses (so overlaps, contradictions in their assumptions, gaps such as something one thesis addresses that another doesn't). IntuitionAnswer is essentially a Q&A answer with citations (references to observation IDs provided in context). DailySummary captures main narratives of the day, any risks to portfolio or theses, and references to which theses or positions are relevant.
- **Prompt Design:** For each function:
 - **llm_review_thesis:** We will construct a prompt like:
 - System role: "You are a macro investment analyst AI. You will receive a **thesis description** and its **quantitative evaluation results**. Analyze the thesis's soundness, identify any risks or weaknesses, and pose clarifying questions to improve it. If some context (like macro drivers or data) is missing and you cannot fully assess, set `insufficient_context` to true in your answer. Respond in JSON only, conforming to this format: { "critique": "...", "questions": ["..."], "risk_flags": ["..."], "insufficient_context": false }."
 - User content: will contain a JSON or structured payload including the thesis details (title, hypothesis, etc.) and summary of evaluation (we might feed in parts of ThesisEvaluationResult like performance metrics and key scenario outcomes). The context builder (from Phase 6 in code) already makes a thesis context JSON; we can use that structure or simplify it.
 - This directs the model to output exactly the fields we want. We explicitly say "Output must be valid JSON and nothing else." The Medium article on prompt engineering suggests including fallback rules in the prompt like returning nulls or blanks if missing info. We can say: "If you lack

information to answer any section, use an empty list or an apologetic note, but still set `insufficient_context` accordingly.”

- **llm_cross_theses:** System prompt: “You are an AI that compares multiple investment theses. Identify any overlapping ideas, any contradictions between them, and any gaps or missing perspectives across them. Only use the thesis information provided. If you don’t have enough info to compare, set `insufficient_context` true. Output JSON: { “overlaps”: [...], “contradictions”: [...], “gaps”: [...], “insufficient_context”: false }.”
 - We will supply the theses (maybe in a condensed form – e.g., a list of each thesis’s key points). The model should parse and then list things like common themes (overlaps), directly conflicting assumptions or conclusions (contradictions), and gaps (areas none of the theses cover or questions left unanswered collectively).
- **llm_query_intuition:** This is effectively a Q&A over the **observations** (which might be news articles, data points related to theses). We use a retrieval-augmented style: the observations list is the retrieved context (perhaps from a memory vector search). Prompt might say: “You are an AI assistant with knowledge of recent market observations. Answer the user’s question using ONLY the provided observations. Each observation is labeled with an ID and content. If the answer is not contained in these observations, do not guess – set `insufficient_context` true and provide no answer. If you do answer, include the IDs of any observation you used as references. Format your answer as JSON: { “answer”: “...”, “references”: [“obs1”, “obs2”], “insufficient_context”: false }.”
 - The observations themselves might be long texts. We might provide summaries to the model if needed (to avoid context length issues). But for now, assume we pass the text in prompt (the context builder might do that). The key is instructing the model not to go out of bounds – the line *“Only use information explicitly provided... If unknown, say ‘I don’t know’.”* is directly applicable here and will be in the prompt.
- **llm_daily_summary:** System prompt: “You are a financial assistant summarizing the portfolio’s status for the day. Given data about today’s market moves, portfolio performance, and any alerts, produce a concise summary including key narratives (what main themes affected the portfolio), any risk highlights (things a portfolio manager should watch), and mention which theses or positions were impacted. If data is insufficient, mark `insufficient_context`. Output JSON: { “key_narratives”: [...], “risk_highlights”: [...], “thesis_references”: [...], “insufficient_context”: false }.”
 - The context will likely include: a brief of portfolio P&L, any triggered alerts (like “Thesis X had a disconfirming event”), and possibly a few important observations of the day. The LLM should weave these into a short summary. `thesis_references` could list IDs or names of theses that are mentioned (so the UI can link to them).
- Across all prompts, we maintain a **consistent style**: JSON only, no extra prose, with correctly named fields. We will likely use the model’s temperature low (to reduce creative deviations) when generating these structured outputs.

- **LLM Client Integration:** We have a SessionOrchestrator in the code that calls the model and returns a SessionResponse (with raw text and latency). We can reuse it. Our llm_review_thesis implementation will do something like:

```
prompt = build_review_prompt(thesis, evaluation)
response = orchestrator.run_session(prompt, options=SessionOptions(/* maybe
model, etc. */))
raw_content = response.content # raw string from LLM
# Attempt to parse JSON
try:
    review = ThesisReview.parse_raw(raw_content)
except Exception as e:
    # Log parse error, maybe attempt a fix or set insufficient_context True
    raise LLMOutputError("Invalid JSON from llm_review_thesis") from e
# (Optionally validate references here)
return review
```

- We may add a small wrapper to strip any non-JSON prefixes the model might accidentally add (sometimes models preface JSON with text). But with strong instructions, ideally it won't. If needed, we can regex to find first "{" and last "}" to extract JSON substring. We'll design tests to catch if that's needed.
- **Reference Validation:** After obtaining (or constructing) the Pydantic object:
 - For ThesisReview: if it mentions tickers or assets outside the thesis's legs or risk_flags that are irrelevant, it's hard to automatically validate correctness. We might skip deep validation there. But for questions and critique, since they're free text, no easy check besides not being empty if context was sufficient.
 - For CrossThesisReport: similar – overlaps/contradictions should ideally reference the theses given. We can ensure it doesn't mention a thesis ID not in input. If theses had IDs 1,2,3, and output says "thesis 4 has ..." that's a hallucination. We could scan for any thesis identifiers not provided (we can include each thesis's title or ID in prompt and ensure output uses those).
 - For IntuitionAnswer: we will validate that each entry in references is indeed an ID of one of the observations passed in. This is straightforward (we know the list of observation IDs in context). If the answer references something not provided, that indicates hallucination or leakage – we would treat that as a **contract violation**. In test, we can simulate a model output referencing an ID not in the input and assert our harness catches it. The handling could be: remove unknown references and set insufficient_context=True because the model clearly drew on something outside. Or mark an evaluation failure. For now, we likely just assert in tests (since we won't actually let a real model run free in test).
 - For DailySummary: validate thesis_references are in the provided thesis list (the daily context will include maybe active thesis IDs). We can similarly cross-check.
- **Metrics & Diagnostics:** We will extend the SessionOrchestrator or the LLM tool functions to record usage:
 - e.g., have a module-level or singleton metrics collector: a dictionary like llm_stats = { "review_calls": 0, "review_errors": 0, "review_avg_latency": 0.0, ... } for each tool. After each call, update counts and a running average (or keep sum and count to compute avg on request). A more robust approach is to keep a list of latencies

and compute p50, p90 etc on demand , but given small scale, average and max latency might suffice.

- The diagnostics/llm endpoint will return JSON like:

```
{
  "llm_tool_metrics": {
    "thesis_review": {"calls": 5, "errors": 0, "avg_latency_ms": 1200},
    "cross_theses": {"calls": 2, "errors": 1, "avg_latency_ms": 950},
    "intuition_qa": {"calls": 3, "errors": 0, "avg_latency_ms": 800},
    "daily_summary": {"calls": 1, "errors": 0, "avg_latency_ms": 1500}
  }
}
```

- - We won't expose extremely fine-grained data (to keep it simple), but this helps identify if a particular tool is failing often or is slow.
 - We gather these metrics by hooking into each function: e.g., a decorator or manual increment at the start and after result. The orchestrator already provides latency_ms , so we use that for timing.

E3-Testing Plan:

- *Schema Contract Tests*: For each LLM function, simulate a scenario and ensure output fits the schema:
 - We will **monkeypatch the LLM call** to return a predetermined JSON string (as if the model responded ideally). For instance, for llm_review_thesis, craft a JSON string:

```
{"critique": "The thesis is plausible but relies too heavily on one driver.",
"questions": ["What if inflation rises faster than expected?"],
"risk_flags": ["Concentration in energy sector"],
"insufficient_context": false}
```

- - Then call our llm_review_thesis(thesis, eval) which will use the patched orchestrator and parse this. Verify that:
 - The returned object is an instance of ThesisReview with fields matching the JSON. (We can assert isinstance(result, ThesisReview) and then compare result.dict() to our input dict.)
 - No exception was raised in parsing.
 - insufficient_context is False as set.
 - If we provided a thesis with multiple legs, and our fake critique mentions something about those legs, we could ensure it's plausible (though we won't have logic to verify content).

We'll do similar for llm_cross_theses: prepare e.g.

```
{
  "overlaps": ["Both theses expect USD to weaken"],
  "contradictions": ["Thesis A is bullish on gold, Thesis B is bearish on gold"],
  "gaps": ["Neither thesis considers the impact of oil prices"],
  "insufficient_context": false
}
```

- - and test that parse works and fields align.
 - *References tests*: For `llm_query_intuition`, patch the LLM to return an answer with references, e.g. `{"answer": "Growth is slowing down.", "references": ["obs2", "obs5"], "insufficient_context": false}`. Provide an observations list to the function that contains `obs2` but not `obs5`. Our code should detect that “`obs5`” was not in the input list. In a test, we expect either an error or that the function cleans it. We have design choices:
 - Easiest: we simply assert that all references in result are in input. If our code does not yet enforce it, this test will fail, prompting us to implement a fix (like filtering out unknown references and possibly toggling `insufficient_context`). We should implement such a fix: e.g., inside `llm_query_intuition`, do:

```
answer = IntuitionAnswer.parse_raw(raw_content)
allowed_ids = {obs.id for obs in observations}
if any(ref not in allowed_ids for ref in answer.references):
    # Remove invalid refs
    answer.references = [ref for ref in answer.references if ref in
allowed_ids]
    answer.insufficient_context = True # mark that something was referenced
that wasn't given
```

- - - This ensures no hallucinated references sneak through. Our test will validate this behavior: after call, `result.references` should contain only “`obs2`” (`obs5` removed) and `result.insufficient_context` became `True`.
 - Also test the branch where context is truly insufficient: patch the LLM to output `{"answer": "", "references": [], "insufficient_context": true}` when we ask a question not covered by provided observations. Ensure our function returns that as `IntuitionAnswer` with `insufficient_context=True` and doesn’t try to fill in an answer. The harness basically passes this through, which is correct.
 - For `llm_daily_summary`, similar approach: fake a JSON output with some content and ensure the parsing yields a `DailySummary` object. We can also simulate insufficient context by sending minimal context and patching LLM output to have `insufficient_context:true`.
- *Robustness tests*: Feed a model response that has minor format issues and see if we handle it:
 - For example, patch the LLM to return `'{"critique": "text", "questions": ["q1", "q2"], "risk_flags": ["r1"], "insufficient_context": false}'` extra text (note extra text after JSON). Our function might currently fail to parse because of the trailing characters. We should test this and update our parsing logic to trim extraneous

text. Perhaps use a regex to extract the JSON substring. After implementing, test again to ensure it now passes.

- Another: model returns single quotes or unescaped quotes that break JSON. We might not fix every case, but our tests can highlight if small issues need addressing. (In a real guardrails implementation, it would auto-correct some of these, but we'll handle common ones if possible.)
- *Latency and metrics tests:* We can simulate a delay in LLM response to test latency capture. For example, monkeypatch the orchestrator's `run_session` to sleep 0.1s and then return a response with `latency_ms`. Check that our metrics collector recorded a latency ~100ms. This might be tricky if using actual time in tests (we can fudge by setting `SessionResponse.latency_ms` manually in the fake). Instead, we can directly set `SessionResponse.latency_ms = 1234` in the fake response and ensure after calling `llm_review_thesis`, our metrics `llm_stats["thesis_review"]["avg_latency_ms"]` reflects that value (since with one call, `avg` = that latency).
 - Also simulate an error: patch orchestrator to throw an exception (as if API failed). Our `llm_review_thesis` should catch or propagate it? We likely let it propagate as an exception (so that orchestrator errors are not swallowed). In testing, we might then catch it at a higher level. But ensure in metrics we increment the error count in that case. So test: monkeypatch to raise error, call function inside a `try/except` in test, ignore the exception but then check `llm_stats["thesis_review"]["errors"]` incremented by 1.
- *End-to-end integration tests (small scale):* After unit tests with patched LLM, we might do one integration-like test where we use the actual OpenAI API *only if environment allows* (likely we won't call real API in test due to costs/unpredictability). Instead, we could use a lightweight local LLM or just trust the patched tests. The main integration will be tested in E4/E5 end-to-end tests where the LLM portion is stubbed out for consistency.

E3-Risks & Dependencies:

- *Model behavior unpredictability:* Even with instructions, LLMs sometimes deviate (e.g., they might include a preamble like "Sure, here is the JSON: ..."). Our strict validation might mark that as failure. We should iterate on prompt wording to minimize this. In case of failure, we might need a retry strategy. Due to time, we might decide not to implement automatic retries, but log the event. This means in live runs, a bad response would result in an error that surfaces to the user or gets logged. For evaluation, that's acceptable as we can manually inspect those.
- *Schema evolution:* If we later realize we need more fields in, say, `ThesisReview` (maybe a summary rating or an advice), adding that field requires updating prompt and tests. We should keep the schema minimal but sufficient for now, and note any future extension.
- *OpenAI API limits:* If using actual API, long context (thesis text + evaluation data) might approach token limits. We should be mindful – maybe pass summary stats rather than raw time series into the prompt. Our approach might be to include only key numbers (`total_return`, `sharpe`, a couple of lines from scenarios) to guide the LLM's critique. This

is a prompt design dependency: we rely on the context builder (Phase 6 code) or craft our own mini-context. We assume the theses and data are short enough for our use (since this is an internal eval, it's likely fine).

- *Choice of LLM model:* The quality of outputs depends on GPT-4 vs GPT-3.5, etc. Given we focus on correctness of format and integration, we assume a capable model is used (likely GPT-4 for better adherence). If resource constraints push to a lesser model, we might need to tighten formatting even more or simplify tasks. This is noted as a risk if output quality suffers (e.g., GPT-3.5 might be more likely to include extra text or make mistakes with JSON).
- *Security & Privacy:* We should ensure that if any sensitive data is in context, the model is instructed properly (in our case, mostly market info – low sensitivity). Also, the JSON schemas should not inadvertently expose PII or internal info. This is not a big concern here but worth a mention in a real system.
- *Testing environment:* Since we will stub LLM calls, our tests assume the LLM obeys our prompts in the ideal way. Real-world might throw surprises (like slight format differences). Our harness is designed to catch those, but it means sometimes the orchestrator might have to call the model again or require prompt tweaking. During evaluation phase, we'll manually test with the actual model to calibrate prompts (outside automated tests).
- *Integration with Orchestrator:* The SessionOrchestrator's interface and return types must be well-understood. The codebase indicates SessionResponse.content and SessionResponse.latency_ms. We rely on that. If SessionOrchestrator requires certain parameters (like it might need a SessionOptions specifying which prompt template to use), we'll adapt accordingly.
- *Dependencies on Phase 6 context:* The llm_daily_summary depends on having a DailyContext (which likely comes from ContextBuilder Phase 6). We need to define DailyContext in our own terms to pass to the function. It might contain: active theses list, today's P&L numbers, maybe significant observations. We might create a Pydantic model for DailyContext or just use a dict. For clarity:

```
class DailyContext(BaseModel):  
    portfolio_snapshot: PortfolioSnapshot  
    alerts: list[Alert] # if we define an Alert model for disconfirmers  
    observations: list[Observation] # maybe top 1-3 daily obs  
    date: date
```

- Then llm_daily_summary(context: DailyContext) -> DailySummary. This context can be built in DailyUpdateSession (E4). Our LLM function will then format a prompt out of it.
- *Metrics storage:* We chose an in-memory approach for LLM metrics. This means if the app restarts, metrics reset, and in a multi-process scenario, metrics would be separate. For an evaluation harness, this is fine (we likely run single instance). For production readiness, we'd consider aggregating metrics in a central store or using an APM/observability tool, but that's beyond scope. We document that limitation but accept it here.
- *Ethical and correctness considerations:* The LLM might still produce a subtly wrong analysis that passes our structural tests. For example, it might misidentify a risk. Our harness won't catch that – that would require semantic evaluation, possibly by another AI

or manual check. Given the project scope, we focus on format correctness, assuming a domain-expert would review content. This is a known open question: ensuring **factual accuracy** of LLM outputs is non-trivial and not fully solved by our approach (we mitigate only some aspects like source grounding).

E4 – Orchestrator & End-to-End Sessions

E4-Requirements:

- Implement orchestrated session flows that chain together multiple steps (data retrieval, quant evaluation, LLM analysis, persistence) to achieve higher-level tasks. These sessions encapsulate the end-to-end pipeline for:
 - **Thesis Evaluation Session:** Given a thesis (by ID or object), run the quant evaluation (E2) and then the LLM review (E3), store the results, and return a comprehensive result object.
 - **Daily Update Session:** Perform a “daily update” for the portfolio – meaning: update all active theses/trades with new market data, compute current risk, detect any alerts (e.g., disconfirming evidence events), generate an LLM daily summary, and persist or report those outcomes.
- Define clear **session result DTOs** to represent what each session returns. For example, ThesisEvaluationSessionResult might include the thesis ID, the ThesisEvaluationResult (quant outputs), the ThesisReview (LLM outputs), and perhaps a TradePlan if one is generated for the thesis (bridging to E5).
- Expose these sessions via **API endpoints**:
 - POST /api/v1/thesis/{thesis_id}/evaluate triggers the ThesisEvaluationSession for that thesis and returns the session result JSON.
 - POST /api/v1/session/daily-update triggers the DailyUpdateSession (likely no input or maybe a date) and returns a DailyUpdateResult JSON.
 - Possibly an endpoint for approving trade plans (though that bridges into E5).
- Ensure these orchestrations handle errors gracefully: if any sub-step fails (e.g., LLM service error), the session should catch it and either return a structured error in the response or at least not leave partial data persisted. No session should depend on “future magic” – if something isn’t implemented (like a real risk engine), handle it in the code (e.g., fill with stub data or skip) so the pipeline still runs without manual intervention.
- Provide end-to-end tests that simulate realistic flows: from a thesis in the DB all the way to an API call and database outcome. This will validate that all pieces E1–E3 work together as intended when orchestrated by E4.

E4-Research Summary:

In application architecture, orchestrating multi-step workflows can be done through design patterns like *sagas* or workflow engines, but for our single-service context, a simpler approach is to write a sequential procedure that calls each component in order . The key is to maintain **atomicity** or compensating behavior: the Saga pattern in microservices ensures that if one step fails, previous steps can be undone . In our case, since everything is in one process and mostly read-compute-write steps, we can handle errors by not committing to the database until the end of a successful session, or by flagging entries as incomplete if partial results are saved. For example, we might insert an evaluation record only after both quant and LLM steps succeed. If LLM fails, we could either return an error without saving anything, or save the quant result and mark review as pending. Given this is an evaluation environment, we lean towards all-or-nothing for simplicity: on failure, roll back or do not save partial results, so repeated session runs can be tried without duplicates.

In Python, orchestrating within the service is straightforward – just calling functions in sequence. Ensuring this is testable means injecting dependencies (like using a fake LLM or a test database) and observing final state. We will design the sessions as classes or functions that accept dependency instances (DataAccess, ThesisEvaluationService, etc.) so we can substitute fakes in tests. This is akin to the *dependency injection* approach that improves test isolation .

Testing multi-step flows often requires an **integration test** approach, where you set up a scenario and run the actual code path end-to-end . We will implement such tests at the API level (using FastAPI's TestClient to simulate HTTP calls) to truly go through all layers, ensuring that e.g. the FastAPI route triggers the session, which touches the DB and the LLM stub, etc. This aligns with the idea of combining unit-tested components into an integration test to see they work together . Because these end-to-end tests are broader, we expect to write fewer of them and use controlled data, following the testing pyramid concept (lots of unit tests for pieces in E1–E3, a handful of end-to-end tests in E4) .

E4-Design:

- **Session Classes:**

```
class ThesisEvaluationSession:
    def __init__(self, data_access: DataAccess, eval_service:
ThesisEvaluationService, llm_tools: LLMTools):
        # llm_tools could be an object bundling llm_review_thesis and others,
or we can just call functions
        ...
    def run(self, thesis_id: str) -> ThesisEvaluationSessionResult:
        # 1. Load thesis and related data from DataAccess
        # 2. Call eval_service.evaluate_thesis(thesis) ->
ThesisEvaluationResult
```



```

        # 3. Call llm_review_thesis(thesis, evaluation_result) ->
        ThesisReview
        # 4. Persist results (maybe via some repository or DataAccess
        methods)
        # 5. Return aggregated result DTO
class DailyUpdateSession:
    def __init__(self, data_access: DataAccess, eval_service:
        ThesisEvaluationService, llm_tools: LLMTools, execution_adapter:
        PaperExecutionAdapter):
        ...
    def run(self, as_of_date: date = None) -> DailyUpdateResult:
        # 1. Load all active theses (and maybe their trades) via DataAccess
        # 2. Compute current portfolio snapshot
        (DataAccess.get_current_portfolio())
        # 3. Compute risk snapshot (via risk_engine or
        DataAccess.get_portfolio_depth)
        # 4. Check for alerts: e.g., for each thesis, see if any new
        observation (from DataAccess.get_recent_observations or so) matches a
        disconfirming driver.
        #     If yes, create an Alert object.
        # 5. Build DailyContext with data from steps 2-4
        # 6. Call llm_daily_summary(context) -> DailySummary
        # 7. Persist alerts and summary (perhaps in DB or as logs)
        # 8. Return DailyUpdateResult (which includes summary and maybe
        number of alerts, etc.)

```

- We might define a simple LLMTools class to group the LLM functions, or just use them directly if imported. Grouping might help injection (we can stub the whole LLMTools in tests).

ThesisEvaluationSessionResult DTO:

```

class ThesisEvaluationSessionResult(BaseModel):
    thesis_id: str
    evaluation: ThesisEvaluationResult
    review: ThesisReview
    trade_plan: TradePlan | None

```

- Here, trade_plan (coming from E5) could optionally be included if, at the end of evaluation, we auto-generate a trade plan proposal for that thesis. It's listed in the requirements that the session can optionally return a TradePlan. Perhaps we do: if the evaluation was successful and some criteria met, we could generate a TradePlan via PaperExecutionAdapter.create_plan_from_thesis and include it. But we might also handle plan approval separately (E5). For completeness, we include it so the UI can show "Proposed TradePlan" after evaluation. We can set it always (so user doesn't have to separately call plan generation).

DailyUpdateResult DTO:

```

class DailyUpdateResult(BaseModel):
    date: date
    portfolio_snapshot: PortfolioSnapshot

```

```

alerts: list[Alert]    # define Alert with fields like thesis_id, message,
maybe observation_id that triggered
summary: DailySummary

```

- This captures what happened on the daily run.
- **Persistence within sessions:**
 - For ThesisEvaluationSession: we want to save the quant result and the LLM review. We may create new repository models:
 - Possibly ThesisEvaluationRecord with thesis_id, evaluation JSON (or fields normalized in columns), timestamp.
 - ThesisReviewRecord with thesis_id, review JSON, timestamp.

Alternatively, simply update the Thesis status/fields:

- Mark the thesis as last_evaluated_date (we have a field for that presumably) and attach the evaluation and review as JSON in a column (if schema has a JSONB field or we add one). But the codebase didn't mention such fields.
- Given time, a simpler approach: do not store to DB in E4 (just return to client/UI). But requirement says "persist evaluation + review". So we should implement at least writing them somewhere.
- We can introduce a new repository evaluation_repo if needed. For now, an easier path: store evaluation and review as entries in the Observation table or an existing data store (this is a bit hacky). Better to make a new small table. For design, let's assume we add:

```

class EvaluationRepository:
    def insert_thesis_evaluation(self, thesis_id: str, eval:
ThesisEvaluationResult, review: ThesisReview) -> None: ...
    def get_last_evaluation(self, thesis_id: str) ->
tuple[ThesisEvaluationResult, ThesisReview] | None: ...

```

- - - It will probably store them as JSON strings in the DB (using Pydantic .json() output). This repository isn't in earlier phases, so it's new to implement now.
 - Since adding new tables in the middle of evaluation might be heavy, we could alternatively skip actual DB persistence in tests and just simulate it (i.e., pretend saving succeeded by maybe writing to an in-memory dict). However, the success criteria mentions persistence, so we likely implement it. We'll note as a slight architecture addition in our document that Phase E4 introduces minimal new tables for evaluation logs.
 - For DailyUpdateSession: persistence involves:
 - Saving any generated Alerts. Possibly an AlertRepository with insert. Or reuse Observation: since observations are pieces of news, an alert (like "disconfirming evidence for thesis X") could be stored as a special

Observation with actionable=True and a reference to thesis X. But that overloads the meaning. Better to have a distinct Alert model:

```
class Alert(BaseModel):
    thesis_id: str
    timestamp: datetime
    message: str
    observation_id: str | None # if the alert is tied to a specific
    observation
```

- - - And an AlertRepository to insert and list alerts.
 - Saving the DailySummary: perhaps not needed to persist beyond returning to UI, but could be logged or stored for record. If we want, a DailySummaryRecord table with date and content JSON.
 - For evaluation, we might not implement a full repository for these due to scope but will simulate insertion and retrieval via DataAccess (which currently has no alert methods, we might extend DataAccess or just call repository directly in session).
- **API Endpoints:**
 - In src/slice/api/intelligence_routes.py or a new session_routes.py, define:

```
@router.post("/thesis/{thesis_id}/evaluate")
def evaluate_thesis(thesis_id: str):
    session = ThesisEvaluationSession(data_access=get_data_access_dep(),
    eval_service=get_eval_service_dep(), llm_tools=get_llm_dep())
    try:
        result = session.run(thesis_id)
    except Exception as e:
        # Log and return error response
        raise HTTPException(status_code=500, detail=str(e))
    return result.dict()
```

- - This will produce a JSON response with the structure of ThesisEvaluationSessionResult (evaluation metrics and LLM review embedded).
 - @router.post("/session/daily-update") similar: instantiate DailyUpdateSession and call run. It might take a date param or use today by default. We'll include date in DailyUpdateResult (so UI knows which day's update it is). Error handling similarly.
 - Possibly a POST /thesis/{id}/approve-plan (E5 detail) – we can stub here or implement in E5. We mention it here as it touches orchestrator. It would call execution adapter to execute the plan. We'll flesh out in E5.
- **Error Handling in Sessions:**
 - If DataAccess.get_thesis returns None (thesis not found), we should return 404 to the API. Implement a check and raise HTTP 404 early.

- If `evaluate_thesis` (quant) raises (e.g., validation error), catch it and perhaps include that message in the response. We could return a 400 if it's an input issue (like invalid thesis configuration).
- If `llm_review_thesis` raises (maybe model error), we could still return the quant results and an indication that review failed. But that complicates the schema. Alternatively, treat it as a failure for the session. For now, likely bubble as 500 error. The PM can re-run later.
- In daily-update, even if LLM summary fails, we might still want to save alerts and partial data, since daily P&L update is crucial. In that case, we could return a `DailyUpdateResult` with `summary.insufficient_context=True` or an empty summary, and maybe an error flag. But to keep the interface clean, we might handle it as error vs success. Possibly better: don't break the entire daily update if summary fails – log it and return everything except summary or with summary marked incomplete.
- Logging: we'll include logging statements at each step for traceability (not strictly needed for the deliverable, but good practice).

The goal is no silent failures: either the combined session result is delivered, or a clear error is thrown so it can be caught by tests or UI.

E4-Testing Plan:

- *Thesis Evaluation Flow (Integration Test):*

Set up a test scenario in the database:

1. Insert a Thesis into the test DB (via `ThesisRepository`) with known fields (including a simple expression, e.g., 100% in "TESTASSET").
2. Insert historical price data for "TESTASSET" in the `price_source` stub accessible to `ThesisEvaluationService`. For instance, 10 days of data as in E2 tests.
3. Monkeypatch the LLM function `llm_review_thesis` to a test double that returns a fixed `ThesisReview` (to eliminate real LLM variability). For example, a lambda that just returns `ThesisReview(critique="Looks good", questions=[], risk_flags=[], insufficient_context=False)`. This isolates the focus to orchestration logic.
4. Use FastAPI's `TestClient` to simulate calling `POST /api/v1/thesis/{id}/evaluate`. FastAPI will resolve dependencies – we may need to override dependencies for `DataAccess`, `eval_service`, etc., to inject our test instances (FastAPI's `dependency_overrides` can assign our test `DataAccess` that points to the test DB, and a test `ThesisEvaluationService` configured with our stub `price_source`, plus the patched LLM).
5. Verify the HTTP response:
 - Status code 200.

- The JSON contains keys: thesis_id, evaluation, review (and possibly trade_plan as null or present if we included it).
 - Under evaluation, check that total_return etc. match what we expect for the test data. We already tested ThesisEvaluationService separately, but here we ensure the data flows through JSON. For example, if our price series had an obvious result (like +10%), confirm the response JSON evaluation.total_return ~0.10.
 - Under review, since we patched the function, it should contain exactly the fields from our dummy review. Verify that critique matches “Looks good” (the dummy text).
 - Confirm the DB now has an evaluation record if we implemented saving. We can query the EvaluationRepository or directly the table to ensure something was written for that thesis_id. The content should match the returned result (since presumably we saved what we returned).
6. Also test the case where a thesis ID doesn’t exist: call the endpoint with a fake ID, expect a 404 response.
 7. If possible, test error propagation: one way is to patch ThesisEvaluationService.evaluate_thesis to throw an exception (simulate a calculation error), then call API and verify it returns 500 with error message. This ensures our try/except and HTTPException logic works.
- *Daily Update Flow (Integration Test):*

We need some trades and observations for a realistic scenario:

1. Insert at least one Thesis and its associated Trade (via TradeRepository). For example, Thesis “Gold vs Inflation” with one trade: buy 10 units of GOLD at some price (simulate an existing position).
2. Insert an Observation that should trigger an alert: e.g., an observation with text “Inflation unexpectedly rose 1%” that we mark or link to the thesis’s disconfirming factor (maybe the thesis has a disconfirming indicator “inflation rises”). We can simulate that by checking observation categories or just in code consider any observation containing “rose” and thesis has “inflation” as a keyword triggers alert. If we had a formal link (thesis.monitor_indices?), we use that. For the test, we might cheat by directly expecting an alert to be generated for demonstration.
3. Price data: provide updated prices for current day for the assets in portfolio so that get_current_portfolio can compute P&L. E.g., GOLD’s price moved from purchase price 100 to 105.
4. Patch llm_daily_summary to return a simple DailySummary (like key_narratives [“Gold up on inflation fears”], risk_highlights [“Inflation rising beyond forecasts”], thesis_references [the thesis_id], insufficient_context=False).
5. Call POST /api/v1/session/daily-update.
6. Check response:
 - Contains date (likely today’s date).
 - portfolio_snapshot.totals.portfolio_value reflects updated values (if 10 units of GOLD now 105 each, value 1050, if initial cost was 1000).

- alerts list has at least one alert. Verify the alert's thesis_id matches our thesis and message contains something about inflation (depending on how we implement detection).
 - summary.key_narratives matches our patched summary content.
 - Also ensure any persistence: if we decided to store alerts in DB, verify they got stored (e.g., AlertRepository.list_all returns the new alert).
- 7. Test idempotence or repeatability: If we run daily-update two days in a row (simulate two calls with different dates or price changes), ensure it doesn't crash or duplicate incorrectly. This might be beyond initial scope, but for instance, if day 1 and day 2 both triggered the same alert, we might want only one alert or updated alert. Probably too detailed; we'll at least ensure calling it once works.
- 8. Error case: If no trades in portfolio, daily-update should still succeed (maybe portfolio value 0, no alerts, summary might be insufficient_context because nothing to report). Test that scenario: ensure 200 response with reasonable empty values.
- *Test Orchestrator Error Handling:*
 - Simulate a failure in LLM during daily summary: patch llm_daily_summary to raise an exception. Run daily-update and verify the whole endpoint does not fatally crash the test (we should catch and return 500). Or if we decide to handle it internally, perhaps we set summary.insufficient_context and still return 200. We should decide on consistent approach. For now, likely treat it as error (500) to keep it simple.
 - Simulate partial failures: e.g., what if one thesis evaluation in daily update fails (if we extended daily to eval each thesis quant, which we might not actually do daily). Not applicable if daily-update doesn't run eval_service on each thesis (we might not re-evaluate strategies daily, just update P&L).
- *End-to-end Data Consistency:*
 - After a ThesisEvaluationSession, if subsequently DailyUpdateSession is run, ensure data flows: e.g., if a plan was approved and trades exist, daily update should see them. This is more of E5 interplay: after E5 (approve), a trade exists and daily update picks it up. We will test that in E5.

E4-Risks & Dependencies:

- *Dependency on E1–E3:* Obviously these sessions depend on prior components functioning. If an upstream component returns unexpected data (e.g., ThesisEvaluationResult missing a field due to a bug), the session might fail. Our end-to-end tests are the safety net to catch such integration issues.
- *Data freshness:* The daily session assumes that price data for “today” is available for computing P&L. If our test or environment doesn't update prices, revalue_portfolio might use last known prices, which is okay. In real operation, we'd integrate a price fetch in daily-update (maybe calling price_source to get today's close for each asset). We should either incorporate that (if price_source can get latest price) or assume

`DataAccess.get_current_portfolio` already uses up-to-date prices by calling `price_source` internally.

- *Alert logic uncertainty:* The precise way to detect “disconfirmers triggered” is not fully specified. We assume something like: each thesis has disconfirmers text, and we scan new observations for matching keywords. This is heuristic and could yield false positives or miss things. For evaluation, the exact mechanism can be simple (even manual: e.g., if any new observation is linked to the thesis via `thesis_refs` and has `actionable=True`, generate an alert). We proceed with a basic method. This is a dependency on how theses and observations link, which was indicated by `Observation.thesis_refs`. So if an `Observation`’s `thesis_refs` includes a thesis and maybe a field that it’s a disconfirming evidence, we use that. Because we lack clarity, we state an assumption and implement a reasonable approximation.
- *Transaction management:* If we decide to persist results as we go, we should consider using a transaction so that either both evaluation and review records save or neither. Since we are likely just doing simple inserts, the small risk is one insert succeeds and the next fails, leaving data partially saved. Given low criticality, we may accept that risk or wrap in a transaction (SQLAlchemy session or manually manage). This complexity might be overkill for eval harness, but we note it as a consideration.
- *Running sessions repeatedly:* We should consider idempotency. For example, calling thesis evaluation twice could insert duplicate records. Maybe our `EvaluationRepository` should overwrite existing or have a unique per thesis per date. We didn’t specify a unique constraint; duplicates could occur. Not a big issue in test usage, but for cleanliness: we could delete old evaluations for that thesis before inserting a new one, or better, store a single latest record. Possibly an update rather than insert if one exists. For now, we might ignore duplicates or use timestamp to differentiate.
- *Orchestrator vs direct calls:* We have a `SessionOrchestrator` class in code but it’s oriented towards LLM sessions. We’re effectively writing our own orchestrator logic. This is fine. We just ensure not to confuse it with that orchestrator. Perhaps to avoid naming confusion, we won’t use the term “`SessionOrchestrator`” for our flows, just call them sessions or workflows.
- *API and UI coupling:* The UI (E6) will rely on these endpoints. We must ensure the JSON format is suitable. For example, UI might expect certain keys in the evaluation result. We should align with how the UI will parse it. We’ll clarify that in E6, but basically, our choices (like nesting evaluation under session result) should be convenient for UI. We likely will just pass the JSON to UI and render fields as needed.
- *Time and scheduling:* `DailyUpdateSession` presumably runs once a day (perhaps via a cron job or manual trigger). We are exposing it via API for manual trigger (evaluation stage). In production, one might schedule it. This is outside our immediate design but worth noting. We assume for evaluation, the user will hit the button in UI to simulate a daily run.
- *Open questions:* Should `DailyUpdateSession` re-evaluate theses quant models with fresh data (like re-run backtest with extended data)? Probably not necessary daily. Usually, you wouldn’t re-run historical backtest daily; you would track current performance. So we keep daily update to P&L updates and alerts, not full re-evaluation of strategy hypothetical performance. If needed (like monthly strategy review), that can be separate.

- *Concurrency*: If two sessions run at once (two users evaluating different theses simultaneously), could there be issues? Possibly if they share resources (like writing to same table). But likely fine – separate theses, separate data. We’re not heavily addressing concurrency beyond using the DB’s inherent support.
 - *Test complexity*: End-to-end tests can be complex to set up. We mitigate by using the dependency injection in FastAPI to plug in our test doubles. We must ensure our app allows that (by structuring our dependencies as providers). This is somewhat dependent on how the codebase’s FastAPI is configured. We saw mention of dependency overrides in tests earlier phases. We will follow that pattern.
-

E5 – Paper Trading & Execution Adapter

E5-Requirements:

- Provide a mechanism to go from an evaluated thesis to actual simulated trades (“paper trading”). This includes:
 - A **PaperExecutionAdapter** that can generate a concrete **TradePlan** from a thesis (essentially deciding what trades to execute to implement the thesis’s expression) and then “execute” that plan by creating Trade records.
 - The TradePlan should detail the intended portfolio actions: which assets to buy/sell, in what quantity or percentage, and possibly a target allocation amount. It serves as a reviewable plan a PM can approve.
 - When the plan is approved (triggered by an API call), the adapter will create Trade entries in the TradeRepository (with timestamp, asset, action, quantity, price, thesis_ref, etc.), simulating immediate execution at current market prices (since it’s paper trading).
 - Implement any needed logic for execution: e.g., query current prices for each asset at execution time (so that we record the price at which we “bought/sold”).
 - Compute and track basic P&L from these trades. Specifically, implement a **revalue_portfolio(as_of_date)** function (likely integrated in DataAccess or ExecutionAdapter) that given all trades and latest prices, returns a PortfolioSnapshot (which we actually did in E1 DataAccess using trade_repo + price_source). We might refine it to also calculate P&L per trade or overall.
- Ensure that after execution, the portfolio state is reflected in DataAccess.get_current_portfolio (which should now include those new trades in positions).
- Integrate this with the ThesisEvaluationSession: possibly outputting a TradePlan suggestion post-evaluation (so user can see what would be traded if approved). And an API endpoint to approve a plan:
 - POST /api/v1/thesis/{thesis_id}/approve-plan which triggers execution for that thesis’s plan.

- Testing should confirm that approving a plan leads to correct Trade records and that portfolio values update appropriately on subsequent valuations.

E5-Research Summary:

Paper trading is commonly used to test strategies in real-time without financial risk. The key is to simulate as realistically as possible the act of placing orders and updating portfolio holdings . Many trading systems separate the generation of a trade *plan* from execution: first decide **what** to trade (which assets and how much), then perform the trades. We mirror that separation. The plan acts like a limit order list or target allocation. In quant terms, our thesis expression is already a target allocation (weights), so generating a plan essentially means converting those weights into concrete trades given a certain capital base. This is analogous to portfolio optimization outputs where weights must be turned into buy/sell orders.

The PaperExecutionAdapter's job is similar to a broker interface in a backtesting framework: it takes signals (desired positions) and produces fills (executed trades). For simplicity, we assume immediate fill at current market price (no slippage, no delay) – a common assumption in basic backtests. This is fine for evaluation and aligns with the idea of paper trading as an ideal execution environment.

In designing tests for trading logic, it's typical to use scenarios like: *if I buy at price X and later price is Y, P&L is computed correctly*. We will verify that our revaluation logic matches basic arithmetic. Ensuring that after a series of trades, positions are aggregated correctly (sum of quantities per asset, average cost if needed) is crucial. Many frameworks maintain an internal position state; we have the TradeRepository and we will derive positions on the fly, which is fine given small scale.

We draw on basic accounting: **gross exposure** = sum of absolute positions, **net exposure** = sum of signed positions. P&L for a position can be computed as $(\text{current_price} - \text{average_cost}) * \text{quantity}$ for longs (and inverse for shorts). We should confirm our system can derive that if needed. Since DataAccess.get_current_portfolio already calculates current value, we can get portfolio P&L by comparing current total value to the sum of trade costs (initial invested capital). We might explicitly implement a helper for P&L:

- *Unrealized P&L for each open position* = $(\text{current_price} - \text{execution_price}) * \text{quantity}$ (for long; for short, use $(\text{execution_price} - \text{current_price}) * \text{quantity}$ since profit if price fell).

- We might not need to expose per-trade P&L in the API yet, just ensure totals reflect it. However, the UI might show P&L per thesis on the thesis list. That can be derived by summing trades for that thesis.

The approach to plan execution is straightforward: ensure **idempotency** or that repeated approvals are not allowed unintentionally (you wouldn't want to double-buy the same plan). We can handle that by marking a Thesis as "deployed" or checking if trades exist already. But given the evaluation scenario, it's acceptable if a user accidentally approves twice and it doubles positions – but we likely want to prevent that. Perhaps we set Thesis.status = "ACTIVE" after approval and have the endpoint check status != ACTIVE to proceed. This can be a simple safeguard.

E5-Design:

- **PaperExecutionAdapter Class:** (in src/slice/execution/paper.py)

```
class PaperExecutionAdapter:
    def __init__(self, trade_repo: TradeRepository, price_source):
        self.trade_repo = trade_repo
        self.price_source = price_source

    def create_plan_from_thesis(self, thesis: Thesis, total_value: float) ->
TradePlan:
        # Create a TradePlan proposal for this thesis given an intended total
capital to allocate.

    def execute_plan(self, plan: TradePlan, as_of_date: date = None) ->
list[Trade]:
        # Execute the given plan (assumes plan is for a single thesis),
        # fetch prices (as_of_date or current),
        # create Trade objects and persist via trade_repo.
    ...
```

We included `total\_value` in create_plan – presumably the user might specify how much money to deploy. In absence of that, we could default to current portfolio cash or a preset amount. Perhaps simpler: allow `total\_value` to be an optional parameter; if not provided, we use the portfolio's current total * (thesis.expression_legs sum/100) or just a default (like \$100,000).

```
**TradePlan DTO:**
```python
class TradeLeg(BaseModel):
 asset: str
 direction: str # "long" or "short"
 size_pct: float # percent of portfolio to allocate to this asset

class TradePlan(BaseModel):
 thesis_id: str
 total_value: float
```

```
legs: list[TradeLeg]
```

- This basically mirrors Thesis.expression\_legs plus a notion of total capital. The plan is essentially a restatement of the thesis allocation with a dollar amount. We may also add convenience fields like an expected portfolio after execution, but not necessary.
  - create\_plan\_from\_thesis:
    - Validate thesis legs sum  $\leq 100$  and thesis looks executable (similar to what evaluation did).
    - Decide total\_value. If the user hasn't provided one, perhaps use `get_current_portfolio().totals.portfolio_value * (some fraction)`. But that assumes there's free cash. We might for evaluation simply take the whole current portfolio value or a default constant. Let's say if total\_value param is None, we use \$10000 for testing. (We will note this assumption.)
    - Construct a TradePlan with same legs as thesis. We don't yet convert to specific quantities here, because quantity depends on price at execution time. TradePlan remains weight-based (size\_pct).
  - execute\_plan(plan, as\_of\_date):
    - For each leg in plan, get the current price of leg.asset. Use as\_of\_date if provided (in daily update, might execute at market close of a given date; usually, as\_of\_date = today).
    - Determine quantity:  $\text{quantity} = (\text{plan.total\_value} * \text{leg.size\_pct}/100) / \text{price}$ .
      - If direction == "long", action = "BUY" and quantity as computed (rounded maybe to 2 decimals or whole if we consider lot sizes; but not needed for simulation).
      - If direction == "short", action = "SELL" and quantity computed similarly. This implies opening a short position of that many units.
    - Create a Trade object for each leg:

```
trade = Trade(trade_id=uuid4(), timestamp=as_of_date or datetime.now(),
asset=leg.asset,
 action="BUY" or "SELL", quantity=quantity, price=current_price,
 type="PAPER", thesis_ref=plan.thesis_id, notes=None)
```

- - - (The Trade model has a type field – we'll use "PAPER" to mark simulated trades.)
    - Insert each trade via TradeRepository.insert.
    - Return the list of Trade objects (or we could return nothing and rely on DB, but for tests we'll want to inspect).
  - Possibly, mark the Thesis as Active/Deployed: We might update thesis.status = "ACTIVE" (if status field exists and supports that) or set a flag indicating it's in the portfolio. This can be done via ThesisRepository update if needed. This would prevent re-approving the same thesis. Implementation: either check if `trade_repo.list_by_thesis(thesis_id)` already has trades, and if so, decide not to

duplicate. For evaluation, we may not worry unless explicitly asked, but it's a good safeguard to mention.

- **Integration in E4/E5 flows:**

- In ThesisEvaluationSessionResult, we included trade\_plan. We implement that by calling PaperExecutionAdapter.create\_plan\_from\_thesis(thesis, total\_value) at the end of evaluation (but **do not execute it yet**). The total\_value could default to portfolio\_value or a configured per-thesis capital. We'll likely just use a default (say \$10000) for demonstration. This plan goes to the client for review.
- Add API endpoint:

```
@router.post("/thesis/{thesis_id}/approve-plan")
def approve_plan(thesis_id: str, request: ApprovePlanRequest):
 # ApprovePlanRequest might contain total_value if user can override, or
 # could be empty
 session = PaperExecutionAdapter(trade_repo=get_trade_repo_dep(),
 price_source=get_price_source_dep())
 thesis = data_access.get_thesis(thesis_id)
 if not thesis: raise 404
 # Optionally, if thesis already has trades (active), we might return 400
 # to prevent duplicate.
 plan = session.create_plan_from_thesis(thesis,
 total_value=request.total_value or default)
 trades = session.execute_plan(plan)
 return {"status": "ok", "trades": [t.dict() for t in trades]}
```

- - We might not expose the plan from client side if it's always the same as thesis legs. Possibly the client just calls without any body and we use the plan we already suggested. But for decoupling, I used ApprovePlanRequest (which could include e.g. a confirmation flag).

The response returns the executed trades or simply a status. We might show trades to confirm execution.

- After execution, the trade repository has new trades, and DataAccess.get\_current\_portfolio will now include them. That means if the user goes to portfolio UI or runs daily update, they see the effect.

- **P&L Calculation:**

- DataAccess.get\_current\_portfolio already calculates current portfolio\_value using current prices (we did that in E1).
- If we want explicit P&L, we can compute it as portfolio\_value - sum(trade\_values\_at\_execution). We can derive sum(trade\_values\_at\_execution) by summing quantity\*price for all buys (adds to cost) and the same for sells (which effectively adds cash, i.e., subtract from cost). In other words, maintain a running "cash" or net investment. Alternatively, each Trade could carry a direction or sign to indicate if it's an inflow or outflow of cash.
- A simpler approach: interpret net\_exposure from DataAccess as net invested capital. E.g., if we have only buys, net\_exposure = portfolio\_value, which equals cost plus P&L (here cost = net exposure at inception). If we have shorts,

net\_exposure might be lower than gross. For clarity, maybe we add in PortfolioTotals a unrealized\_pnl field. We can compute it by:

- Sum for each asset: (current\_price - avg\_cost\_for\_asset) \* quantity.
- To get avg\_cost: we can compute average cost per asset from trades (weighted by quantity).
- For short positions, avg cost concept still applies (price at which short was sold).

This might be too detailed; given the evaluation scope, we may skip explicit P&L field and let PM infer from total vs initial or see trade-by-trade.

- For UI, they requested to display P&L on thesis list. This likely means per thesis P&L = sum of P&L of trades linked to that thesis. We can add a method in DataAccess like get\_thesis\_performance(thesis\_id) to compute that. Or incorporate in portfolio\_depth's thesis\_exposures: e.g., return not just current value but also P&L. But not specified originally, so we might not add new fields, just note that P&L can be derived if needed.

## E5-Testing Plan:

- *TradePlan generation test:*
  1. Create a Thesis object with legs, e.g., 60% asset X (long), 40% asset Y (long).
  2. Call create\_plan\_from\_thesis(thesis, total\_value=10000).
  3. Verify the returned TradePlan has thesis\_id matching, total\_value=10000, and legs corresponding to thesis legs (60% and 40%). Essentially ensure it's copying properly and not altering percentages.
  4. Try with a short leg: Thesis with 100% short X. Plan should reflect direction "short". Confirm that.
  5. Edge: if thesis had legs summing less than 100 (say 50% cash implicit), plan should just include given legs and presumably leave 50% unallocated. That's fine; we don't explicitly represent cash in TradePlan. But maybe we should note that leftover 50% would remain cash (which in paper trading we currently don't track as a position; we implicitly assume it just means not all capital used).
  6. If thesis sum >100, create\_plan should probably raise an error. Test that: make a thesis 120% allocation, expect a ValueError or similar. (This is similar to evaluation validation.)
- *Plan execution test (no existing trades scenario):*
  1. Use the TradePlan from above (60/40, total 10000).
  2. Prepare price\_source stub: for asset X and Y, provide current prices (e.g., X=50, Y=200).
  3. Call execute\_plan(plan, as\_of\_date=today).

4. Verify it returns a list of Trade objects (or the method might not return but let's say it does).
  - There should be 2 trades: one BUY X, one BUY Y.
  - Check quantities: For X: 60% of 10000 = \$6000 allocated, at \$50 price => quantity ~120. For Y: \$4000 at \$200 price => quantity 20.
  - Minor rounding differences can occur; we may allow small tolerance (or design to truncate or round to 2 decimals). For test, maybe we assume no rounding or one decimal.
  - Check Trade fields: asset names correct, action "BUY", price equal to the stub price, thesis\_ref = thesis\_id, type="PAPER".
  - Confirm they were persisted: call trade\_repo.list\_by\_thesis(thesis\_id) and ensure these trades appear. Or directly check an in-memory list if our TradeRepository in test is a fake. If using a real DB, we query it.
  - Confirm timestamp set to today (the given as\_of\_date).
5. Now call DataAccess.get\_current\_portfolio(). It should pick up these trades:
  - positions: X with ~120 qty, Y with 20 qty, etc.
  - totals: portfolio\_value = 6000 + 4000 = \$10000 (assuming prices unchanged right after trade), net\_exposure=10000 (all long), gross\_exposure=10000.
  - If we had any prior portfolio value or trades, ensure it's adding correctly. But here it's first time, so straightforward.
6. *Approve endpoint test*: Simulate the full user action:
  - This requires integrating with API: first run the evaluation (or at least ensure a thesis exists).
  - Then call POST /api/v1/thesis/{id}/approve-plan.
  - We can modify our dependency injection so that PaperExecutionAdapter in that route uses the same test price\_source and DB.
  - The response should be {"status": "ok", "trades": [...]} as per our design. Check that trade count and details in response match expectation.
  - After that call, call GET /ui/portfolio (or relevant route if any) or directly DataAccess to verify portfolio updated.
  - Also, potentially check that thesis status changed if we implemented that (if we updated Thesis.status to 'ACTIVE', verify it).
  - If the endpoint is supposed to prevent re-execution, call it second time and expect maybe a 400 or no new trades. If we didn't implement a check, second call would duplicate trades (which test could catch by seeing trade count doubled). This is not ideal but if we didn't implement safeguard, we document it as limitation. We can implement a simple check: if TradeRepository.list\_by\_thesis not empty, return 400 "Already executed". Test accordingly if implemented.
- *P&L computation test*:
  1. After executing trades, simulate market move:
    - E.g., asset X price goes from 50 to 60, Y from 200 to 180.
    - Use price\_source such that DataAccess now sees those new prices.
    - Call get\_current\_portfolio again:

- New portfolio value =  $12060 + 20180 = 7200 + 3600 = \$10800$ .
  - The initial invested was \$10000, so unrealized P&L = +800 (8%).
  - We check if our system can reflect this: portfolio\_value is now 10800. We might not explicitly list P&L, but the increase indicates it.
  - If we implement an explicit P&L field or method, we test it: e.g., if DataAccess had get\_portfolio\_performance() returning current value and initial cost, verify it yields 10000 and 10800 appropriately.
  - We can also test get\_portfolio\_depth().thesis\_exposures: it might say {thesis\_id: 10800} as current value. If we had multiple theses, it would distribute accordingly. Good to ensure it sums up to total.
2. If short trades are executed, test P&L logic in that scenario:
- E.g., plan with 100% short X, executed at 50, quantity 200 short for \$10000. If price falls to 40, portfolio value from that position is -  $(200 * 40) = -\$8000$  (since it's a liability of buying back 200 at 40 would cost \$8000; we had \$10000 from initial sale, so net equity = \$2000 profit).
  - DataAccess would show portfolio\_value = -\$8000 (just positions). This is where our earlier discussion surfaces: it doesn't include the cash from the short sale. If we want net equity, we should add cash \$10000 to that, equaling \$2000 net. Our DataAccess currently would not include cash, so it would understate net equity for short positions.
  - This is a limitation; we note it. We could adjust get\_current\_portfolio to include cash = sum(sell trades) - sum(buy trades). That's a possible enhancement: track cash as position or in totals.
  - For evaluation, perhaps avoid heavy short scenarios or simply interpret negative portfolio\_value as having that much cash externally. We might mention this edge and not deeply fix it unless we choose to track a 'cash' position.
  - In testing, we can either skip scenario with only short for portfolio (because of missing cash tracking), or implement a quick fix: treat net\_exposure differently. Possibly:
    - When calculating positions, also calculate cash = sum(sell trades value) - sum(buy trades value) and then net equity = cash + sum(position values). For long-only, cash = -sum(buy cost) (a negative number representing money spent), and position values positive, net = position values + (-cost) = P&L (which starts 0). For short, cash positive, positions negative, net adds up.
    - This might be too much to implement last-minute, but it's doable. If we implement, we add a synthetic "Cash" Position in PortfolioSnapshot if net cash is nonzero.
  - If we do implement, test it: short scenario yields a Cash position of \$10000, a short X position -\$8000, so totals: portfolio\_value = \$2000 net, gross\_exposure = \$10000 (abs of short value, ignoring cash?), net\_exposure maybe \$2000. This is complex to get exactly right; likely we avoid for now and just caution that short handling is simplified.

- *End-to-end user scenario test:* Combine E4 and E5:
  - Simulate full flow:
    - Call evaluate for a thesis (as in E4 test, get plan in response).
    - Then call approve endpoint.
    - Then call daily-update:
      - The daily-update should now consider the trades from approve. Portfolio snapshot should no longer be empty. If an observation triggers an alert, and summary runs, etc. This ties everything: a thesis created, evaluated, executed, portfolio updated, then daily update runs on that portfolio.
    - We verify at each stage the system's outputs are consistent (no missing fields, correct values).

## E5-Risks & Dependencies:

- *Financial realism:* Our simulation ignores transaction costs, liquidity, etc. It's fine for evaluation. But one notable issue: the lack of cash tracking for shorts as discussed. This can lead to confusing portfolio values if shorting. Given the typical use-case might focus on long or small short exposures, and our evaluation environment tolerance, we document the limitation but do not fully implement cash unless there's time. It's an assumption: "no leverage constraints or cash tracking – effectively assuming infinite margin."
- *Partial execution:* We assume full execution at given prices. In reality, if the portfolio had existing positions, a plan might net out some trades (e.g., if already 5% in an asset and target 10%, you only buy 5% more). Our current plan doesn't account for existing holdings. If a thesis is newly being implemented, that's fine (no existing trades for it). If we allow rebalancing a thesis, we'd need to incorporate current position to avoid doubling. Out of scope now, but we should note: our approve is mainly for new theses.
- *Double approval:* As mentioned, we might implement a guard to not approve twice, or allow it if user intentionally adds more. To be safe, we'll likely guard. This depends on setting or checking a thesis status or existing trades. If our TradeRepository is easy to query, simply do:

```
if trade_repo.list_by_thesis(thesis_id):
 raise Exception("Thesis already has trades, cannot approve again")
```

- in approve handler. That solves it simply. We include that and test it.
- *Atomic execution:* If inserting one trade fails (e.g., DB error on one), we might end up with partial portfolio. Probability low since these are simple inserts. If using SQLAlchemy, we can do it in a transaction. Our test DB likely uses autocommit per insert. We assume it works, but if a failure happens mid-way, some trades may be saved. This is a minor risk; in a robust system, you'd wrap in a transaction. Not doing that explicitly due to time, but mention it.
- *Time synchronization:* We should ensure that when executing with as\_of\_date, the price we get corresponds to that date's close. If using a daily price series, our price\_source



might either give last available price if date is beyond range. We need to be careful in tests to use the right date that we have data for. Possibly just use None to get current price which our stub can define as latest. It's fine as long as it's consistent.

- *Integration with UI:* After plan approval, the UI “Portfolio” view should reflect new positions. Also, the thesis detail could update status to show it's active and possibly show P&L. We'll ensure the data is there (e.g., DataAccess can fetch current positions by thesis for UI if needed). Possibly UI will just use portfolio data for that.
- *Backtest vs paper drift:* The evaluation results are based on historical data up to now. Once we start paper trading forward, the thesis will have a live P&L which can diverge from theoretical. This is expected and indeed why we do paper trading. The system should allow comparing evaluation (what was expected historically) vs actual going forward. Right now we don't explicitly connect these two beyond referencing the thesis. That's fine; a PM can manually compare evaluation metrics (like expected vol) to realized.
- *Scalability:* For each trade execution, we query price for each asset. If many assets, that's fine. Trade count is equal to number of legs in thesis (small, typically < 10). No issue. If many theses, we handle one at a time anyway for approval.
- *Dependencies:* PriceSource is key. If price not available at execution time for an asset (maybe market closed?), we either pick last price or fail. Simulate stable environment in tests (price stub always returns something).
- *Test environment DB persistence:* Must ensure tests that run approval endpoint see the trades in the same test transaction or connection. This depends on how we manage DB in testing. Likely, each test uses a fresh SQLite or the same Postgres but in a transaction. We should commit trades so subsequent queries see them. If using an ORM session, ensure session handling doesn't hide uncommitted data. Possibly easier: for test, use an in-memory repository stub to track trades rather than a real DB (because coordinating FastAPI test client DB state can be tricky). We might employ a FastAPI override of TradeRepository with an in-memory list for tests to assert on easily. This is doable with dependency override if we register TradeRepository as a dependency. We need to carefully hook dependencies for all (DataAccess uses repos, ExecutionAdapter uses repo, etc.). Alternatively, use a real DB with known state and query directly with the same session as the app. For comprehensive integration, might accept using the real test DB.

---

## E6 – UI & Evaluation Dashboard (“Use Stage”)

### E6-Requirements:

- Develop a simple UI (Streamlit or similar web dashboard) that surfaces the entire pipeline to the end-user (PM). The UI should provide:
  - **Thesis List Page:** Lists all theses in the system with key info and actions.  
Columns: Title, Status (e.g., Draft, Active), Last Evaluated date, Current P&L or

performance if active, and any risk flags. Each row should have an “Evaluate” button to run E4 on that thesis, and an “Open” or “View” button to go to detail.

- **Thesis Detail Page:** For a selected thesis, display:
  - Thesis content (title, hypothesis, drivers, disconfirmers, expression legs).
  - Latest **Quant Evaluation** results (performance metrics like total\_return, sharpe, etc., perhaps in a small table, and scenario outcomes).
  - Latest **LLM Review** (critique text, list of questions, risk flags – format nicely, e.g., bullets for questions and flags).
  - If a TradePlan is available (after evaluation), show its summary (e.g., “Proposed allocation: 60% X, 40% Y on \$10000”).
  - Actions: a “Run Evaluation” button (if wanting to re-run) and if a trade plan is present and not yet executed, an “Approve Plan” button.
  - If the thesis is already active (plan executed), indicate so (maybe disable the approve button or show “Active since [date]”).
- **Portfolio View Page:** Shows the current portfolio (aggregation of all executed trades):
  - A table of current positions: Asset, Quantity, Current Value, P&L (if we calculate it).
  - Totals: total portfolio value, gross exposure, net exposure (perhaps as a small summary).
  - Basic risk info: e.g., largest position as % (we can highlight the top position), maybe any simple factor if available (this might be trivial, but e.g., if we had a breakdown by asset class or currency).
  - Possibly a time-series graph of portfolio value over time if we store history (we did not implement storing daily equity, except through backtest; but since go-live, we could accumulate daily values in a list. Simpler: skip graph in this eval version).
- **Alerts & Runs Page:**
  - A section listing any Alerts generated by DailyUpdate. For example, a list of messages: “[Date]: Thesis X – disconfirming signal: [some observation or flag]”. Possibly with a link to thesis detail.
  - A section showing recent daily update runs: could be a table with Date, status (Success/Fail), and maybe runtime in ms. This info comes from what we instrumented (we could log each daily run outcome with timestamp). Since we have diagnostics metrics and can infer when daily ran by alerts or summary timestamp, we can fill this manually: e.g., show the last run date and latency (we can store last daily run latency as part of diagnostics).
  - The page might also have a button “Run Daily Update Now” to manually trigger /session/daily-update (for testing convenience).
- **Observations & Q&A Page:**
  - Display recent observations (maybe a list of the last N observations from DB, with timestamp and maybe a snippet of text) so the PM can see incoming data.
  - Provide a text input field for the user to ask an intuition question, e.g., “What is the outlook for inflation?” along with a dropdown or multi-select

- to pick relevant theses or categories if needed (or it could just use all observations or a filter).
  - On submitting the question, call our `llm_query_intuition` via an API endpoint (we may expose `POST /api/v1/intel/qa` similar to existing code). Then display the answer from `IntuitionAnswer` (the answer text and possibly the list of references, which we could show as footnotes or clickable to see the observation content).
  - Also possibly allow selecting a specific thesis to filter observations (for example, if user only wants intuition from observations related to a particular thesis).
- The UI should be minimal in styling but **clear and logically laid out**, per user's emphasis on readability. Use headings, subheadings, and bullet points to avoid big text walls. Ensure that when embedding images or figures (like maybe a logo or chart), we follow image citation rules (though likely we won't embed external images in this context).
- The UI will use our API endpoints to fetch data or trigger actions. Since Streamlit runs Python backend, it might call `DataAccess` directly instead of via HTTP, but given separation, it might be cleaner to call the FastAPI endpoints (or directly use the underlying functions since it's same codebase).
- The "Use Stage" scenario in the prompt should be fully supported by the UI: that scenario described creating/loading a thesis, running evaluation, approving trade, running daily update, and seeing results. We need to ensure each of those steps is doable via the UI components designed above.

## E6-Research Summary:

In designing dashboards, especially for financial data, it's crucial to highlight key metrics and keep the interface uncluttered. The UI should let a PM glance at what's important (like which thesis is performing well or which has risk flags) without wading through raw data. We apply dashboard design best practices like creating clear visual hierarchies and making critical insights immediately visible. For example, on the thesis list, we might use colored icons or badges to indicate if a thesis has a risk flag from LLM (red icon) or if an alert was triggered recently. However, given text-based format, we might simply list flags as text.

The principle of *the 5-second rule* in dashboards is that a viewer should grasp the main points within a few seconds. To that end, we'll ensure totals and P&L are prominently placed, and that tabular data is formatted nicely (e.g., numbers with two decimals, percentages with % sign). We'll use short paragraphs or bulleted lists for any textual explanation (like listing risk flags or LLM questions) to improve readability.

We also consider user flow: The scenario suggests a user might iterate on a thesis (re-evaluate after making changes). Our UI allows re-running evaluation easily. Also, after running daily updates, the user can inspect what's changed (via alerts and summary).

Streamlit is our chosen tool (as mentioned). It allows easy creation of interactive components. We have to be careful to maintain state if needed (e.g., after evaluation, holding the returned results to display without requiring immediate DB refresh). But we could also fetch from DB after operations.

Observability in UI: We included an Alerts & Runs page to surface some operational info (like run durations from diagnostics). This provides transparency (like if LLM calls are slow or failing often, PM might see in metrics or logs). In a real ops dashboard, one might show charts of latency, but that's beyond our textual interface. A simple text summary of metrics from diagnostics/llm (like "LLM calls: review: avg 1.2s, 0 errors") could be included perhaps at bottom of the page or in a separate admin section. Given it's asked to expose metrics, we will include at least in the Runs page or a separate "Diagnostics" page some of those numbers.

## E6-Design:

- **UI Architecture:** We have a Streamlit app (likely a `ui/app.py`) that defines multiple pages or tabs:
  - We can use `st.tabs(["Theses", "Portfolio", "Alerts & Daily Runs", "Q&A"])` to create four sections in one app, or use Streamlit's multipage feature. The prompt suggests a dashboard with sections, so tabs make sense for quick switch.
- **Thesis List Implementation:**
  - Use `DataAccess` (or an API call `/ui/theses` if exists; codebase had some UI routes for contexts but not one for listing theses, we can either create an API route or directly use `DataAccess` in Streamlit since we are in the same deployment).
  - For each thesis from `DataAccess.list_all_theses`, display a row. Possibly use `st.table` for static table or manual using columns:
    - Show Title (clickable: Streamlit doesn't have hyperlinks easily for internal navigation, we might instead have a selection box or button).
    - Status: if `thesis.status` exists (the code had an enum likely), display it. If we implemented active/draft logic, reflect that.
    - Last Evaluated: we can store last evaluation time in Thesis (if not, our `EvaluationRepository` could record a timestamp and we could retrieve latest eval for each thesis to show date). If that's too complex now, we may leave blank or show "Never" if no record. Alternatively, our `EvaluationRepository` could give us last eval time for each (we can add that easily if we stored evaluations with timestamp).

- P&L: if thesis is active (has trades), compute current P&L = current value - cost. DataAccess doesn't give directly per-thesis P&L; we could derive by summing trades from TradeRepo for that thesis. Possibly simply show current thesis exposure from portfolio\_depth minus initial notional. If we know initial notional (like in TradePlan total or sum of trade costs), we can compute. For demonstration, even showing current value is good. We can display as e.g. "\$10,800 (+8%)" format if we compute percent.
- Flags: If LLM review had any risk\_flags not empty, show an icon or list of those flags here to catch attention. For example, if risk\_flags = ["liquidity risk"], put an emoji or "⚠ liquidity risk". If insufficient\_context was True on review, maybe mark that (meaning review not reliable).
- Provide an "Evaluate" button in each row. Streamlit can create a button with a key per thesis (like if st.button("Evaluate", key=thesis.id): ...). When clicked, it will trigger evaluation. We need to handle that call and update UI. E.g., call API or directly call ThesisEvaluationSession. Possibly simpler to call API and wait for response.
- Provide a "Open" button to jump to detail. In Streamlit, we can implement this by setting a session state variable selected\_thesis = id and then switching tab or rerendering the detail section. Or we could just always display detail below when one is selected. Simpler: have the Title itself be clickable. Streamlit doesn't support hyperlink to another tab well; perhaps we use st.selectbox to choose a thesis to view.
- Another approach: the list page could allow selecting a thesis from a dropdown or clicking a row, and on selection, display the detail info on the same page or navigate to detail page. However, since we plan multiple tabs, maybe use separate tab for detail. But we weren't told separate tab, they just said "Open -> detail". It might be simpler to have the Theses tab show either the list or, if one is selected, the detail below.
- For clarity, maybe break it: Theses tab: left half list, right half detail of selected. But given limited time, a simpler flow: choose from list, then detail view appears (embedding one page in another isn't straightforward in Streamlit, but using st.expander or conditional logic can simulate it).
- **Thesis Detail Implementation:**
  - Triggered by selection (from list or separate function if we do multipage).
  - Once we know which thesis to show (by ID), we retrieve it (DataAccess.get\_thesis) and related data:
    - Get last evaluation result and review: We can call EvaluationRepository.get\_last(thesis\_id) or an API if we made one. Alternatively, store the result in session state when evaluation was run and reuse if user just ran it. If not just ran now, we fetch from DB if available. Since we persisted evaluation and review, we can fetch them.
    - Display thesis info: st.markdown or st.write for hypothesis, drivers, etc. Use bullet points if drivers/disconfirmers are lists.
    - Display quant metrics: we can create a small table of metrics: e.g.,

Metric	Value
-----	-----

Total Return | 10%  
CAGR | 7.5%  
Volatility | 5%  
Sharpe | 1.5  
Max Drawdown | 8%

- - 
  - Using `st.table` with a pandas DataFrame or `st.metric` components for a few key ones (Streamlit has `st.metric(label, value, delta)` which can nicely show e.g., “Total Return: 10%, delta vs something” but delta not needed here).
  - Show scenario outcomes if available: maybe a `st.write("Scenario impacts:")` followed by a small dataframe of scenario name and pnl%.
  - Display LLM review content:
    - The critique text as a `st.markdown` (maybe italic or blockquote).
    - The questions as a bulleted list: e.g., `st.markdown("* Questions the model posed:")` then loop for `q` in `review.questions`:  
`st.markdown(f"- {q}")`.
    - Risk flags similarly.
    - If `insufficient_context` true, maybe highlight: “(Review could not be generated due to insufficient context)”.
  - If `trade_plan` exists (and thesis not active yet): show “Proposed Trade Plan:” and list legs (like “Buy 120 shares of X, 20 shares of Y (allocating \$10k total)”).
    - Actually, `create_plan` didn’t compute quantity; we only compute at execution. So maybe better: For the plan, we can compute approximate quantities with current price just for display. Or display in terms of percent and total. e.g., “60% X, 40% Y of \$10,000”.
  - Provide Run Evaluation button (which triggers evaluation function again and refreshes these results on UI).
  - Provide Approve Plan button if plan exists and not executed. If pressed, call `approve` API and then:
    - Possibly immediately mark thesis as Active in UI and reflect that trades executed. We could then prompt the user to check portfolio tab.
    - Or update the detail page to show “Plan executed on [date].”
  - If thesis is active (detect by if any trades exist via `trade_repo` or maybe `Thesis.status`): then instead of approve button, maybe show “Thesis is active with \$X invested. Current P&L = ...”. And possibly allow adding more or rebalancing, but that’s out of scope.
  - Error feedback: if evaluation or approval fails, show an error message (Streamlit can display via `st.error`). E.g., if API returns 500, show error detail.
- **Portfolio View Implementation:**
  - Use `DataAccess.get_current_portfolio` to get positions and totals.

- Display totals on top: e.g., `st.metric("Portfolio Value", f"${value:,.0f}")` and maybe another metric for net or day change if we had it. Since we don't store daily changes explicitly, maybe skip daily change. Could show net vs initial if we had that (not easily, initial total would be sum of all buy trades, which we can compute from trades).
- Present positions in a table: Asset | Quantity | Current Price | Value | % of Portfolio | P&L (if computed).
  - Compute % =  $\text{value} / \text{portfolio\_value}$ .
  - P&L for each: if we compute average cost:

we can compute cost = sum of trades for that asset (buy positive, sell negative). Then current value - cost = P&L.

Alternatively, if we want approximate P&L, we could cheat by using RiskSnapshot exposures or scenario if we had mark-to-market. But let's do directly:

get all trades, for each asset:

- `total_quantity, total_cost = sum(quantity), sum(quantity * trade_price * sign)` where sign = +1 for buy, -1 for sell maybe. Actually, for cost of asset held:

if buys exceed sells, net positive quantity with some net cost. For short, net\_quantity negative, and total\_cost negative (which means cash gained).

- $\text{average cost} = \text{total\_cost} / \text{total\_quantity}$  (taking care with sign).

This might be too much to implement in UI. Instead, possibly we lean on the portfolio depth or DataAccess extension to provide that.

- Or, simpler: we can retrieve all Trade objects via `TradeRepository.list_all` and do the math in the UI code since we have them. It's fine for small data. We'll do that.
- Use `st.dataframe` or `st.table` for positions.
- Risk metrics: if `get_portfolio_depth` provides concentration etc., we can list a couple like top asset concentration. If `depth.concentration` dict exists, take the max item and display "Top position: Asset X = 50% of portfolio".
- If any factor exposures given (we likely have none meaningful), skip or show stub.
- Also if `depth.thesis_exposures` available, we might show how portfolio is split among theses. That could be a pie chart or just text (e.g., Thesis A 60%, Thesis B 40%). A pie would be nice; Streamlit can do simple Altair or Plotly charts. But

text is okay in a pinch. Maybe `st.write("By thesis allocation: ...")` with listing percentages.

- **Alerts & Runs Page Implementation:**

- Alerts: fetch recent alerts. If we have `AlertRepository` or via `DataAccess` (if we added something), we can list all alerts sorted by date desc, or last N. Display each as:
  - date, thesis title, message (which might already include thesis name or observation snippet).
  - Possibly use `st.warning` or colored text for alerts to stand out.
  - Could also provide a button or link to mark as acknowledged or to view the thesis.
- Runs: We can fetch some log of daily updates. If we instrumented daily update runs, maybe store last run time and status in a global variable or DB. We did not explicitly store runs aside from metrics. But we could use the LLM diagnostics metrics as proxy for how many daily summary calls (should equal runs count). Alternatively, we can create a simple log: each `DailyUpdateSession` when run could append to an in-memory list or a file in `/data`. Simpler: we use a global list `daily_runs_log` where we append (date, success, latency).
  - We can expose that to UI by either reading from an API (we didn't create one, except maybe our metrics can tell number of daily runs by checking `daily_summary` calls). Not robust.
  - For demonstration, we might just rely on the metrics for LLM daily summary: if calls >0, assume those correspond to runs. We could then say "Last daily update: today at X time, took Y ms." This is a bit hacky. Perhaps in `DailyUpdateSession.run`, we set a global `last_daily_run_time` and `last_daily_run_duration`.
  - We'll do a straightforward solution: maintain `last_daily_run = {"time": ..., "duration": ..., "status": ...}` in module-level variable when daily-update endpoint is called. Tests can verify it's set. Then UI reads that.
  - The Runs page can then display that one (or if we had a list, could show a few).
- Provide "Run Daily Update" button on this page. If clicked, call the daily-update API. Show a spinner during call (Streamlit has `st.spinner` context). Then refresh alerts and runs info. This allows the PM to manually trigger an update (for testing out-of-schedule).
- Also on this page or separate, show LLM diagnostics maybe:
  - We could include a section "LLM Usage Stats" that calls `/api/v1/diagnostics/llm` and prints each tool's call count and avg latency. This helps see if any tool is being very slow or error-prone (in eval environment, interesting to track).
  - Format: e.g., "ThesisReview: 5 calls, 0 errors, avg latency 1.2s; Intuition QA: 2 calls, 0 errors, avg 0.8s; ..." etc. Maybe use `st.write` or `st.json` for raw output.

- **Observations & Q&A Page Implementation:**

- List observations: call `ObservationRepository.list_recent` or `DataAccess.get_recent_observations(10)`. For each obs, display perhaps: date –



short text snippet (first 100 chars). If obs has categories or link to thesis, show that too (like tag the related thesis titles if any). We can use `st.markdown` with bold date and normal text. Possibly inside an `st.expander(obs.title)` to allow reading full text.

- Q&A input: `st.text_input("Ask a question about the observations:")` and a button or use `st.form` to handle submission.
- On submit:
  - Call `llm_query_intuition` via API or directly (since it's pure function we could call the function if we can easily build context here). But better to hit API for consistency, plus it uses orchestrator/LLM.
  - Show a spinner while waiting (LLM could take a couple seconds).
  - Display answer: `st.markdown(answer.answer)`.
  - Display references: list the referenced obs IDs with maybe a tooltip or expander showing the observation content. We can find those obs from the earlier list (or fetch by id if not in list).
  - If `insufficient_context` came back, display a message "The assistant could not answer with the given information." Or some user-friendly phrasing.
- Optionally allow filtering observations by thesis or category:
  - We could have a multiselect of thesis titles to filter obs list and also pass those to the query (like context limited to selected thesis's obs). For now, maybe not necessary unless observation list is very large. We'll assume it's manageable to just use recent all.
- We ensure the prompt to model only includes provided obs, which our API already does.
- **General UI notes:**
  - Use appropriate headings: Each tab can have a main header (H1) like "# Theses", "# Portfolio" etc., making it clear.
  - Use subheaders (H2 or H3) within for sections (like on detail page: "Evaluation Results", "LLM Review").
  - Keep paragraphs short, use bullet lists for lists of items (we already plan to do that).
  - The UI should not mention any internal things like "error 500" without explanation – we'll catch exceptions and show friendly messages.

## E6-Testing Plan:

- Since UI is mostly manual, we will describe a use-case script (the Use Stage scenario). Automated testing of Streamlit is non-trivial; we rely on integration tests of APIs and manually verifying UI logic through that scenario.
- However, we can simulate some UI function calls:
  - We can instantiate `DataAccess` and repositories with test data and directly call our UI functions (like if we implemented UI logic in functions we could call them). But Streamlit requires an event loop context typically. Instead, we trust manual inspection.

- The Use Stage script requested will effectively be our test: we'll outline how one would use the UI and what they should see, step by step (this will be in the final output in section C).
- We might do a final integration test with everything:
  - Create a thesis, do evaluation (via API or direct call in test), approve, daily update, and then simulate UI retrieving data after each step to ensure consistency. But much of that we already tested in earlier phases.
  - The main addition is verifying that after all actions, the expected values are in place:
    - The thesis status is Active.
    - Trade exists and portfolio snapshot is correct.
    - Alert is generated if applicable.
    - LLM responses were stored and can be retrieved.
    - Q&A can retrieve observations and produce an answer (this we tested in E3 partially).
  - We may not code automated tests for UI due to complexity, but rely on the scenario demonstration to validate the flow.

## E6-Risks & Dependencies:

- *UI vs API Direct:* We have to decide if UI calls the FastAPI endpoints or uses the same internal classes. Since Streamlit is running in the same server environment as FastAPI (presumably), it might be simpler to import DataAccess, etc., and call directly to avoid overhead and complexity of making HTTP calls to itself. But to mimic real client usage, calling the API endpoints via requests might be more realistic. However, running requests against the local server in the same process might not work well (port conflicts). A compromise: in development, treat Streamlit as separate client that uses the REST API (maybe launched separately). But in our integrated environment, direct calls are fine for now. We'll note that ideally UI uses the API, but we may implement direct calls to saved objects for simplicity in this evaluation harness.
- *State management:* Streamlit runs script from top on any interaction, so we need to use `st.session_state` to keep track of things like selected thesis or last evaluation results so that after a button press, we still know what to display. We will implement minimal state usage (like `session_state['selected_thesis']` and maybe store last eval result to avoid re-fetch).
- *Concurrent usage:* Not a big concern, single user PM scenario.
- *Data refresh:* If a user runs evaluation, the UI should update automatically with the result (since our code after calling API can update `session_state` with the returned data and then we render it). Similarly for approve (update status).
- *Visual clarity:* We don't have a front-end styling beyond default Streamlit. We rely on layout and text. Possibly could use a bit of markdown for emphasis (like **bold** for labels).
- *Large data:* Not anticipated (maybe at most a dozen theses, few observations). Our UI is not optimized for large scale, but fine for evaluation.

- *User input errors:* We have limited inputs (question text). We handle insufficient context gracefully. If user input were required for something else (like adding a thesis), currently our UI does not cover thesis creation. The scenario implies PM can create or load a thesis. If needed, we could allow adding a thesis in UI (a small form). However, that might be outside scope (the plan didn't explicitly say to implement adding new theses, but it said "use stage where PM can create or load a thesis"). We likely assume some theses pre-exist from earlier in pipeline. If needed, we could quickly allow adding a new thesis in UI for completeness:
  - A form to input title, hypothesis, etc., and a simplified legs input (like text fields for asset and percent).
  - This would call `ThesisRepository.insert` and refresh list.
  - But due to complexity of multiple fields, might skip in this iteration, and just load pre-seeded theses or direct DB insert via a script or code. We'll mention how to create in scenario maybe by using an existing method or `psql`.
- *Documentation:* The UI itself can serve as a demo, but we should produce a *Use Stage script* (section C) describing how to go through it. That script effectively verifies UI meets requirements by describing the interactions and expected outcomes.
- *Checklist matching:* We will have an Implementation Checklist in section D verifying each component exists which indirectly covers UI components (like "Thesis list view", "Evaluate button triggers API", etc.).
- *Session alignment with evaluation plan:* The use stage scenario likely expects that at the end, they see everything working: after daily update, an alert was shown and a summary produced. We must ensure our design indeed triggers an alert in that scenario. So we should calibrate scenario such that one thesis has a disconfirming condition and we simulate an observation to trigger it. For instance, scenario A: Gold thesis has disconfirmers if inflation rises; we ensure an observation "Inflation rose" is present and daily update will catch it. We should set that up or mention to do so. Possibly in our script, we explicitly add an observation then run daily update.
- *User satisfaction:* Since the user explicitly requested a comprehensive blueprint with all these details, having a functioning UI closes the loop to show the pipeline in action. It is the culmination of previous phases. So even if minimalistic, it must not have obvious gaps. We'll double-check that every value we want to show is either available via some implemented method or we noted how to compute it. If any trivial method is missing, we add it.

Now, I'll proceed to compose the final answer using this structured plan, making sure each section includes all required details and citations we prepared.## A. Phase-by-Phase Design Specs (E1–E6)

## E1 – Data & Repository Integrity

## E1-Requirements:

- Establish robust repository classes for each core model (Thesis, Observation, Trade) providing CRUD operations as specified. This includes adding missing methods: e.g. ThesisRepository.get\_by\_id, list\_all, list\_recent, etc., to eliminate any direct database queries scattered in code.
- Implement a coherent DataAccess layer that uses these repositories (no ad-hoc hasattr checks) and returns Pydantic-validated DTOs. DataAccess will have methods to fetch theses, related observations, and portfolio snapshots in a consistent manner.
- Define Pydantic models for portfolio views: **PortfolioSnapshot** (with list of positions and aggregate totals) and **PortfolioDepthSnapshot** (with concentration, factor, and thesis exposure breakdowns). These provide a typed interface for higher layers (LLM, UI) to consume portfolio/risk data.
- Ensure data integrity with round-trip tests: inserting an object via a repository and then retrieving it should yield the same data. Verify DataAccess methods return correct types (not NoneType or raw dicts) – e.g., get\_current\_portfolio() should return a PortfolioSnapshot whose totals.portfolio\_value is a float, not missing .
- Remove or refactor any brittle code in context builder or UI that assumed missing fields (like the prior portfolio["total\_value"] issue). With proper DTOs, such attributes will always exist (even if value is 0).
- Use a test database or transactional tests to avoid persistence side effects. The repository and DataAccess code should be testable in isolation (by dependency-injecting an in-memory or test DB session).

## E1-Research Summary:

The **Repository pattern** will decouple our business logic from direct database calls, improving testability and maintainability . We implement explicit interfaces (e.g., ThesisRepository.insert/get/list) to standardize how data is accessed and allow mocking in tests. This avoids tight coupling to ORM or raw SQL in the service layer. A repository abstraction makes unit tests easier by allowing use of in-memory implementations . In practice, trying to unit-test database operations can blur into integration testing ; we accept that and plan for **integration tests** against a test DB for repository correctness. For example, rather than mocking SQLAlchemy internals, we insert sample records and assert retrieval matches – this aligns with treating database interactions as integration concerns .

Using **Pydantic** for data models ensures that data leaving the data layer conforms to expected schema (types, required fields) . We'll leverage Pydantic's BaseModel to define PortfolioSnapshot and others. Pydantic can even create models from ORM objects (orm\_mode=True in Config) , which means if our repositories return SQLAlchemy models, we can parse them into our schema easily. This validation layer catches issues early – e.g., if a

repository forgot to populate a field, Pydantic would raise an error. It also eliminates the kind of runtime attribute errors we previously saw, by making the contract explicit.

For testing repository integrity, we will use **pytest** with a setup that provides a fresh database (or transaction rollback) for each test. According to best practices, each test should be independent and test the public interface of the repository class (not the internal SQL) . For instance, to test `ObservationRepository.list_for_thesis`, we will insert multiple observations with known `thesis_refs` and ensure only the matching ones are returned. We treat these as integration tests because they involve the DB, but they are scoped to a small component and thus remain fast and deterministic.

## E1-Design:

- **Repository Classes:** We extend existing repository modules in `src/slice/repositories/`. Each repository will encapsulate database access for its model:

```
class ThesisRepository:
 def __init__(self, db_session):
 self.db = db_session # or engine/connection
 def insert(self, thesis: Thesis) -> Thesis:
 # SQL or ORM code to insert and commit
 return thesis # possibly with id set if generated
 def get_by_id(self, thesis_id: str) -> Thesis | None:
 # SELECT * FROM thesis WHERE id = ?
 def list_all(self) -> list[Thesis]:
 # SELECT * FROM thesis
 def list_recent(self, limit: int) -> list[Thesis]:
 # SELECT * FROM thesis ORDER BY created_at DESC LIMIT ?
```

- We'll implement similarly for `ObservationRepository` and `TradeRepository` with the methods outlined in E1 requirements. For example, `TradeRepository.list_by_thesis(thesis_id)` will do `SELECT * FROM trades WHERE thesis_ref = ?`. These methods hide the SQL/ORM details from higher layers. The codebase uses raw SQL via `slice.db.get_engine()` currently; we might continue with raw SQL for consistency (keeping queries simple for these basic CRUDs). If using SQLAlchemy ORM models (like `Thesis` as a mapped class), we can use session queries . Either way, the interface is the same.

We also consider a **ScenarioRepository** if scenario definitions are stored in DB (Phase 2 of original plan mentioned it). If scenarios are static or computed, we may not need to store them now – possibly skip unless needed by `DataAccess`.

- **DataAccess Layer:** Located at src/slice/data\_access.py (or ../intelligence/context/data\_access.py as in current code). This will be initialized with repository instances:

```
class DataAccess:
 def __init__(self, thesis_repo: ThesisRepository, obs_repo:
ObservationRepository,
 trade_repo: TradeRepository, scenario_repo:
ScenarioRepository = None):
 self.thesis_repo = thesis_repo
 self.obs_repo = obs_repo
 self.trade_repo = trade_repo
 self.scenario_repo = scenario_repo
```

- **Methods to implement (with type signatures):**

```
def get_thesis(self, thesis_id: str) -> Thesis | None:
 thesis = self.thesis_repo.get_by_id(thesis_id)
 return thesis # (could return Pydantic ThesisModel if we define one,
or the ORM object - both workable; often we might define a ThesisSchema
Pydantic model for external use)
def get_all_theses(self) -> list[Thesis]:
 return self.thesis_repo.list_all()
def get_observations_for_thesis(self, thesis_id: str) ->
list[Observation]:
 return self.obs_repo.list_for_thesis(thesis_id)
def get_recent_observations(self, limit: int = 10) -> list[Observation]:
 return self.obs_repo.list_recent(limit)
```

- **For portfolio-related methods:**

```
def get_current_portfolio(self) -> PortfolioSnapshot:
 trades = self.trade_repo.list_all()
 # Aggregate positions from trades
 positions = []
 totals = {"portfolio_value": 0.0, "gross_exposure": 0.0,
"net_exposure": 0.0}
 # Use price_source to get current prices for each unique asset
 for asset, trades_group in group_by_asset(trades):
 quantity = sum(t.quantity if t.action == "BUY" else -t.quantity
for t in trades_group)
 if quantity == 0:
 continue
 price = price_source.get_current_price(asset)
 value = quantity * price
 positions.append(Position(asset=asset, quantity=quantity,
value=value))
 totals["portfolio_value"] += value
 totals["gross_exposure"] += abs(value)
 totals["net_exposure"] += value
 return PortfolioSnapshot(positions=positions,
totals=PortfolioTotals(**totals))
def get_portfolio_depth(self) -> PortfolioDepthSnapshot:
 # For now, derive simple breakdowns:
```

```

concentration: e.g., {largest_asset: largest_value/portfolio_value}
factors: {} (we have no factor data yet, stub empty)
thesis_exposures: sum of values of trades per thesis
theses = self.thesis_repo.list_all()
thesis_exposures = {t.id: 0.0 for t in theses}
for trade in self.trade_repo.list_all():
 thesis_exposures[trade.thesis_ref] += trade.quantity *
price_source.get_current_price(trade.asset) * (1 if trade.action == "BUY"
else -1)
compute concentration
values_by_asset = {pos.asset: pos.value for pos in
self.get_current_portfolio().positions}
concentration = {}
if totals := values_by_asset:
 total_val = sum(abs(v) for v in values_by_asset.values())
 # e.g., list top 3 positions by absolute value
 for asset, val in sorted(values_by_asset.items(), key=lambda x:
abs(x[1]), reverse=True)[:3]:
 concentration[asset] = abs(val) / total_val if total_val else
0.0
 return PortfolioDepthSnapshot(concentration=concentration,
factors={}, thesis_exposures=thesis_exposures)

```

- (This pseudo-code shows how we aggregate portfolio info. We utilize price\_source – an abstraction for getting current prices, which will be provided in E2. We assume here DataAccess has access or it's global. We'll clarify integration with E2's price\_source when we get there.)

The DataAccess methods return either domain objects or Pydantic models. For internal service use (like LLM context building), returning domain objects (Thesis, Observation) might suffice. But to avoid confusion and to leverage Pydantic validation, we could define Pydantic equivalents:

- e.g., ThesisModel(BaseModel) reflecting Thesis fields. If we use those, get\_thesis would do ThesisModel.from\_orm(thesis) . Given time, we may skip separate models for theses/obs and just use the ORM/dataclass instances directly (since they are likely simple dataclasses as per codebase summary).
- **DTO Models:** Create src/slice/models/portfolio.py (to keep models organized):

```

from pydantic import BaseModel
class Position(BaseModel):
 asset: str
 quantity: float
 value: float
class PortfolioTotals(BaseModel):
 portfolio_value: float
 gross_exposure: float
 net_exposure: float
class PortfolioSnapshot(BaseModel):
 positions: list[Position]
 totals: PortfolioTotals
class Config:
 orm_mode = True

```

```
class PortfolioDepthSnapshot(BaseModel):
 concentration: dict[str, float]
 factors: dict[str, float]
 thesis_exposures: dict[str, float]
```

- The `orm_mode=True` might not be needed unless we generate these from ORM objects, but it's included for completeness. Now, `get_current_portfolio()` will construct a `PortfolioSnapshot` by populating `Position` entries and totals and returning it (Pydantic will validate types e.g., ensuring `portfolio_value` is float) .

We interpret **gross\_exposure** as the sum of absolute position values (long \$100 + short \$100 = \$200 gross), and **net\_exposure** as the sum of signed values (long \$100 + short \$100 = \$0 net). For a fully invested long-only portfolio, `gross = net = portfolio_value`. These metrics help risk evaluation.

**Note:** If short positions exist, `portfolio_value` in our snapshot will show net value (assets minus liabilities), effectively the equity. That could be lower than gross. We consider that acceptable (the difference is the cash from short sales, which we don't list as a position explicitly – we assume it's embedded in net exposure). We highlight that if needed in risk discussion.

- **Data Integrity Improvements:** The `DataAccess` and `DTO` approach prevents missing keys. For example, before, UI expected `portfolio["total_value"]` which wasn't set; now UI will access `snapshot.totals.portfolio_value`, and Pydantic guarantees `portfolio_value` is there (or validation would fail). This addresses the earlier architecture vagueness around portfolio structure.
- **Error handling:** If a repo method returns `None` (e.g., thesis not found), `DataAccess` passes that through or returns `None/empty list`. Higher layers will check for `None` and handle (e.g., API returning 404 if `get_thesis` is `None`). We won't let `None` propagate to places expecting an object without handling.

## E1-Testing Plan:

- *Repository Tests:* Using a temporary database (like SQLite in-memory for convenience), create instances of each repository pointing to it. For each repository method:
  - **Insert and Get:** Create a `Thesis` object (e.g., `Thesis(title="Test", hypothesis="...")` with minimal fields), insert it. Assert that the returned object (or a subsequent `get_by_id`) has a non-null ID and matches the input fields . Similarly, insert an `Observation` and retrieve it. Check edge cases: `get_by_id` on non-existing ID returns `None` (not exception).
  - **Listings:** Insert 3 theses with known titles or timestamps. Call `list_all()` – expect a list of length 3. Optionally, ensure ordering if not guaranteed by DB (`list_all` may not specify order, which is fine). Call `list_recent(2)` – expect 2 results, presumably the ones with latest creation time (we might simulate creation times by inserting in sequence if using autoincrement IDs as a proxy). Assert no duplicates, and that



each method respects the limit. For ObservationRepository, test `list_for_thesis`: insert observations for two different theses, ensure the filter returns only those tied to the specified thesis ID. If Observation.thesis\_refs is a list field (possibly many-to-many), our implementation might need to query a join table; in test, we might simplify and assume a single thesis\_ref field for now (since codebase summary suggests Observation.thesis\_refs is list of IDs – we might need to store that in a link table; for test, assume one observation can reference one thesis to simplify).

- **TradeRepository:** Insert a couple of Trade records (for one thesis and another). Test `list_by_thesis(thesis_id)` returns correct subset, and `list_all` returns all. If using raw SQL, ensure our mapping to Trade model is correct (price as float, etc.). Also test that trades persist with correct data (e.g., after insertion, a trade has an autogenerated ID or timestamp set).
- For each, test behavior on empty DB: `list_all` on no records returns an empty list (not None or error), `list_for_thesis` on thesis with no obs returns []. These ensure stability (no exceptions thrown for no data).
- *DataAccess Tests:* Use either the real repository connected to test DB or a stub that returns known values:
  - **get\_thesis / get\_all\_theses:** We can reuse inserted data from repo tests. E.g., after inserting 3 theses, `DataAccess.get_all_theses()` should return a list of length 3 (possibly of Thesis objects or Pydantic models). If our DataAccess is just passing through repository data for theses, this is trivial – mostly ensuring no type conversion issues. If we introduced a Pydantic ThesisModel, we would validate that here (e.g., ensure a field like title is present and correct type).
  - **get\_observations\_for\_thesis:** Insert a thesis and two observations linking to it. Verify DataAccess returns those two and not others. If Observation.thesis\_refs is a list, in test perhaps assign the thesis\_id to that list and the repository query filters by containment (or join logic). This might be complex to implement purely with raw SQL; if too complex, we could simplify by saying each observation has a thesis\_id foreign key for now (limiting one thesis link), since we only need to illustrate capability.
  - **get\_current\_portfolio (no trades):** If no trades in DB, it should return PortfolioSnapshot with positions=[] and totals.portfolio\_value = 0. Our implementation should handle empty gracefully (the loop will result in no positions and totals remain 0). We add an explicit test for this initial state. This was partially covered in Phase 9 tests in codebase, so we ensure to meet that expectation.
  - **get\_current\_portfolio (with trades):** Simulate a simple portfolio. For example, insert two trades:
    - Trade1: BUY 10 shares of “ABC” at \$100 (action=“BUY”, quantity=10, price=100, thesis\_ref=“T1”).
    - Trade2: BUY 5 shares of “XYZ” at \$200 (thesis\_ref=“T1” or another).

Now we need price\_source to supply current prices. In DataAccess, we might incorporate price\_source as a global or passed in. For test, we can monkeypatch price\_source.get\_current\_price to return known values (e.g.,

\$110 for ABC, \$180 for XYZ). Then call `get_current_portfolio()`. The result `PortfolioSnapshot` should have:

- 2 positions (ABC and XYZ).
- ABC position: quantity 10, value  $10 * \$110 = 1100$ . XYZ: quantity 5, value  $5 * \$180 = 900$ .
- `totals.portfolio_value = 2000`, `gross_exposure = 2000` (all long), `net_exposure = 2000`.

Assert those values. Also check types: `portfolio_value` is float, not string or Decimal.

- Additionally test a mix of buys and sells: e.g., add `Trade3: SELL 2 shares of ABC at $120` (representing maybe a short or selling some holdings). Then `get_current_portfolio` should net ABC quantity = 8 ( $10 - 2$ ) and use sign correctly. If we allowed short positions (quantity could become negative if sells exceed buys).
  - E.g., if we SELL 15 shares of XYZ (5 we had long plus 10 short), then net quantity = -10 (short 10). The position value =  $-10 * \text{current\_price}$ . The snapshot should include that negative value. Test that `net_exposure` accounts for negative properly (net will drop). For example, if we shorted, `portfolio_value` will reflect that liability (we might see `portfolio_value < gross_exposure`). We assert our computation does what's expected (and not, say, double-count).
  - These scenarios ensure the aggregator logic (summing buys and sells) is correct.
- **get\_portfolio\_depth:** Build on the above portfolio:
  - concentration: after those 2 positions (1100 and 900), the largest asset is ABC ~55% of total. Our method should list `{'ABC': 0.55, 'XYZ': 0.45}` (approx). Check presence and correctness. If we limit to top 3, both appear since we have 2.
  - thesis\_exposures: if both trades were `thesis_ref "T1"`, then T1 exposure = \$2000. If one trade had another `thesis_ref`, we'd see split. We can test with at least two theses: e.g., mark one trade with thesis "T2". Then expected exposures: T1 ~\$1100, T2 ~\$900. Confirm sum of exposures equals `portfolio_value` (2000). This verifies that we correctly map trades to thesis exposures.
  - factors: since we stubbed as empty dict, just assert it's an empty dict (and no error).
  - Edge: if portfolio is empty, depth might return all zeros or empty dicts. We handle that (our code sets `total_val=0` leads to no entries in concentration, that's fine). Test that no exception occurs on empty (maybe an empty concentration dict and exposures all zero).
- All data returned by `DataAccess` should be either Pydantic models or Python built-ins. If we returned Pydantic models (`PortfolioSnapshot` etc.), we can check

that `isinstance(result, PortfolioSnapshot)` is True and that its `.dict()` matches expected structure. Pydantic also ensures any missing fields would have raised error already, which is implicitly tested by simply calling these methods without exceptions.

- *Performance & Misc:* Though not explicitly required, we can write a quick test to ensure repository methods aren't insanely slow (e.g., calling `list_all` with 1000 entries returns quickly). But as this is not heavy data, and given evaluation context, we focus on correctness.

## E1-Risks & Dependencies:

- *Database Schema Alignment:* We assume the underlying DB has tables and columns matching our model fields. A risk is that if the schema is different (or missing a column we expect), our repository calls might fail. We mitigate by using the codebase's model definitions as truth. For example, Trade likely has fields (`trade_id`, `asset`, `action`, `quantity`, `price`, `thesis_ref`); we ensure our SQL uses the correct names. If the schema for Observation's `thesis_refs` is a JSON array or separate table, our implementation might need adjusting. For now, we'll assume a simple design or adjust once we scan actual DB DDL (if available) – else we document assumptions and possibly simplify to one-to-many for ease.
- *Multiple thesis references in Observation:* If an Observation can reference multiple theses, `list_for_thesis` should return any observation that mentions that thesis. Our implementation using a single foreign key would miss cross-links. A robust approach is a join table (`ObservationThesisLink`). Given time constraints, we might handle one-to-many and note that extending to many-to-many is a to-do (with either an explicit join query or using Postgres array contains). We mark this as a simplification assumption.
- *Time fields:* `list_recent` likely relies on a timestamp. If our Thesis model has `created_at`, we should use it. If not, using ID ordering (assuming sequential). We prefer to add a `created_at` field if not present, as it's more explicit. In tests, we might simulate by inserting in a known sequence and checking that sequence reversed equals `list_recent` output for a given limit.
- *DataAccess Price Dependency:* `get_current_portfolio` uses `price_source`. We need an actual price provider by E2. For now, in tests we'll provide a stub. In production, `DataAccess` might get a singleton price service (e.g., from dependency injection). We have to ensure that exists or design `DataAccess` to accept a `price_source` param as well. Perhaps better: in `DataAccess` methods like `get_current_portfolio`, instead of directly calling `price_source`, we call a `PriceService` global or static method. We'll coordinate with E2 design where we introduce `price_source`. For now, treat it as an external dependency to be passed or set (we'll likely add `price_source` attribute to `DataAccess` too).
- *Pydantic model performance:* For very large lists, Pydantic parsing can be slow. But our data volumes are small (tens of items), so not an issue. The clarity benefits outweigh any minor overhead.

- *Ensuring no “phase-dependent magic”*: The original architecture sometimes assumed a component exists later (e.g., risk engine) without defining it. Here in E1, risk is minimal (just portfolio depth). We consciously implement at least a stub for risk (concentration, exposures) so that Phase 4/6, which expects some risk snapshot, isn’t left waiting on undefined interfaces. We have now defined PortfolioDepthSnapshot and DataAccess.get\_portfolio\_depth – this replaces the vague “risk engine will exist” with a concrete contract.
- *Integration with existing tests*: Since the codebase had some Phase 9 tests expecting certain behaviors, we must ensure not to break them. For instance, test\_data\_access\_portfolio expecting portfolio\_value in totals. Our design meets that by PortfolioTotals.portfolio\_value. But if the test was directly indexing a dict, we might adapt (maybe DataAccess.get\_current\_portfolio initially returned a dict). However, since we return a Pydantic model, if the API or UI still expects a dict, we can easily convert via .dict(). We’ll double-check how UI routes were implemented (the summary said UI routes directly returned context dicts). Possibly, our UI routes will use .dict() on these models before sending JSON. That should satisfy any front-end expecting keys. We’ll note this conversion in Phase 4 (API endpoints).
- *Assumption clarity*: We explicitly assume a single currency environment (all values in USD, etc.), so portfolio\_value is meaningful sum. Multi-currency would complicate net exposure, but out of scope.
- *Future extension*: If new repository methods are needed (e.g., update or delete), those can be added with similar patterns. For evaluation, CRUD read operations were priority and we covered those.
- *Memory vs DB sync*: Using an actual database in tests covers integration well, but in-memory SQLite may not exactly match Postgres types (like JSON field behavior). But for simple types used, it should be fine. We mention that if something like thesis\_refs is JSON in Postgres, our test using SQLite might differ. We can circumvent by not testing that specific feature deeply or using Postgres in tests (with docker, but that’s heavy for evaluation). We’ll likely accept slight differences and focus on logical correctness.

By solidifying E1, we lay a trustworthy foundation: the rest of the pipeline (evaluation, LLM, etc.) can rely on these interfaces for consistent data access, which was a weak point in earlier phases. The code will be cleaner and tests will ensure no regressions in basic data handling.

## E2 – Thesis → Quant → Risk/Scenario Path

### E2-Requirements:

- Develop a **ThesisEvaluationService** module that takes a Thesis and evaluates its investment expression quantitatively. The service will produce a

**ThesisEvaluationResult** containing performance metrics, a portfolio equity curve, basic risk metrics, and scenario analysis results.

- Implement a **static allocation backtest**: interpret the thesis's expression legs as target portfolio weights (long or short) and simulate the portfolio value over time using historical price data. Use daily rebalancing to maintain those weights (simplest assumption). This yields a deterministic equity curve for the period.
- Compute standard **performance metrics** from the equity curve: total return (fractional and/or %), CAGR (annualized return), annualized volatility, Sharpe ratio (using risk-free 0 for simplicity), and max drawdown. These will populate `ThesisEvaluationResult.performance` fields.
- Compute a set of **risk metrics** (`ThesisEvaluationResult.risk_metrics`): at least include concentration (e.g. highest asset weight or contribution) and a simple Value-at-Risk or drawdown-based risk measure. For example, 95% historical VaR (worst 5th percentile daily return) or simply the max drawdown as a risk indicator. This provides a quantitative risk snapshot to complement LLM analysis.
- Include **timeseries output**: Provide the equity curve as a list of points (date and portfolio value) in the result so it can be graphed or inspected. We define an `EquityPoint(date, value)` or use a date->value mapping. The length equals the backtest period's trading days.
- Implement a rudimentary **scenario analysis**: define some hypothetical market scenarios (shock events) and compute the thesis portfolio's P&L under each. For instance, scenarios like "Equity -10%" or "Oil +20%" affecting certain assets. For each scenario, output absolute and percentage P&L for the portfolio. This will populate `ThesisEvaluationResult.scenarios` as a list of `ScenarioImpact` entries.
- Integrate validation: the service should validate that the Thesis's expression is evaluatable (e.g., total weight  $\leq 100\%$ , known asset tickers) before running. If validation fails, throw an error or return a flagged result. Only "executable" legs (with valid asset symbols and directions "long"/"short") should be processed.
- The `ThesisEvaluationService` should be usable independently (without LLM), and its results will feed into LLM critique in E3. Ensure it is deterministic (given same input data) to support reproducible tests.
- Write unit tests with **synthetic price data** to verify that the backtest calculations and metrics are correct, independent of external data sources.

## E2-Research Summary:

**Backtesting** is the simulation of an investment strategy on historical data to evaluate performance. It's a critical practice to see if a thesis would have made money in the past and how risky it was. By rebalancing daily to fixed weights, we effectively create a **constant-mix portfolio**, which is a common baseline strategy. This simplifies calculations: portfolio return on each day =  $\sum(\text{weight}_i * \text{asset\_return}_i\_\text{day})$ . Summing through time gives the equity curve. This method is akin to what one would do in a spreadsheet to apply weights to asset price series and compute cumulative returns. Using a vectorized approach (via pandas or numpy) is wise for efficiency and clarity.

We will calculate **performance metrics** following standard formulas:

- *Total Return* = (Ending Portfolio Value / Starting Value) - 1. For reporting, we might express it as a percentage.
- *CAGR* (Compound Annual Growth Rate) translates the total return to an annualized figure:  $\text{CAGR} = (1 + \text{TotalReturn})^{1/T} - 1$ , where T is number of years in the period. If our backtest spans N trading days, we can take  $T = N/252$  (assuming ~252 trading days/year) .
- *Volatility* = annualized standard deviation of daily returns. Compute daily returns from equity curve, get  $\text{stdev}(\text{returns}) = \sigma_{\text{daily}}$ , then  $\text{volatility} = \sigma_{\text{daily}} * \sqrt{252}$  .
- *Sharpe Ratio* = (Annualized Return - RiskFree) / Volatility. Assuming RiskFree ~ 0, it's essentially Return/Vol. If using CAGR for return and the volatility as defined, Sharpe = CAGR/Vol. We ensure to use consistent periods (both annualized) . Sharpe gives a risk-adjusted performance measure.
- *Max Drawdown*: we traverse the equity curve to find the largest peak-to-trough drop . A common algorithm: track the running maximum value, and at each point compute  $\text{drawdown} = (\text{current\_value} - \text{running\_max}) / \text{running\_max}$ . The most negative drawdown is the max drawdown (expressed as a negative percentage or positive magnitude of drop). We'll output it as a positive percentage for clarity (e.g., 20% drawdown as 0.20). This is a key risk metric as it tells how deep losses got at worst.

These metrics (Return, Vol, Sharpe, MDD) are widely used in evaluating portfolios. By providing them, we allow both the PM and the LLM to objectively gauge performance .

**Risk metrics:** besides max drawdown which we already have, we consider adding at least one VaR-like metric. A simple approach: sort daily returns and pick the 5th percentile as an estimate of 95% one-day VaR (as a negative number). For example, if the worst 5% daily loss was -2%,  $\text{VaR}_{95} = 2\%$ . This gives a sense of tail risk . We might include that in `risk_metrics`. Also, concentration risk can be derived from weights (e.g., if one asset weight is 80%, that's high concentration risk). We can include the top weight (or the Herfindahl index, but likely just top weight as a %). These quantitative risk flags complement qualitative LLM risk flags.

**Scenario analysis:** Professional risk management often includes stress tests: *"If equities crash 20%, what happens to our portfolio?"* We simulate such scenarios by shocking asset prices and recomputing portfolio value difference. For instance, define a scenario Equity -20%: for each asset that is an equity, reduce its price by 20%; for bonds or others, maybe no change or a smaller related move. Then calculate new portfolio value vs current. The result is scenario P&L . We will create a small set of scenarios for demonstration:

- **Down Market:** All assets drop 10% (pessimistic global shock).
- **Up Market:** All assets up 10% (optimistic scenario).
- We can also add scenarios tailored to asset classes if we know them (e.g., if any gold in portfolio, a scenario where gold -10%; or if any FX, shock USD). But lacking asset classification, we might stick to broad ones or ones derived from tickers (e.g., if ticker contains “Gold”, we can define a “Gold -10%” scenario). Simplicity is key, since we mainly want to showcase that the pipeline supports scenario analysis.

This scenario output helps identify sensitivities – e.g., a portfolio heavily in equities will show big negative P&L in “Down Market” scenario, highlighting risk to LLM and user.

**External libraries vs custom:** We note that robust backtesting can be done with libraries like **Backtrader** or **VectorBT**, which handle portfolio accounting, but integrating them might be heavy. Instead, we implement it directly with pandas for our static allocation, which is straightforward: one can multiply price relatives (daily returns) by weights and cumulate . This custom approach is fine given limited scope and allows easier insertion of our metrics calculations and tests. It also ensures we thoroughly understand and verify each step (no black-box) – crucial in an evaluation context.

## E2-Design:

- **PriceSource and Data Fetching:** We introduce a price\_source component that ThesisEvaluationService will use to get historical prices. Likely an interface like:

```
class PriceSource:
 def get_history(self, asset: str, start: date, end: date) -> pd.Series:
 ...
 def get_current_price(self, asset: str) -> float:
 ...
```

- For now, we can implement a simple version using a local data store (maybe CSV or in-memory dict for test). In production, this could call an API or database of prices. The ThesisEvaluationService will be initialized with a price\_source (as seen in requirements). This decouples the evaluation from actual data retrieval, allowing us to inject fake data in tests or switch sources easily (CSV, API, etc.) .
- **ThesisEvaluationService Implementation:** In src/slice/evaluation/thesis\_evaluation.py:

```
import pandas as pd
from datetime import datetime

class ThesisEvaluationService:
 def __init__(self, price_source):
```

```

self.price_source = price_source

def evaluate_thesis(self, thesis: Thesis) -> ThesisEvaluationResult:
 # 0. Validate thesis
 if sum(leg.size_pct for leg in thesis.expression_legs) > 100.0:
 raise ValueError("Thesis allocation exceeds 100%")
 if not thesis.expression_legs:
 raise ValueError("Thesis has no expression legs to evaluate")
 for leg in thesis.expression_legs:
 if leg.direction not in ("long", "short"):
 raise ValueError(f"Invalid direction {leg.direction} in
thesis leg")

 # 1. Determine backtest period
 # Could use thesis.start_date and thesis.end_date if present; if not,
choose a default window (e.g., last 2 years)
 start_date = thesis.start_date or datetime(2015, 1, 1)
 end_date = thesis.end_date or datetime.now()

 # 2. Fetch historical price series for each asset
 asset_prices = {}
 for leg in thesis.expression_legs:
 series = self.price_source.get_history(leg.asset, start_date,
end_date)
 if series is None or series.empty:
 raise ValueError(f"No price data for asset {leg.asset}")
 asset_prices[leg.asset] = series.copy() # pd.Series indexed by
date

 # Align dates (inner join on index to ensure all have same dates)
 prices_df = pd.DataFrame(asset_prices)
 prices_df.dropna(inplace=True) # ensure no NaNs (or we could
forward-fill, but drop for strictness)

 # 3. Compute daily portfolio value series
 # Start with initial portfolio value = 1 (or 100 for percentage)
 weights = {leg.asset: (leg.size_pct / 100.0) * (1 if leg.direction ==
"long" else -1)
 for leg in thesis.expression_legs}
 # Normalize if sum of weights < 1 (meaning remainder is cash which
yields no return, we can implicitly handle that by assuming cash is part of
portfolio that just doesn't grow)
 # Actually, if sum < 1, we'll treat rest as cash not affecting
returns (so portfolio won't be fully invested).
 total_weight = sum(abs(w) for w in weights.values())
 # Note: if total_weight < 1, then effectively portfolio had (1-
total_weight) in cash doing nothing.
 # We can proceed without adjustment; the performance will be based on
invested part.

 # Calculate daily returns using percentage changes
 daily_returns = prices_df.pct_change().fillna(0) # first day
pct_change NaN -> 0
 # Weighted daily return = sum(weights_i * return_i)
 daily_portfolio_returns =
daily_returns.mul(pd.Series(weights)).sum(axis=1)

```



```

Compute equity curve: cumulative product of (1 + daily_return)
equity_curve = (1 + daily_portfolio_returns).cumprod()

4. Compute performance metrics
final_value = equity_curve.iloc[-1]
total_return = final_value - 1.0
num_days = equity_curve.shape[0]
years = num_days / 252.0
cagr = (final_value ** (1/years) - 1.0) if years > 0 else 0.0
vol_daily = daily_portfolio_returns.std()
volatility = vol_daily * (252 ** 0.5)
sharpe = (cagr / volatility) if volatility != 0 else float('inf')
Max drawdown calculation
running_max = equity_curve.cummax()
drawdowns = (equity_curve - running_max) / running_max
max_drawdown = drawdowns.min() * -1 # as positive number

5. Compute risk metrics
risk_metrics = {}
Concentration: max weight (abs)
if weights:
 max_weight = max(abs(w) for w in weights.values())
 risk_metrics["max_weight_pct"] = max_weight * 100
Historical VaR 95
if len(daily_portfolio_returns) > 0:
 var95 = daily_portfolio_returns.quantile(0.05)
 risk_metrics["VaR_95"] = -var95 # as positive percentage loss
Max drawdown as risk
risk_metrics["max_drawdown_pct"] = max_drawdown * 100

6. Compute scenarios
scenarios = []
Example scenarios (shock price level, i.e., shock returns):
current_values = {asset: prices_df[asset].iloc[-1] for asset in
prices_df.columns}
Scenario: All assets -10%
new_values = {asset: val * 0.90 for asset, val in
current_values.items()}
pnl = sum(weights[a] * (new_values[a] - current_values[a]) /
current_values[a] for a in current_values)
scenarios.append(ScenarioImpact(name="All -10%", pnl_pct=pnl*100,
pnl_abs=pnl * final_value))
Scenario: All assets +10%
new_values = {asset: val * 1.10 for asset, val in
current_values.items()}
pnl = sum(weights[a] * (new_values[a] - current_values[a]) /
current_values[a] for a in current_values)
scenarios.append(ScenarioImpact(name="All +10%", pnl_pct=pnl*100,
pnl_abs=pnl * final_value))
(Additional scenarios could be added depending on asset types)

7. Package results
perf = {
 "total_return": total_return * 100,
 "cagr": cagr * 100,
 "volatility": volatility * 100,
 "sharpe": sharpe,

```

```

 "max_drawdown": max_drawdown * 100
 }
 equity_points = [EquityPoint(date=idx, value=val) for idx, val in
equity_curve.items()]
 return ThesisEvaluationResult(performance=perf,
timeseries=equity_points,
 risk_metrics=risk_metrics,
scenarios=scenarios)

```

- Several notes on this implementation:

- We use pandas for simplicity; this assumes the data from PriceSource comes as a pandas Series (which is convenient for alignment). If PriceSource returns a list of (date, price), we can easily build a Series.
- We start portfolio at 1 (meaning 100%). Final value is the factor by which initial capital grew; so total\_return = final - 1. We output performance metrics likely as percentages (e.g., 10% as 10). We must be consistent. In code above, I multiplied by 100 where needed to express in percentage terms for output. We will clarify in result schema whether we store 10 or 0.10 for 10%. Likely as percentage numbers (10 for 10%).
- We treat unallocated weight as cash: if sum of weights < 1, that portion just yields 0 returns. This is naturally handled since we didn't include cash in daily\_portfolio\_returns calculation, effectively meaning that portion doesn't contribute (which is correct).
- We handle short by weights being negative. E.g., if 50% long A, 50% short B, weights = {A: 0.5, B: -0.5}. Then daily portfolio return = 0.5\*ret\_A + (-0.5)\*ret\_B. If B goes up, that yields negative contribution (loss).
- We drop NaN dates (so we require all asset series to have data on same dates). This means if one asset has shorter history, we restrict to common range. It's a design choice: we prefer consistent comparison. If an asset doesn't exist for entire period, maybe the thesis couldn't exist then. In reality, we might forward/backfill or drop that asset for earlier dates. Simplicity: use common overlap.
- We choose an evaluation period (start\_date). If not specified by user, I put a default of 2015–present. Alternatively, last 5 years could be default. We should mention this default assumption. We could also use the earliest data available for assets. For rigorous approach: find max(start dates of all series) as start. But that may shorten period too much if one asset has short history. Perhaps better to allow partial: if one asset missing early, treat its weight as in cash until it appears. That's complex. Given our focus, we might assume all assets have data since start\_date. We'll note this assumption.
- We compute scenario P&L in a simplified way:  $\text{pnl}_{\%} = \sum_i w_i * \Delta P_i / P_i$ . For small shocks, this approximates actual portfolio % change. For "All -10%" scenario with weights summing to 1, it yields -10% if fully invested. For partial invest or shorts, it reflects appropriately. We output both pnl\_pct and pnl\_abs (the latter we computed as fraction \* final portfolio value).
- We create a Pydantic model ScenarioImpact and EquityPoint. They would be defined in models (similar to PortfolioSnapshot). For instance:

```

class ScenarioImpact(BaseModel):

```

```

name: str
pnl_abs: float
pnl_pct: float
class EquityPoint(BaseModel):
 date: datetime
 value: float
class ThesisEvaluationResult(BaseModel):
 performance: dict[str, float]
 timeseries: list[EquityPoint]
 risk_metrics: dict[str, float]
 scenarios: list[ScenarioImpact]

```

- - We use dict for performance and risk\_metrics for flexibility and because the keys are known small set. We could also break them out explicitly in the model if preferred (fields for each metric).
  - Validate metric calculations on a simple known case (in tests). For example, if portfolio is flat (no change), total\_return=0, volatility=0, sharpe= $\infty$  (we may represent that as inf or handle by setting 0 since no risk).
  - If volatility is 0 (no movement), Sharpe is set to infinity in code. In practice, Sharpe undefined if vol=0. We could cap or set to None. Perhaps better to set sharpe to 0 if vol=0 *and* return=0, or very high if return positive. We output float('inf') which when converting to JSON might become null or string (inf isn't JSON serializable). We may convert that to some large number or None. This is an edge scenario (like if one asset constant price).
  - We store performance metrics in percent form (ex: 5% as 5.0). It's important to document that for UI/interpretation.
- **Integration with DataAccess / E1:** The evaluation service needs price data. Possibly, DataAccess could have provided price series (if we stored prices in DB). But likely, price data is external. We treat PriceSource separate. DataAccess get\_current\_portfolio uses price\_source for current prices; similarly, evaluation uses price\_source for history. We ensure we can use the same or a consistent price source (maybe a TwelveData API or Pandas DataReader). For testing, we'll implement a dummy PriceSource with known series.
- **Testing Approach for ThesisEvaluationService:**
  - We will test the backtest logic with **controlled synthetic data** to verify correctness:
    - Create a dummy PriceSource that returns a fixed price series. For example:
      - Asset "A": price starts at 100 and increases by 1 each day (so returns are positive).
      - Asset "B": price starts at 100 and decreases by 1 each day (so returns negative).
      - Both for, say, 10 days.
    - Define a thesis with 50% long A, 50% long B. Over this period, A went up ~+9%, B went down ~-9%. We can manually calculate expected outcomes: since weights equal, the portfolio is roughly flat (A's gains offset B's losses). Specifically, if A +9%, B -9%, the total return should be

~0%. We expect the backtest to yield total\_return near 0, small volatility, Sharpe ~0, max\_drawdown maybe small. We compare results:

- If A grows monotonic and B declines monotonic, the worst drawdown might occur in middle when B's loss outpaced A's gain slightly.
- We can compute each day's portfolio by hand for a short series and verify the output's equity\_curve and metrics.
- Test with 100% in one asset to verify trivial cases:
  - 100% in Asset A (increasing): portfolio return should equal A's return. If A went from 100 to 109 (9% gain), total\_return ~9%. Sharpe > 0, etc. We verify fields exactly:
    - cagr ~9% (if period ~0.04 years, but annualizing such short period might give huge number, but let's use entire year simulation or adjust period).
  - 100% in Asset B (decreasing): total\_return ~-9%, Sharpe negative.
  - Check that risk\_metrics "max\_weight\_pct" = 100 in these single-asset cases, and VaR\_95 roughly equals the worst daily drop of that asset.
- Test short positions:
  - Asset A up-trending, if thesis is 100% short A, the portfolio should *lose* what A gains. E.g., A +9%, portfolio -9%. Ensure our result reflects negative total\_return and Sharpe is negative (since cagr negative).
  - Check that risk\_metrics still show 100% concentration, and scenario analysis for "All -10%" (which is actually a 10% drop in A price) yields a positive P&L for a short (because short would profit if asset falls). Specifically, if short 100% A, scenario "All -10%" should yield +10% pnl\_pct approximately. We verify the sign is handled: our scenario formula uses weights, which for short is negative, so negative \* (-10% change) = + positive result, as expected.
- Test partial cash:
  - Thesis: 50% long A, 0% elsewhere (so effectively 50% cash). The portfolio should earn roughly half of A's return. If A +10%, portfolio +5%. We verify that the code not normalizing weights means only 0.5 weight active, so daily\_portfolio\_returns = 0.5 \* ret\_A, effectively 0.5 portion in cash (0 return) is automatically accounted. Confirm total\_return ~5%. Also check that volatility is half of A's, etc. This ensures our handling of <100% allocation (cash remainder) is correct (we might explicitly mention that any unallocated portion is assumed in cash, so returns diluted).
- Metrics sanity:
  - For all tests, ensure no division by zero or weird values (like if constant prices, vol=0, our code yields sharpe inf – in tests, we can detect result.performance['sharpe'] and if it's Python inf, consider that acceptable given design or decide to output 0).

- Confirm max\_drawdown: In monotonic increase, drawdown 0%. In monotonic decrease, drawdown = total loss (like 9%). In a mixed or volatile series, test a known pattern: e.g., if asset goes 100 -> 80 -> 120 (down 20%, then up 50%), the max drawdown is 20% (peak 100 to trough 80) even though final return +20%. We can simulate that in data and verify result max\_drawdown\_pct ~20.
  - Scenario output:
    - For scenario “All -10%” in a long-only portfolio, expect ~ -10% result (subject to weight if not fully invested). We ensure the pnl\_pct in output corresponds. We should also ensure pnl\_abs = pnl\_pct/100 \* final portfolio value (which our code sets).
    - We can add a scenario specific to an asset: e.g., test that if portfolio is fully in A, and we simulate “All -10%”, that is exactly -10% P&L. Or if we had a scenario “Asset A -10%” (we didn’t implement asset-specific scenarios here, but if needed, we could).
  - **Edge:** If trading days count is less than one year, CAGR calculation might overshoot. E.g., 10 trading days ~0.04 years; if final\_value=1.09 (9% in 0.04yr),  $cagr = (1.09^{(1/0.04)} - 1)$  which is huge (~109%). That’s not meaningful for short period. But that’s how CAGR formula works. In our test on short series, we’ll see large cagr. We might choose to not compute CAGR if period < 1 year (or still show it, maybe the PM knows to interpret carefully). We’ll note in documentation that for very short periods, CAGR can be misleading. In a typical use, we hope to use at least ~1 year of data.
  - Confirm ThesisEvaluationResult structure: we expect performance keys ‘total\_return’, ‘cagr’, etc. Test ensures those keys exist and values type float. Check that timeseries list length equals input data length and first value = 1 (starting equity) or rather after cumprod, first value might still be 1 if we set initial day ret to 0. Actually, we fill NA with 0, so day0 ret = 0, equity day0 = 1, day1 = (1+ret1)\*1, etc. So yes, first equity point corresponds to start\_date with value 1.0. We might consider whether to drop the initial point (some would present equity from day1 onward). But including start as 1.0 is fine.
  - Risk\_metrics keys: ‘max\_weight\_pct’, ‘VaR\_95’, ‘max\_drawdown\_pct’. Confirm presence. We know in our code some might not always be present (if weights empty or series empty). Our validation and raise should prevent empty series. If weights empty (no legs), we raised earlier. If one asset, VaR\_95 exists as that one’s daily ret 5th percentile. If just a few days of data, percentile might equal min or so. That’s fine.
- If feasible, also test integration with DataAccess: We might simulate linking DataAccess with ThesisEvaluationService in an integration test:
    - e.g., DataAccess could have a method evaluate\_and\_store\_thesis(thesis\_id) that orchestrates these calls. But that’s E4 orchestrator territory. For E2, focus on the service itself.

## E2-Risks & Dependencies:

- *Data availability & quality:* If price data has gaps or differing date ranges, results can be skewed. Our approach of inner joining on dates means if one asset had missing days (maybe it was a holiday for one market), those days get dropped entirely, which could slightly bias results. A more sophisticated approach is forward-fill missing prices or assume no movement on missing days. We simplify by requiring continuous data. This risk is manageable in evaluation context by choosing assets from same market or cleaning data. We document this assumption that data is aligned (for multi-asset theses across regions, one might fill holidays).
- *Using 252 trading days for annualization:* This is standard in equities. If assets are not equities (e.g., daily data of any financial series, 252 is still a fair approximation for trading days per year). It's a small assumption. If we had monthly data, formula differs, but we assume daily frequency.
- *Short positions margin:* Our backtest implicitly assumes we can short without cost and we have infinite margin (no margin interest or constraint). In reality, shorting would require borrowing stock and possibly paying interest or being subject to margin calls if it goes up a lot. We ignore these complexities. We note that for extreme moves (stock goes up a lot, short position could theoretically lead to >100% loss), our model can handle beyond -100% equity (equity\_curve could go to negative if shorts not capped). Actually, a negative equity might appear if losses exceed initial capital (which is possible in shorting). Our simulation would show equity dropping below 0, which is mathematically possible in our cumprod and indicates bankruptcy. But our model doesn't cut it off. This is fine for theoretical evaluation, but a real risk system would cut off at -100%. We'll accept that limitation and maybe avoid test cases where short blows up completely (which would give weird results). For instance, if short and asset doubles, equity goes negative in our calc. We might clamp equity at a small positive to avoid math issues (though cumprod of negative returns still works and can go negative). We will at least highlight that a negative equity value in timeseries means strategy would have been bankrupt (which is a risk itself).
- *Transaction costs and slippage:* We ignore them. So our performance might be slightly optimistic. In evaluation, it's okay; if needed we could include a small cost assumption (like subtract 0.1% each rebalance), but probably not necessary now.
- *Model oversimplification:* Daily rebalancing is not how a discretionary PM operates (they might buy once and hold). Rebalancing means our strategy realization is that of a continuously managed portfolio. If an asset skyrockets, we sell some to maintain weights; if it plummets, we buy more. This can sometimes lead to better risk control or worse performance depending on trend. But we choose it for consistency and ease. We could mention this as a difference: "We assume continuous rebalancing to target weights daily, which may not reflect actual trading frequency of the thesis." It's an evaluation compromise that yields a smooth representation of the strategy defined by weights.
- *Outputs formatting:* We must ensure the output from ThesisEvaluationService can be easily serialized (for UI or LLM). Pydantic models handle that – e.g., ThesisEvaluationResult.performance is a dict of float, fine for JSON. timeseries is list of EquityPoint which Pydantic can turn into list of dicts like {"date": "...ISO8601...", "value": 1.234}. We should ensure date serialization is consistent (Pydantic will likely

ISO-format datetimes). Alternatively, we might convert dates to strings or timestamps for consistency. We'll rely on default for now (ISO8601 is fine).

- *Integration with Phase 3 (LLM)*: The LLM will see these metrics and timeseries. We should consider summarizing them before feeding to LLM to avoid prompt length issues. Possibly the context builder will not pass every point of timeseries, maybe just performance summary and perhaps a few scenario outcomes. We'll plan accordingly in E3 (LLM) research. The key is that the evaluation result is structured, so we can pick and choose fields for LLM or UI. No hidden coupling: everything the LLM needs from quant side is encapsulated here.
- *Parameterization*: We might consider making rebalancing frequency adjustable or scenario definitions customizable, but not needed now. We take daily and two example scenarios as fixed for demonstration.
- *Unit vs integration tests*: The heavy use of pandas means we rely on that library's correctness. We'll mainly test that our usage is correct (like weights applied properly). There's a slight risk of floating point drift in cumprod if a very long series – but usually negligible. For consistent tests, we can tolerate tiny differences (like if expecting 5.00% and get 4.999%, we round or assert math.isclose).
- *Performance*: Evaluating long time series (e.g., 10+ years daily data for multiple assets) in pure Python/pandas is fine; it's vectorized so quite fast. The LLM critique later may be slower; our quant evaluation likely is not the bottleneck. For perspective, 2520 trading days \* 10 assets handled in milliseconds. So no issues in an evaluation environment (and no need to pre-optimize). If needed, we can mention the possibility of caching price data between evaluations if the same asset is evaluated repeatedly, but probably unnecessary now.
- *Data dependency on external APIs*: If we were to fetch from Yahoo/AlphaVantage in real time, tests could become flaky or slow. Instead, for consistency we'll use static test data or local CSV. This isolates tests from internet. In a deployed scenario, we would integrate an API or DB, but not in the evaluation harness context unless provided by the environment. We'll clarify that PriceSource should be implemented with actual data for real usage; for now, we simulate it.
- *Completeness of metrics*: We might not capture all a PM might want (like beta to S&P, etc.), but the requirement says minimal quant outputs, and our chosen set is standard and sufficient for demonstration.

By implementing E2 as above, we turn a thesis into concrete quantitative evidence. We also ensure the output is rigorous enough to catch the LLM's attention: e.g., if Sharpe is low or max drawdown high, the LLM in E3 can remark on that. The structured output sets the stage for a meaningful LLM critique rather than a generic one.

## **E3 – LLM Tools & Evaluation Harness**

### E3-Requirements:

- Design a suite of **LLM-enabled “tools”** to analyze and summarize Slice’s data, each producing structured outputs defined by Pydantic schemas:
  1. **ThesisReview**: Given a thesis description and its quant evaluation (from E2), have the LLM critique the thesis, list questions, and flag risks, outputting a ThesisReview object. This tool mimics an expert analyst reviewing the thesis .
  2. **CrossThesisReport**: Given multiple theses, have the LLM compare them for overlaps or contradictions, outputting a CrossThesisReport (e.g., a PM wants to see if their theses are diversifying or inadvertently doubling down on the same bet).
  3. **IntuitionAnswer**: Given a question and a set of observations (news/data points), have the LLM answer the question using only that context (like a mini Q&A chatbot for the PM’s data), returning an IntuitionAnswer. This should include references to which observations were used, to maintain transparency.
  4. **DailySummary**: Given a “DailyContext” (which might include that day’s portfolio changes, alerts, notable observations), have the LLM produce a concise summary of what happened and key points to note, outputting a DailySummary. This emulates a daily newsletter generated for the PM.
- **Schema enforcement**: The LLM must output in a JSON schema for each tool, to eliminate free-form unpredictability. We define Pydantic models for ThesisReview, CrossThesisReport, IntuitionAnswer, DailySummary with specific fields. The LLM prompt will instruct to output *only JSON* conforming to that schema . Our code will parse the LLM response and validate it against these models. If parsing fails or content is missing, we treat it as a contract violation (error).
- **Grounding and hallucination control**: The LLM should be constrained to use only provided information. We implement this by robust prompting: e.g., “Use **ONLY** the given data. If you lack info, set `insufficient_context=true`” . Each model includes an `insufficient_context`: bool field that the LLM will set to True if it cannot complete the task with given inputs. For instance, if we ask a question unrelated to the provided observations, the IntuitionAnswer should indicate insufficient context instead of making something up .
- **No external calls** from LLM: The tools get all needed context as function arguments. E.g., `llm_review_thesis(thesis, evaluation)` receives the raw thesis text and key metrics as input context – the LLM will not query the DB on its own. This isolation makes behavior deterministic and testable.
- **Evaluation harness**: Develop test routines (maybe as special “dry-run” LLM clients or dummy models) to verify that:
  - Each tool’s output can be parsed into the correct Pydantic model (the JSON structure is correct).
  - The outputs do not contain references to things outside the provided context. We will cross-check IDs and tickers in the output against inputs to catch hallucinations. If any are found, mark `insufficient_context` or error.
  - The `insufficient_context` flag is correctly triggered. For example, call `llm_query_intuition` with an empty observations list and a question – the expected output’s `insufficient_context` should be True and maybe a null answer.



- **Metrics & Observability:** Track usage of each LLM tool: how many calls, average latency, error count. This will help evaluate performance overhead of LLM integration. We'll instrument each function to record latency (provided by SessionOrchestrator) and success/failure, and expose these stats via an API or log for debugging .
- Implement these tools in a way that they can be plugged into the overall pipeline (Phase 4 orchestrator will call them). The LLM interactions should be encapsulated (likely via a SessionOrchestrator or direct OpenAI API calls internally), so replacing the LLM backend or model (e.g., GPT-4 vs GPT-3.5) is easy.

### E3-Research Summary:

Large Language Models can be powerful analysis assistants, but they require careful prompt engineering and output validation to be reliable in a finserve setting. We leverage known techniques for **structured prompting** and **guardrails**: instructing the model to output JSON and using a tool like Pydantic or **Guardrails AI** to validate it . OpenAI's newer function calling/structured output feature directly supports JSON schema enforcement, virtually guaranteeing a correct format . If available, we would use that: define the Pydantic schema, have the OpenAI API call with that as expected output. This dramatically reduces the chance of formatting errors (the model is forced to comply or API will error). If not using that feature, we at least include the JSON schema in the prompt and double-check on our side.

To minimize hallucinations, we use two strategies:

1. **Context limitation:** Provide only relevant info to the model and explicitly forbid using outside knowledge . For instance, for IntuitionAnswer, we feed the model a bunch of retrieved observation texts and say "If the answer isn't in these, say you don't know." This is a core idea in retrieval-augmented generation (RAG) and stops the model from making up facts .
2. **Insufficient context signaling:** We instruct the model to explicitly acknowledge when data is missing by setting a flag or saying "I don't know." This is recommended in prompt engineering to handle gaps . We integrated insufficient\_context field in each output schema so the model has a formal way to indicate it. For example, our daily summary prompt will say, "If you are not given certain data, set insufficient\_context=true in the JSON." This approach is in line with best practices to reduce hallucination by encouraging abstention when unsure .

We will also incorporate other prompt best practices:

- **Role prompting:** We set the system persona (e.g., "You are an expert macro analyst...") to guide style and rigor.

- **Explicit formatting instructions:** e.g., “Respond ONLY with a JSON object as follows: ...” to prevent any extra text .
- **Examples** if needed: we could include a brief example in the prompt if the model tends to misbehave (though with function calling mode, example might not be needed).
- **Toolformer approach:** We essentially are making the LLM into a function that outputs a certain format – akin to OpenAI function calling which we treat as our primary approach if possible .

We incorporate a dummy LLM for testing (maybe a simple function that returns a pre-baked JSON given an input). This allows us to test our parsing and validation logic without hitting the real API (which is costly and nondeterministic). For example, we can simulate that for a known thesis input, the LLM returns a certain JSON string we craft, and then ensure our code produces the correct Pydantic object from it.

Observability: LLM calls can introduce latency and potential failure points. Best practice is to monitor these as part of system performance . We’ll collect metrics like count and latency percentiles. E.g., if ThesisReview calls start taking >5s, we want to notice that. We’ll store and later display such diagnostics (Phase 6 UI can show them for evaluation).

### E3-Design:

- **Pydantic Schemas:** (in src/slice/models/llm\_outputs.py)

```
class ThesisReview(BaseModel):
 critique: str
 questions: list[str]
 risk_flags: list[str]
 insufficient_context: bool

class CrossThesisReport(BaseModel):
 overlaps: list[str]
 contradictions: list[str]
 gaps: list[str]
 insufficient_context: bool

class IntuitionAnswer(BaseModel):
 answer: str
 references: list[str] # e.g., observation IDs used
 insufficient_context: bool

class DailySummary(BaseModel):
 key_narratives: list[str]
 risk_highlights: list[str]
 thesis_references: list[str] # e.g., thesis IDs mentioned
```

```
insufficient_context: bool
```

- Each has an `insufficient_context` field. We considered defaulting it to `False`, but we want the model to explicitly decide and output it, so it's required in output. (If using OpenAI function calling, we'll get either `True` or `False` in JSON every time, which is fine.)
- **LLM Orchestrator Integration:** We likely have a `SessionOrchestrator` class (from the code scan) that wraps OpenAI API calls. It returns a `SessionResponse` with `content` (the LLM message) and `latency_ms`. We will leverage that: e.g.

```
orchestrator = SessionOrchestrator(openai_api_key, model="gpt-4")
options = SessionOptions() # might include temperature, etc.
response = orchestrator.run_session(prompt, options)
raw_content = response.content
```

- Then we parse `raw_content` as JSON into the Pydantic model:

```
try:
 review = ThesisReview.parse_raw(raw_content)
except Exception as e:
 # If parse fails, we'll mark this as an error or attempt a fix.
 raise e
```

- This covers formatting issues. If `insufficient_context` came as `True`, the content of other fields might be empty or partial; that's fine – it's a valid outcome meaning the LLM couldn't fully answer.
- **Prompt Templates:** For each tool, define a function that constructs the prompt from inputs:
  - *ThesisReview:* We have thesis text fields (title, hypothesis, drivers, disconfirmers, expression legs) and evaluation metrics (perhaps use the `ThesisEvaluationResult` from E2). The context builder could produce a JSON like:

```
{
 "thesis": { "title": ..., "hypothesis": ..., "drivers": [...],
 "disconfirmers": [...], "legs": [{"asset":..., "direction":...,
 "size_pct":...}, ...] },
 "evaluation": { "total_return": X, "cagr": Y, "volatility": Z, "sharpe": S,
 "max_drawdown": D, "scenarios": [{"name":..., "pnl_pct":...,
 "pnl_abs":...}, ...] }
}
```

- - Or we can present evaluation in prose if easier (but JSON context is fine too). Since the LLM can parse JSON easily, giving it structured data might be effective. However, including an entire timeseries might be too verbose. We likely skip timeseries detail here (LLM doesn't need every daily point; key stats suffice).

Prompt might be:

System: "You are a financial analyst. You will critique an investment thesis using data provided. Only use the given information."

User: "Thesis details and evaluation results:

THESIS:

Title: ...  
Hypothesis: ...  
Drivers: ...  
Disconfirmers: ...  
Expression: ...

EVALUATION:

Total Return: X%  
CAGR: Y%  
Volatility: Z%  
Sharpe: S  
Max Drawdown: D%  
Scenario: All -10% -> Portfolio -M%, All +10% -> Portfolio +N%

Provide a JSON output with the following format:

```
{
 "critique": "...",
 "questions": ["..."],
 "risk_flags": ["..."],
 "insufficient_context": false
}
```

Critique the thesis (coherence, plausibility), list questions to explore, and flag any risks (from data or logic). If you lack sufficient info to do so, set `insufficient_context` true and make critique minimal."

- - This prompt ensures the model knows what to do and how to output. We emphasize JSON format. We explicitly tell it that if something is missing (maybe no drivers or disconfirmers given, or evaluation has gaps), then `insufficient_context` should be true. The model then might still fill critique with something like "Not enough data provided to assess performance." – which is fine, but the flag alerts us.
  - *CrossThesisReport*: Prompt this tool with summaries of each thesis. Possibly context like:

"Thesis A: Hypothesis..., Expression summary...  
Thesis B: Hypothesis..., Expression summary..."

- - and ask for JSON with overlaps, contradictions, gaps.

System prompt: "You are an AI analyzing relationships between investment theses."

User prompt lists each thesis. Then: "Compare these theses. Overlaps: where they have similar ideas or exposures. Contradictions: where they make opposing assumptions. Gaps: issues or angles neither addresses. Provide JSON: { overlaps: [...], contradictions: [...], gaps: [...], insufficient\_context: false }."

This guides the model to focus on differences and commonalities.

- *IntuitionAnswer*: We need retrieval. In practice, we would have already fetched relevant observations (like via vector search on embeddings). In our design, `llm_query_intuition(question, observations)` expects the caller (e.g. orchestrator or UI) to supply observations (probably as a list of Observation objects or texts). Our implementation will format these observations as context, for example:

```
"CONTEXT:
[Obs1] 2025-11-01: Federal Reserve raises interest rates by 0.25%.
[Obs2] 2025-11-03: Gold prices fall 2% as USD strengthens.
...
QUESTION: 'What is the outlook for inflation given recent central bank
actions?'"
```

- - We then instruct: “Answer using only the above observations. If the answer is not directly provided, say `insufficient_context`. Respond in JSON: { `answer`: “...”, `references`: [...], `insufficient_context`: false }. Put the IDs of any observations you used in ‘references’.”

This is a common RAG prompt style . We emphasize not to use outside info or guess. The references field is critical to trust: the PM can trace which observation backs the answer. We will validate those IDs. The model should ideally include any obs that support statements in its answer. We’ll have to rely on prompt adherence for that.

- *DailySummary*: The context (DailyContext) could contain:
  - maybe a list of today’s alerts (e.g., “Thesis X triggered disconfirmer Y”),
  - maybe changes in portfolio (like “Portfolio +0.5% today, primarily due to Gold rising”),
  - maybe top news (a couple of observations).

The LLM then should summarize: e.g., “Gold rose as inflation concerns increased, benefiting Thesis A. However, Thesis B suffered from rising yields...” etc.

Prompt:

```
"You are an AI that writes a brief daily summary for a portfolio manager. Use
the provided data (portfolio changes, alerts, observations) to write key
narratives and any risks to watch.
```

```
DATA:
```

```
- Portfolio: +0.4% today (Value $1,000,400).
- Alert: Thesis 'Gold vs Inflation' had a disconfirming event ('Inflation
higher than expected').
- Observation: US 10y yield rose to 3.5%.
- Observation: Gold price fell 1%.
```

REQUIRED OUTPUT (JSON):

```
{
 "key_narratives": ["...", "..."],
 "risk_highlights": ["...", "..."],
 "thesis_references": ["thesis_id1", "thesis_id2"],
 "insufficient_context": false
}
```

Only use the provided data. If not enough data for a meaningful summary, set `insufficient_context true`."

- - The model should then produce, say, two narratives (“Inflation surprise led to portfolio gains as gold hedge paid off”, etc.), some risk highlights (“Rising yields could pressure tech thesis...”), and list any specific theses by id that were mentioned. We include `thesis_references` so that if summary says “Gold thesis benefited”, it lists that thesis’s id, enabling UI to link to it. The model can get the thesis id from the context we provide (we will ensure to annotate context lines with thesis IDs if needed). If not, we can instruct to use thesis titles as references which we later map to IDs, but IDs directly is better if possible.

It’s tricky for the model to know the ID unless we include it in data (so we should supply something like “Thesis Gold vs Inflation (ID: T1): ...”).

- **LLM Tool Functions Implementation:** We create functions that wrap orchestrator calls:

```
def llm_review_thesis(thesis: Thesis, evaluation: ThesisEvaluationResult) -> ThesisReview:
 prompt = build_thesis_review_prompt(thesis, evaluation)
 start_time = time.time()
 resp = orchestrator.run_session(prompt,
options=SessionOptions(max_tokens=500))
 latency = resp.latency_ms or int((time.time() - start_time)*1000)
 raw = resp.content
 try:
 result = ThesisReview.parse_raw(raw)
 except Exception as e:
 # If parsing fails, attempt to fix minor issues if possible
 # e.g., if model included some explanation outside JSON, try to
extract JSON substring
 json_str = extract_json(raw) # a helper that finds first and last
brace
 try:
 result = ThesisReview.parse_raw(json_str)
 except Exception as e2:
 record_llm_call("thesis_review", latency, success=False)
 raise ValueError(f"LLM output invalid: {raw}") from e2
 # Basic post-validation: ensure references are within context if
applicable (not much to check for review)
 record_llm_call("thesis_review", latency, success=True)
 return result
```

- Similarly for other functions. Some notes:

- We measure latency either from orchestrator's provided value or manually.
- We use a helper `extract_json` to salvage JSON if the model added extraneous text (shouldn't happen with function calling mode, but if not using it, sometimes model says "Sure, here's JSON:" then JSON – we strip the prefix).
- After parse, for `IntuitionAnswer` and `DailySummary`, we do additional **reference validation**:
  - For `IntuitionAnswer`: after getting `result = IntuitionAnswer(...)`, do:

```
provided_ids = {obs.id for obs in observations}
out_ids = set(result.references)
unknown_ids = out_ids - provided_ids
if unknown_ids:
 # The model referenced an obs that wasn't given - hallucination.
 # We can remove them:
 result.references = [ref for ref in result.references if ref in
provided_ids]
 result.insufficient_context = True
 # Optionally, also modify answer: we might append " (insufficient
context)" or simply trust the flag.
```

- - - This way, if the model fabricated a ref, we drop it and mark that it didn't have full context to answer (because it relied on something outside given obs).

If the answer itself contains content not supported by any obs, we wouldn't catch that via IDs alone. But at least if it cites a non-existent source, we catch that. To detect unsupported statements, one could parse answer and check similarity to obs text, but that's complex. We'll rely on manual eval and context discipline to minimize that.

- For `ThesisReview` and `CrossThesisReport`: they don't have explicit references. We could do a lighter check, e.g., ensure that any ticker or asset mentioned was in thesis expression. But models might mention related factors not explicitly in input (like "liquidity risk" flag even if not provided – that might be a reasonable inference, not a hallucination per se). We primarily focus on concrete references like IDs or tickers:
  - If `review.critique` or `risk_flags` mentions an asset not in thesis, that might be a hallucination. E.g., thesis about gold and model says "Risk: rising oil prices" which is unrelated. But maybe the model assumed something. Hard to systematically catch. Perhaps skip heavy validation there; domain experts will judge if critique is nonsense. We'll just parse it.
- For `DailySummary`: `thesis_references` we verify similarly to `IntuitionAnswer`:

Provided an input list of theses (with ids) in context, ensure each id in summary references is in that set. If model included an id that wasn't provided, remove it (though if it wasn't provided, likely means it hallucinated a thesis or got an ID wrong).

- **Error handling:** If any step fails (e.g., OpenAI API error or parse error we can't fix), we propagate an exception upward (the orchestrator or session will catch and handle it by marking session failure). We also record it in metrics as a failure.
- **record\_llm\_call:** a function to update a dict like:

```
llm_stats["thesis_review"]["count"] += 1
llm_stats["thesis_review"]["total_latency"] += latency
if success: llm_stats["thesis_review"]["success"] += 1 else
llm_stats["thesis_review"]["errors"] += 1
```

- - Then avg latency can be computed or stored as well. We'll do similar for cross\_theses, intuition\_qa, daily\_summary keys.
  - Possibly wrap each call in a retry if format not correct: E.g., after parse fail, we could attempt to re-prompt the model: "Please output JSON only." But that doubles cost and complicates flow. Instead, easier is to try to fix minor format issues ourselves (like removing trailing text). We'll implement minimal fixes. If it still fails, treat as error (which in evaluation, we'll see and can adjust prompts accordingly).
  - **OpenAI Model selection:** likely use GPT-4 for better compliance and analysis quality. If using 3.5, might need more prompt strictness and examples. We allow config via orchestrator. Our code doesn't hardcode model except perhaps as default in orchestrator initialization. The SessionOptions can set temperature=0.2 (a bit deterministic) for consistent outputs. Low temp so it doesn't wander.
- **Contract Testing (with Fake LLM):** Instead of calling real API in tests, we can monkeypatch orchestrator.run\_session to a dummy function that returns a predetermined JSON. For example:
  - For llm\_review\_thesis, we create a fake response content:

```
{
 "critique": "The thesis is logically sound but relies on a single
assumption of declining inflation.",
 "questions": ["What if inflation does not decline as expected?"],
 "risk_flags": ["High concentration in gold", "Inflation forecast risk"],
 "insufficient_context": false
}
```

- - Our test will patch the orchestrator to return this string for any prompt, then call llm\_review\_thesis(thesis, eval) and expect a ThesisReview object with those fields. We assert that critique text and lists match exactly, and insufficient\_context is False.



Also test a branch with `insufficient_context` True:

e.g., patch orchestrator to return:

```
{"critique": "Not enough data to assess.", "questions": [], "risk_flags": [],
"insufficient_context": true}
```

- - and ensure our function returns that model with flag true and does not error. Possibly the orchestrator might not return such by itself, but we simulate as if it did.
  - For `llm_query_intuition`, supply observations context and patch orchestrator to respond with an answer JSON. Example:

Input observations: [“Obs1: inflation up”, “Obs2: gold down”]. If question “What happened to gold?”, we fake model output:

```
{"answer": "Gold fell due to rising inflation causing yield increases.",
"references": ["Obs1", "Obs2"], "insufficient_context": false}
```

- - After our `llm_query_intuition`, check `IntuitionAnswer` object:
    - answer matches,
    - references contains “Obs1”, “Obs2”,
    - `insufficient_context=False`.

Also test hallucination filter:

- If fake output references [“ObsX”] not in provided obs IDs, our function should remove “ObsX” and set `insufficient_context=True`. We simulate that and assert the changes. For instance, provided obs IDs = {“Obs1”, “Obs2”}, model output references [“Obs1”, “Obs3”]. After function, expect references [“Obs1”], `insufficient_context` True.
- If model output had an unknown reference, that means it hallucinated some source or got ID wrong. We then mark incomplete context. It’s a bit aggressive (maybe it just had a typo but info was from given obs). However, such mismatches are rare if we number obs sequentially.
- For `llm_daily_summary`, we can craft a scenario:

Input context: portfolio up, an alert about Thesis1, etc. Patch model to output:

```
{"key_narratives": ["Portfolio rose mainly due to gold gains."],
"risk_highlights": ["Watch for rising yields affecting USD thesis."],
"thesis_references": ["T1"],
"insufficient_context": false}
```

-

- After `llm_daily_summary(context)`, expect a `DailySummary` with those fields. Also test insufficient context scenario: if context was empty and model returns `insufficient_context true` JSON, ensure we handle it (just parse it).
- For `llm_cross_theses`, simulate:

Theses inputs: e.g., Thesis A (USD long) and Thesis B (USD short). Fake output:

```
{
 "overlaps": ["Both involve USD assumptions"],
 "contradictions": ["One expects USD to strengthen, the other expects it to weaken"],
 "gaps": ["Neither addresses Fed policy impact"],
 "insufficient_context": false
}
```

- 
- After `llm_cross_theses([thesisA, thesisB])`, check `CrossThesisReport` fields match. If we provided no theses or identical theses, maybe `insufficient_context` scenario can be tried.
- Parse error simulation:

If the model returned something like `"{ \"answer\": \"...\" } Some trailing text."`, our `extract_json` should fix it. We test that specifically: patch orchestrator to give that string, ensure our function still returns correct `IntuitionAnswer` (and log a success).

Also test a truly malformed JSON: patch orchestrator to return `"Not JSON"`. Our code should then raise an error (`ValueError`). We catch it in test and assert that error is thrown and metrics recorded as failure. This tests our error handling path.

- Metrics:

After each fake call, we check that `llm_stats[...]` entry updated correctly (calls count increment, latency recorded, etc.). We might simulate two calls and see that avg latency computed matches. If our record function stores sum and count, we could compute avg in diagnostics API. For test, just ensure the counts and errors values move as expected when we simulate success or forced error.

- Ensure that after a normal call, `insufficient_context` `False` remains false (i.e., if no issues), and if model sets it true or we set it due to reference check, it is true. We verify our code does not override the model's decision incorrectly (we only override to true if we detect a problem; we never set something to false that model indicated true).

## E3-Risks & Dependencies:

- *LLM Reliability*: Even with structure, the quality of content depends on the model. If the model outputs subtly wrong info but in the right format, our pipeline will pass it. For example, ThesisReview might say “Sharpe is high” when it’s actually low – a misinterpretation. Our system wouldn’t catch that; it’s a correctness issue beyond format. We partly mitigate by giving the model the actual numbers (reducing chance of error) . But some higher-level judgments could be subjective. This is where PM oversight is needed. Since this is an evaluation harness, highlighting these capabilities is more important than guaranteeing all content is correct. We’ll note that the PM should verify LLM outputs and that we focus on eliminating obvious errors (format, hallucination).
- *Parsing and Schema limitations*: The Pydantic models require all fields present. If the model omits a field, parse will fail. For instance, if it forgets risk\_flags, we treat as invalid. We might consider making fields optional with default empty (especially in CrossThesisReport where maybe no overlaps found -> could model omit field or give "overlaps": []? Likely it will give empty list, which is fine). We defined them as required expecting model always outputs them (with possibly empty list). Given function calling mode or our prompt explicitly listing them, the model usually includes them even if empty. We should be safe, but if not, our code raising error will surface it. We prefer strictness to catch any format deviance.
- *Model verbosity*: There’s a risk the model might include explanatory text outside JSON if it doesn’t follow instructions (especially GPT-3.5). Our use of system prompt and explicit JSON demand should minimize that. We also can use OpenAI’s function calling to eliminate that risk (the API will directly give parsed JSON). If our environment allows that (we need to pass a schema to the call). That might complicate our orchestrator usage. Alternatively, we stick to prompting but keep an eye. We implemented extract\_json as fallback for minor extraneous text. If model goes off rails (like a long essay), our parse will fail and we’ll catch it. In evaluation, a failure like that indicates prompt needs adjustment or to use a better model. We’ll mention that we assume usage of a model version that supports structured output well (GPT-4 or GPT-4 function calling) .
- *Cost and speed*: LLM calls, especially GPT-4, are slow (1-5 seconds) and costly. In an evaluation harness, that’s acceptable given likely small number of calls (one per session step). We track latency to ensure it’s not too high. If needed, we could throttle or use shorter prompts (e.g., don’t feed entire timeseries to LLM). We already limited LLM inputs to key stats. This dependency on an external API means internet connectivity or local model usage. If environment restricts external calls, we might have to use a stub or ensure to have results cached. For our design, we assume we can call OpenAI’s API (or at least that’s conceptually available).
- *Resource constraints in testing*: We won’t actually call the real API in automated tests (due to unpredictability and cost). We’ll rely on stubs. So our tests won’t truly confirm the OpenAI integration, but will confirm our processing around it. Before deployment, a manual test with actual API is needed to fine-tune prompts. We treat that as a given interactive step.
- *LLM Changing Behavior*: As models update, they might produce slightly different output. Our structure expectation should hold, but minor differences in phrasing inside critique or ordering of list items might occur – that’s fine. The schema doesn’t constrain content beyond type. We only worry if it stops following JSON format. With function calling, that shouldn’t happen at all.

- *Language and domain knowledge:* The model might not know some specific financial terms or tickers if they are obscure (though GPT-4 knows a lot, but e.g., if a thesis is about a niche index, it might hallucinate knowledge). By providing context (like key metrics, or if observation text includes details), we mitigate some. We also direct it to focus on given info. But if insufficient, it might produce generic answers or mark `insufficient_context`. That's acceptable (better to admit not knowing). The PM can then supply more info or just note that the LLM couldn't add value there.
- *Ensuring no DB or external calls by LLM:* Our instructions will strongly forbid it (and the model in API can't actually query DB, it only sees what we send). So we're fine. Just making sure we don't accidentally encourage it by something like "`<#SQL> ...`".
- *Chaining LLM tasks:* We currently have each tool separate. But consider daily summary: it might implicitly use LLM's analysis ability across theses and obs. That's fine as one prompt. Or `cross_theses`: if many theses, the context could be long. We must ensure the prompt size doesn't exceed model input limit (~8k tokens for GPT-4). If someone has, say, 10 theses each with a long hypothesis, the prompt might approach a few thousand tokens – still okay. If more, we may need summarization or to drop some details (like just use titles and one-liner summary per thesis). For now, assume manageable number of theses in comparisons (the user likely wouldn't compare more than, say, 5 at once).
- *Observations length:* If an observation text is large (like a paragraph or full article), feeding many of those into `IntuitionAnswer` could be huge. We likely need to pre-truncate or summarize observations (e.g., store only key sentences). A typical approach: retrieve top relevant sentences. Our design currently takes a list of `Observation` objects. We might decide to pass only `obs.text[:200]` chars for each to limit size, or trust that context builder will do that. We'll note that large observation sets should be filtered by a retrieval step (embedding similarity). That retrieval system is presumably part of the "memory app" in original Phase 6. But to keep this focused, we assume that we only pass relevant observations (the orchestrator or context builder would have done retrieval).
- *Open questions:* We integrated `insufficient_context` flags in output. But how do we use them? The orchestrator in Phase 4 might decide:
  - If `ThesisReview.insufficient_context` is true, maybe mark the thesis as needing more data or prompt user to input missing data. But at least it tells the PM the LLM isn't confident (which is better than a hallucinated answer). In our pipeline, we might just display the critique (which might be minimal) and note that context was insufficient (the UI can e.g., highlight that flag). Similarly, daily summary with `insufficient_context` means "we don't have enough info to make a summary today", which could happen if no new data.
  - It's good to present that to user rather than either silent fail or nonsense summary.
- *Alternate approach with guardrails lib:* We could integrate guardrails (a library to enforce schema and even check content for hallucinations by verifying references) . But given our environment, we stick to manual validation and `Pydantic`.
- *Public figure / PII in output:* Possibly irrelevant – our use is internal financial analysis. Observations might have names of Fed chair etc., which is fine. LLM might mention them (e.g., "Powell signaled..."). That's okay. There's no policy violation on that. We instruct only use context, so if name was in obs, it's allowed.

In summary, E3 equips our system with a structured LLM evaluation layer, turning raw data into insightful narratives, questions, and warnings, all while keeping a tight leash on the LLM to avoid missteps. This provides significant value-add (explanations, qualitative insight) on top of the purely quantitative output, in a controlled and testable manner.

## E4 – Orchestrator & End-to-End Sessions

### E4-Requirements:

- Build orchestrator logic to tie together data retrieval (Phase E1), quant evaluation (E2), and LLM analysis (E3) into coherent **session workflows** that represent major user operations. The primary sessions include:
  - **ThesisEvaluationSession**: End-to-end evaluation of a single thesis. Steps:
    1. Load thesis (and any associated observations, if needed for LLM context) via `DataAccess`.
    2. Run `ThesisEvaluationService` to get quant results.
    3. Run LLM Review on the thesis and quant results to get a `ThesisReview`.
    4. Optionally, generate a trade plan proposal (E5) for this thesis (if in scope for evaluation phase).
    5. Persist the evaluation outcome: store `ThesisEvaluationResult` and `ThesisReview` in the database (or an in-memory log) for future reference (so UI can display it without rerunning).
    6. Aggregate results into a `ThesisEvaluationSessionResult` DTO and return it. This result contains everything (thesis id, evaluation metrics, review content, and maybe a proposed trade plan).
  - **DailyUpdateSession**: A workflow for daily portfolio monitoring. Steps:
    1. Load all active theses and current portfolio state (positions) via `DataAccess`.
    2. Compute an updated risk snapshot (`DataAccess.get_portfolio_depth`) and note any changes (like concentration changes or breaches).
    3. Identify any **alerts**: e.g., if a new observation came in that triggers a thesis's disconfirmer (from E1's Observation repository and thesis definitions). Mark those as alerts for the day.
    4. Prepare a `DailyContext` (which includes portfolio changes, risk snapshot, and alert highlights, maybe major market observations).
    5. Run LLM `DailySummary` with that context to get a `DailySummary`.
    6. Persist or log the alerts and summary (e.g., store alerts in an `Alert` table, store summary text or simply be ready to serve it via API).
    7. Return a `DailyUpdateSessionResult` containing the summary and any alerts.

- Possibly **CrossThesisAnalysisSession** (though not explicitly in requirements, but since we have CrossThesisReport, maybe triggered by user on-demand).

If implemented, it would gather selected theses and run `llm_cross_theses`, returning that report.

- Possibly an **IntuitionQASession** if needed (or Intuition can be direct API).
- Define **API endpoints** for triggering these sessions:
  - POST `/api/v1/thesis/{id}/evaluate` -> triggers ThesisEvaluationSession and returns its result as JSON .
  - POST `/api/v1/session/daily-update` -> triggers DailyUpdateSession, returns summary/alerts.
  - Optionally, POST `/api/v1/thesis/compare` (with body listing thesis IDs) for cross-thesis, returns CrossThesisReport.
  - POST `/api/v1/intelligence/qa` (with question and optional filter) for intuition Q&A, returns IntuitionAnswer (though this might already exist as per codebase `intel_routes`).
- Implement these endpoints in FastAPI by orchestrating calls to our services and handling exceptions:
  - On success, return 200 with structured result (the Pydantic model's dict).
  - On failure (e.g., LLM error or invalid input), return appropriate HTTP error (400 for bad input, 500 for internal error) along with message.
- The orchestrator should ensure **error isolation**: if one component fails, handle it gracefully rather than leaving partial data:
  - E.g., if LLM review fails but quant succeeded, we might still return the quant results and an indication that review is unavailable (or we could treat the whole session as failed). For evaluation phase, might treat it as failure because user expects the combined output. But we could include a field in result like `review`: null or a flag to indicate LLM step failed. Simpler: propagate error (500) so user can retry or know it didn't complete fully.
  - If DataAccess fails (thesis not found), respond 404 early.
  - If quant fails (e.g., missing price data), probably a 400 because input can't be evaluated.
  - We'll catch exceptions from each step and decide how to report.
- **Transaction management**: When persisting results, we want either all-or-nothing. For example, if we insert an EvaluationRecord and then LLM fails, we might roll back the insert to not have orphan quant data without review. Using the same DB session with rollback on error would achieve that . If using independent inserts, we'd explicitly delete partial on error or mark it incomplete. We prefer to not save anything unless fully successful.
- Provide a unified **session result DTO** for each session type to encapsulate results:
  - ThesisEvaluationSessionResult: fields: `thesis_id`, ThesisEvaluationResult, ThesisReview, TradePlan (optional), maybe timestamp.
  - DailyUpdateSessionResult: fields: `date`, DailySummary, list of Alert objects (with `thesis_id` and message), maybe portfolio snapshot or key metrics after update.

- Write **integration tests** that simulate the entire flow on a sample scenario to ensure all pieces E1–E3 (and E5 if included) work together:
  - Example: create a thesis in test DB, simulate price data, run the `/thesis/{id}/evaluate` endpoint (with LLM stubbed) and verify the JSON includes combined info (metrics from quant and text from LLM) . Also confirm the DB now has evaluation records if we expected them to be saved.
  - For daily update: simulate trades in DB, add an observation that should trigger an alert, call the endpoint, verify response summary mentions the alert and an Alert entry is saved.
  - Essentially, do a dry-run of the “Use Stage Script” (which is required) through the API endpoints programmatically, and assert expected outcomes at each step.

#### E4-Research Summary:

This phase is about implementing a workflow akin to a **Saga** or orchestrated pipeline, but within a single service context. In microservice architectures, the Saga pattern ensures that if a multi-step process fails mid-way, compensating actions roll back previous steps . In our case, since all steps are in-memory calls and DB writes, we can manage consistency with transactions or careful order of operations.

We aim for each session to be **idempotent** or at least not damaging if repeated. For example, running thesis evaluation twice should either produce two records (maybe only latest matters) or overwrite the previous evaluation to avoid duplicates. We might choose to **update** an existing evaluation record for that thesis if one exists (so there’s one canonical latest evaluation per thesis). Or we give each evaluation a timestamp and keep history (which could be useful for audit). For simplicity, we might replace the old one or mark it superseded. We’ll define in design: likely one active evaluation per thesis (overwriting previous), because storing lots of evaluation results isn’t required unless explicitly needed. If evaluation is fast, PM can just re-run. We’ll implement either update or a delete-then-insert.

Testing and delivering these orchestrations require treating them like **integration tests** in the testing pyramid: they test multiple units together . We’ll do exactly that, as said, by simulating a full scenario (like loading a test DB state and calling the API as a client would) to see if everything links properly – the repository returns correct data to evaluation, evaluation output flows to LLM properly, etc. This ensures our contracts (from E1, E2, E3) align.

We also incorporate **observability** at the session level: we have LLM metrics and perhaps we should track session metrics (like how long an entire thesis evaluation session took, and was it

successful). We can log that or include in a diagnostics. But since requested in E3, we likely expose primarily LLM metrics. Maybe also track number of sessions run, and success vs fail, to see reliability.

### Design trade-offs / decisions:

We had to decide what to do if LLM step fails but quant succeeded. It's arguably better to still save the quant result so at least the PM sees performance numbers, even if critique is missing. So we might implement partial persistence: always save quant evaluation; if LLM fails, mark review as failed. We can reflect that in the UI by showing quant metrics and a note "LLM review unavailable." However, the requirement goal likely expects the full pipeline or error. For an evaluation harness, either approach is acceptable as long as we note which approach we took.

We'll aim to **not silently ignore** any failure. Every error should either result in an HTTP error or an explicit field in output that signals an issue. This transparency is key in evaluation: if LLM had a problem, the user should know (rather than just seeing missing data and being confused).

### E4-Design:

- **Session Classes and Contracts:**

We can implement sessions either as classes with a run() method or as simple functions inside the orchestrator module. Classes may hold context like data\_access, services, etc. We'll do classes to mirror how one might extend or configure sessions easily.

*ThesisEvaluationSession:*

```
class ThesisEvaluationSession:
 def __init__(self, data_access: DataAccess, eval_service:
ThesisEvaluationService, llm_tools: LLMTTools, exec_adapter:
PaperExecutionAdapter = None):
 self.data = data_access
 self.eval_service = eval_service
 self.llm = llm_tools # llm_tools could be just a namespace or object
with methods llm_review_thesis, etc.
 self.exec_adapter = exec_adapter # optional, for trade plan
generation
 def run(self, thesis_id: str) -> ThesisEvaluationSessionResult:
 # Step 1: Load thesis
 thesis = self.data.get_thesis(thesis_id)
 if not thesis:
 raise ValueError(f"Thesis {thesis_id} not found")
```



```

 # (We might also load related observations if we want to provide them
to LLM, but not strictly needed for review - review mainly needs thesis &
eval)

 # Step 2: Quant evaluation
 try:
 eval_result = self.eval_service.evaluate_thesis(thesis)
 except Exception as e:
 # e.g., missing price data => can't evaluate
 raise e # propagate for now (caller will turn into 400/500)

 # Step 3: LLM critique
 try:
 review = self.llm.review_thesis(thesis, eval_result)
 except Exception as e:
 # LLM failed - choose to continue or not. We'll mark review as
None and continue to plan, but ultimately likely return error.
 review = None
 review_error = str(e)

 # Step 4: Trade plan proposal (if exec_adapter provided)
 trade_plan = None
 if self.exec_adapter:
 try:
 # Use a default notional or from thesis settings
 notional = 1000000.0 # 1M default for plan, or
get_current_portfolio_value if deploying whole portfolio?
 trade_plan =
self.exec_adapter.create_plan_from_thesis(thesis, total_value=notional)
 except Exception as e:
 # If plan generation fails, we can still continue; likely no
fail scenario unless thesis legs invalid (which we validated earlier)
 trade_plan = None

 # Step 5: Persistence
 # Save eval_result and review to repository (EvaluationRepository)
 if hasattr(self.data, 'evaluation_repo'):
 # If we created a repository for evaluation results
 if review is not None:
 self.data.evaluation_repo.save_evaluation(thesis_id,
eval_result, review)
 else:
 self.data.evaluation_repo.save_evaluation(thesis_id,
eval_result, None, error=review_error)

 # Step 6: Prepare result DTO
 return ThesisEvaluationSessionResult(
 thesis_id=thesis_id,
 evaluation=eval_result,
 review=review,
 trade_plan=trade_plan,
 # possibly include a flag or error message if review is None
)

```

- We need to define ThesisEvaluationSessionResult Pydantic model:

```

class ThesisEvaluationSessionResult(BaseModel):
 thesis_id: str
 evaluation: ThesisEvaluationResult
 review: ThesisReview | None
 trade_plan: TradePlan | None

```

- The JSON for that will nest the evaluation metrics and review content, which is fine. If review is None (due to error), the field will be null in JSON. We might add another field like review\_error if we want to convey the reason, but that might not be user-friendly to show. Possibly better to not catch LLM exception and let it propagate as 500 to user (with error message), unless we want partial result behavior. It's a design choice:
  - If we propagate error, the entire API returns 500 (with error detail).
  - If we catch and return partial, the endpoint still returns 200 but with review=null. The UI can then indicate "LLM review unavailable".

In an evaluation environment, seeing partial could be useful to still use the quant results. But it might also complicate automated test expectations.

I'll lean toward propagating as error (fail the session) if LLM fails, for simplicity in orchestrator. But I'll keep the above structure to show how partial could be handled. In final code, I might actually do:

```
review = self.llm.review_thesis(...) # if it raises, let it raise out
```

- so that the whole run raises and the API returns error. Then the user can re-try or know it didn't complete. We can note in Risks that an enhancement could be to allow partial results.

### *DailyUpdateSession:*

```
class DailyUpdateSession:
 def __init__(self, data_access: DataAccess, llm_tools: LLMTTools):
 self.data = data_access
 self.llm = llm_tools
 def run(self) -> DailyUpdateSessionResult:
 # Step 1: Load active theses and current portfolio
 theses = [t for t in self.data.get_all_theses() if t.status ==
'ACTIVE'] # assume Thesis.status field exists
 portfolio = self.data.get_current_portfolio()
 # Step 2: Recompute risk snapshot
 depth = self.data.get_portfolio_depth()
 # Step 3: Detect alerts
 alerts = []
 recent_obs = self.data.get_recent_observations(limit=20)
 for thesis in theses:
 # if any recent observation text or category matches a
thesis.disconfirmers keyword, trigger an alert
 for obs in recent_obs:
 # simplistic check: if any disconfirmer substring in obs.text
 if any(dc.lower() in obs.text.lower() for dc in
thesis.disconfirmers):
 alerts.append(Alert(thesis_id=thesis.id,
message=f"Disconfirming evidence: {obs.text[:50]}...",
timestamp=obs.timestamp))
 break
 # Save alerts to repository if needed
 if hasattr(self.data, 'alert_repo'):
 for alert in alerts:
```

```

 self.data.alert_repo.insert(alert)
 # Step 4: Build DailyContext for LLM
 daily_context = DailyContext(
 date=datetime.now().date(),
 portfolio_change=f"{{(portfolio.totals.portfolio_value -
self.data.last_portfolio_value)/self.data.last_portfolio_value*100:.2f}}%" if
hasattr(self.data, 'last_portfolio_value') else None,
 top_alerts=[f"Thesis {{a.thesis_id}}: {{a.message}}" for a in
alerts],
 notable_observations=[obs.text for obs in recent_obs[:3]] #
maybe top 3 obs for context
)
 # (DailyContext could include as much info as we want; here just a
few pieces)
 # Step 5: LLM summary
 summary = None
 try:
 summary = self.llm.daily_summary(daily_context)
 except Exception as e:
 # propagate or handle
 raise e
 # Step 6: Persist summary (if we have a place to save, e.g., in a
DailySummaryRepository or just keep in memory)
 if hasattr(self.data, 'summary_repo'):
 self.data.summary_repo.insert(summary)
 # Also store last_portfolio_value for next run (for delta calc)
 self.data.last_portfolio_value = portfolio.totals.portfolio_value
 # Step 7: Prepare result
 return DailyUpdateSessionResult(date=daily_context.date,
summary=summary, alerts=alerts)

```

- We define Alert and DailyUpdateSessionResult models:

```

class Alert(BaseModel):
 thesis_id: str
 message: str
 timestamp: datetime

class DailyUpdateSessionResult(BaseModel):
 date: date
 summary: DailySummary
 alerts: list[Alert]

```

- Here I assumed DailyContext Pydantic model exists for LLM input, likely similar to:

```

class DailyContext(BaseModel):
 date: date
 portfolio_change: str | None
 top_alerts: list[str]
 notable_observations: list[str]

```

- It might include more like current portfolio metrics, but fine for demonstration. The llm.daily\_summary will convert that to prompt as earlier discussed.

The alert detection logic is simplistic (substring match). We should do something more robust if possible (like check categories or an `Observation.thesis_refs` field which is ideal: if observation is explicitly linked to a thesis with actionable flag). The code summary said `Observation` has `thesis_refs` and actionable flag. If that's the case, we can:

```
for obs in recent_obs:
 if thesis.id in obs.thesis_refs and obs.actionable:
 alerts.append(Alert(...))
```

- That's cleaner. We'll prefer that if data available (we have to have filled `thesis_refs` on observations ingestion).

We also maintain `last_portfolio_value` to compute daily change. In production, we'd derive it from historical values, but we keep a variable or store yesterday's snapshot. We might not implement storing history now, so just having a static in `DataAccess` or in session (the session doesn't persist between runs easily though, unless we keep it at `DataAccess` or global).

Alternatively, we skip `portfolio_change` if we can't easily get yesterday's. It's not crucial. We'll mention it but it's optional.

If any LLM step fails, we propagate error. So `/session/daily-update` might 500 if summary generation fails. Maybe not ideal for a daily job – you might still want to save alerts and proceed. But in eval context, simpler to treat it as one atomic job.

- **API Endpoints Implementation (FastAPI):**

We'll set up, e.g., in `session_routes.py`:

```
@router.post("/api/v1/thesis/{thesis_id}/evaluate")
def evaluate_thesis_endpoint(thesis_id: str,
 data_access=Depends(DataAccessDep)):
 session = ThesisEvaluationSession(data_access,
 eval_service=ThesisEvaluationService(price_source), llm_tools=LLMTools(),
 exec_adapter=PaperExecutionAdapter(...))
 try:
 result = session.run(thesis_id)
 except ValueError as e:
 # likely thesis not found or invalid input
 raise HTTPException(status_code=404 if "not found" in str(e).lower()
 else 400, detail=str(e))
 except Exception as e:
 raise HTTPException(status_code=500, detail="Evaluation failed: " +
 str(e))
 return result.dict()
```

- We inject `DataAccess` (with real repositories) via FastAPI depends. We also need to create instances of `ThesisEvaluationService` and `LLMTools` (which wraps our functions `llm_review_thesis` etc., maybe just grouping them).

We might instantiate these services at app startup and inject via Depends as well (so they aren't re-created each call, which might be fine but if heavy, better to reuse).

Similarly:

```
@router.post("/api/v1/session/daily-update")
def daily_update_endpoint(data_access=Depends(DataAccessDep)):
 session = DailyUpdateSession(data_access, llm_tools=LLMTools())
 try:
 result = session.run()
 except Exception as e:
 raise HTTPException(status_code=500, detail="Daily update failed: " +
str(e))
 return result.dict()
```

- We also define routes for intelliQ&A if needed:

If an existing route doesn't exist, we do:

```
@router.post("/api/v1/intel/qa")
def intuition_qa_endpoint(request: QARequest):
 # QARequest might have question: str, thesis_id: optional for filtering
 obs = data_access.get_recent_observations(50)
 # If request.thesis_id given, filter obs to those related to that thesis
 if request.thesis_id:
 obs = [o for o in obs if request.thesis_id in o.thesis_refs]
 try:
 answer = llm_tools.query_intuition(request.question, obs)
 except Exception as e:
 raise HTTPException(status_code=500, detail="LLM QA failed: " +
str(e))
 return answer.dict()
```

- But possibly such route exists in intelligence\_routes of code (the summary said something about POST /api/v1/intel/qa). We might reuse or refine it.
- **Testing Plan (integration):**

We will orchestrate a scenario through the API in tests:

- **Scenario Setup:** Use the repository layer to insert:
  - A Thesis (with a known hypothesis, etc., and maybe one expression leg).
  - Price data for the leg's asset (if our ThesisEvaluationService will fetch from PriceSource, we need to ensure PriceSource in test returns our synthetic data for that asset).
  - If needed, mark the thesis status so daily update will consider it (e.g., status="ACTIVE" after approve in E5, but for test maybe we directly set it as active to simulate an already approved thesis for daily update).
  - Insert an Observation referencing that thesis's disconfirmers to test alert generation.
- **Thesis Evaluation Test:**

- Patch LLM calls: monkeypatch `llm_review_thesis` to a fake as done in E3 tests, to avoid real API.
  - Hit POST `/api/v1/thesis/{id}/evaluate` via `TestClient`.
  - Check response JSON: It should have `thesis_id`, `evaluation` (with `total_return` etc.), `review` (with `critique`, `questions`, `risk_flags` from our fake), and `trade_plan` (if we included `ExecAdapter` in session with stub, maybe it created a plan).
    - If we didn't implement `ExecAdapter` or left it `None`, `trade_plan` in result will be null. The output should not error, just missing plan, which is fine.
  - Also verify side effects: if we had an `EvaluationRepository`, query it to see if an entry saved. (We may not implement repository fully for evaluation due to time, we might just rely on returning result. If we did, test that it updated existing or inserted new).
  - Simulate an LLM failure scenario by patching `llm_review_thesis` to throw an exception. Hit endpoint, expect HTTP 500. Or if we decided to return partial (`review = null` with 200), our test should reflect that design. Let's say we propagate error: we test that we get status 500 and the error message in response JSON.
  - Also test input validation: call endpoint with an invalid `thesis_id` (one that doesn't exist). Expect a 404 response.
- **Daily Update Test:**
- Mark a thesis as active and with a `disconfirmers` field containing e.g., "Inflation rises".
  - Insert an `Observation` with text "Inflation rises 5%..." and mark it actionable or link to thesis. Could do `obs.thesis_refs = [that thesis_id]`, `obs.actionable=True`.
  - Insert a `Trade` or two to simulate portfolio (or use `DataAccess` to set up positions) – since `DataAccess.get_current_portfolio` uses `trade_repo`, we should insert a trade for that thesis so that portfolio isn't empty. For example, 100 shares of asset at price X. Also, maybe set `DataAccess.last_portfolio_value = some number`, to simulate previous day. But if not, `portfolio_change` may be `None` or we skip that feature.
  - Patch `llm_daily_summary` to fake output (we test orchestrator logic, not LLM content quality).
  - Call POST `/api/v1/session/daily-update`.
  - Check response:
    - "alerts": should contain one alert for that thesis (with correct `thesis_id` and message snippet).
    - "summary": matches our fake summary (one or two narratives, etc.).
  - Verify persistence: if we have `alert_repo` or `summary_repo`, fetch from them:
    - For example, `alert_repo.list_all()` should include the new alert.
    - If summary saved, check that too.

- Also test scenario of no alerts: if no observation triggers, ensure alerts list empty (and summary still returns 200).
- Possibly test incomplete data: if no active theses, daily-update might just summarize portfolio and no alerts. The LLM might still produce something (maybe `insufficient_context` true if nothing to say). Ensure it handles gracefully (not throw error on empty theses).
- Test error handling: if LLM summary fails (patch it to throw), endpoint should return 500. If `DataAccess` had an issue (like `trade_repo` failing), also 500.
- **CrossThesis and QA (if endpoints exist):**
  - For `crossThesis`: create two theses, patch `llm_cross_theses` to fake output, call endpoint with those IDs, verify output. Not essential if not explicitly required by user request but nice to validate completeness.
  - For QA: insert a couple observations, patch `llm_query_intuition`, call endpoint with question, verify `IntuitionAnswer` output.
- Ensure throughout that our dependency injection in FastAPI is correctly providing test instances (like test DB and stub LLM) to the endpoint. We might override dependencies in `TestClient` or use FastAPI `dependency_overrides`. For example, override `DataAccessDep` to yield our test `DataAccess` connected to an in-memory DB. Also override `LLMTools` in session objects to use our stub functions instead of real OpenAI calls.

In sum, these integration tests confirm that given certain DB state and stubbed external interactions, the orchestrated endpoints produce the expected end-to-end outputs. This is the final proof that E1–E3 (and part of E5) are working in concert.

#### E4-Risks & Dependencies:

- *State management between sessions:* The daily update uses data from previous days (like last portfolio value, and alerts accumulate). If our service restarts or daily updates run in separate stateless contexts, we need to fetch historical data from DB instead of relying on in-memory `last_portfolio_value`. A robust design would store yesterday's portfolio value or store daily P&L in DB. Our simplified use of an attribute on `DataAccess` will reset if the app restarts, so for a production scenario that's not reliable. For evaluation, we often run it in one go, so it's okay. We call this out as a limitation. A workaround could be storing portfolio value in a config table or so, but not necessary now.
- *Atomic transactions:* In the evaluate session, if LLM fails after saving quant result, we considered rolling back. We did mention using one repository call at end (`save_evaluation`) to store everything at once. If that repository uses a DB transaction, it will commit both results together. If LLM fails, we skip calling `save`, so nothing saved. This approach ensures atomicity logically. If we had to save quant earlier (for long quant tasks) we might commit interim results, but that's not needed now (quant is quick).
- *Concurrency issues:* If two eval sessions for same thesis run concurrently, our overwriting scheme might cause an inconsistent state (one might override or they could

intermix). This is unlikely since user triggers evaluation manually per thesis. If it did, last one wins; not critical.

- *Orchestrator error handling complexity*: As discussed, decision between partial vs full failure. We opted to propagate errors by default for simplicity and clarity. This means if any part fails, the user gets an error and no DB changes (except perhaps logs). This is safer to ensure no inconsistent data. The downside: user might have wanted partial info. In an eval harness, full success or clear failure might be preferable.
- *Integration of E5 (Paper trading)*: We included trade plan generation in ThesisEvaluationSession optionally. If not provided or fails, it doesn't break anything (trade\_plan stays None). E5 itself (approve-plan session) will be next, but here we just consider preparing a plan as part of evaluation output. This is a forward-looking design: even if not fully used in this phase, it shows how we would incorporate it.
- *Data consistency between sessions*: Evaluate session might mark a thesis as evaluated (we could update thesis.status from "DRAFT" to "EVALUATED" or similar). Not required, but could help filter which theses have results. Since user might edit thesis after evaluation, status management is tricky. We avoid for now.

Daily update expects theses with status "ACTIVE" meaning their strategies are in portfolio. We should ensure that when user approves a plan (E5), we set thesis.status = ACTIVE. We will handle that in E5, but E4 daily update logic expects that to differentiate which theses to monitor. In test, we directly set it.

- *Sequence of operations (E1–E6 usage path)*: The typical path is: user creates thesis (status DRAFT) -> run evaluate -> get results and trade plan -> user approves plan -> trades go live, thesis status -> ACTIVE -> daily updates monitor it. We must ensure transitions and data flows are accounted:
  - After approve, trades exist for daily update to pick up. We'll confirm in E5 that happens.
  - If user evaluates again after approval (the thesis is active), our evaluate session still works, just now maybe using updated trade history if needed (though quant backtest is unchanged by being active).
  - If user doesn't approve, thesis remains not in portfolio, daily update might ignore it (so status might remain DRAFT or 'EVALUATED'). That's fine.
- *Completeness of daily context for summary*: If daily context lacks some needed info, the LLM summary might be generic or insufficient\_context. We provided some basics (portfolio change, alerts, notable obs). It should suffice for a meaningful summary like "Portfolio was flat. No major alerts except [something]." If insufficient\_context flag is returned consistently because our context is too sparse, we might need to enrich it (like provide a bit of yesterday vs today breakdown of each asset or factor). For now, minimal context is okay to test pipeline.
- *Time scheduling*: We manually trigger daily update via API for eval. In production, one would schedule session.run() daily at a set time. That scheduling is outside our code scope, but our architecture supports it by having an endpoint or function to call. The evaluation plan did not require implementing a scheduler, but it's good to note it.
- *UI coupling*: The UI (E6) will call these endpoints and then display results or feed them into the appropriate places. We should ensure the results have everything needed for UI:



- For evaluate: UI will show metrics (evaluation), LLM critique (review), trade plan (maybe just present or allow approval).
- For daily update: UI will show summary text and list of alerts. We should ensure alert objects have enough detail (maybe include thesis title for readability). Right now, Alert has thesis\_id and partial obs text. UI can look up thesis title by id via DataAccess if needed. Or we include thesis title in Alert message to simplify. Actually, in top\_alerts we gave “Thesis {id}: snippet...” which LLM summary might incorporate or UI can show directly in an alerts panel. Perhaps we should include thesis title in Alert object for direct UI use. We can modify:

```
class Alert(BaseModel):
 thesis_id: str
 thesis_title: str
 message: str
 timestamp: datetime
```

- - and fill title when creating alert. That saves UI an extra lookup. We’ll do that if possible (we have thesis object in daily\_update loop, so easy).
  - For daily summary, we list thesis\_references (ids) so UI could hyperlink to them. That’s good, but UI needs a way to convert id to name for display. It can call DataAccess.get\_thesis for each id or have them cached from listing. Or we could have included thesis titles in summary directly. We left it as ids to keep output compact and linkable.
- *Testing complexity:* Orchestrator integration tests involve multiple moving parts and likely need extensive use of test doubles. We must carefully isolate external dependencies:
  - PriceSource in ThesisEvaluationService (for integration, we might inject a TestPriceSource that returns fixed data).
  - LLM tools in sessions (we patch them as before).
  - DB (we use our test DB with known entries).

Many of these we did similarly in earlier phase tests, but now combined. We might reuse some fakes from those.

There is a risk of flaky tests if not properly isolating; but since we control all random/external factors via stubs, it should be stable. We’ll ensure to use same stub outputs as used individually so the combined result is predictable.

- *Follow the “no phase depends on future magic” rule:* by implementing explicit interfaces now (EvaluationRepository, AlertRepository placeholders, etc.), we avoid leaving undefined things. The original plan had vague “risk engine” which we replaced with explicit get\_portfolio\_depth. “Memory app” for observations we replaced by DataAccess observation queries. The orchestrator doesn’t assume any component beyond what we have built.

One potential future magic would be if we said “somehow an observation will trigger alert” without mechanism. We defined at least a simple mechanism. Could be refined but it’s a concrete approach. So we meet that quality rule.

With E4 done, we have a fully connected pipeline: from input data to user-facing output, with testing verifying each step’s integration. The architecture is now ready for the UI to consume and for actual user testing, addressing earlier architecture’s vagueness by clearly defining modules, data contracts, and error handling.

## E5 – Paper Trading & Execution Adapter

*(The Phase E5 implementational details are included based on earlier plan, to show readiness for trade execution. Though not explicitly demanded in the question text, it was in the E-phases list, so we provide the blueprint. Engineers can implement it now or in the next step.)*

### E5-Requirements:

- Introduce a **PaperExecutionAdapter** that handles converting a thesis’s target allocation into actual trades (a TradePlan) and executing those trades on a simulated (paper) portfolio. Key responsibilities:
  - `create_plan_from_thesis(thesis)`: produce a TradePlan DTO listing each asset, direction (buy/sell), and size or quantity to trade to achieve the thesis’s expression. This plan essentially mirrors the thesis legs but might incorporate current portfolio context (e.g., if some position already exists, plan would be incremental). Initially, for simplicity, assume no existing positions (plan is as if starting fresh with notional capital).
  - `execute_plan(plan, as_of_date)`: simulate executing all trades in the plan at given date’s prices. This means creating Trade records for each leg (with `timestamp=as_of_date` or `now`, price from PriceSource, quantity computed). Save these trades via TradeRepository and return them (or their IDs). After this, the portfolio now contains those positions.
- Ensure that after execution, `DataAccess.get_current_portfolio` will reflect the new trades (positions updated), and that daily updates will include these in calculations. Also, mark the thesis as active in DB (so daily update will track it).
- Provide an API endpoint to approve a plan: `POST /api/v1/thesis/{id}/approve-plan` which triggers execution for that thesis:

1. It might accept a JSON body if any confirmation needed (or just use default notional if not provided).
  2. It will call PaperExecutionAdapter to execute trades.
  3. If successful, update Thesis.status to ACTIVE and return a response (perhaps the executed trades or a success message). If fails (price fetch issue or DB error), return error.
- The ExecutionAdapter should be testable with synthetic price data like ThesisEvaluationService was (they both rely on PriceSource). In fact, the execution uses current price (which could be yesterday's close or realtime price). For testing, we can use static price or a stub.
  - P&L calculation: ensure that as trades accumulate, we can calculate unrealized P&L for positions. We might augment DataAccess or have a method to compute P&L (difference between current price and execution price \* quantity for each trade). Possibly add cost\_basis info to trades or compute average cost on the fly. We can add a field in PortfolioSnapshot or as part of PortfolioDepthSnapshot to show total unrealized P&L and maybe per thesis P&L. This wasn't explicitly asked, but since the UI wants P&L on thesis list, we should provide a way:
    - We can store initial trade plan notional as cost, or derive cost from trade history (sum of buy costs minus sum of sell proceeds). For now, we can compute portfolio P&L in UI by comparing current portfolio\_value vs sum of all trade buy values (treating sells as negative cost).
    - Possibly extend PortfolioTotals with an unrealized\_pnl field for convenience. Our earlier DataAccess didn't, but we can compute it in UI or in DataAccess if needed.
  - **Testing:**
    - Simulate approving a plan for a thesis:

*Precondition:* ThesisEvaluationSessionResult has a TradePlan (maybe notional \$10000 allocation). In test, we either generate that with ExecutionAdapter or fabricate it.

- Patch PriceSource to provide a price for each asset on execution date.
- Call approve-plan API.
- Check that new Trade entries are in DB (with correct asset, quantity, price).
- Check that Thesis.status is now "ACTIVE".
- Check response content: maybe list of executed trades or a success flag.
- Then call get\_current\_portfolio and ensure positions correspond to those trades (which was tested in E1/E2).
- Also ensure that if approve endpoint called twice, it either prevents second (returns 400 "already active") or doubles positions (which we might not want). We likely protect from double approval:
  - e.g., in endpoint, if thesis.status already ACTIVE, return 400 "Plan already executed".
  - Test that: after one approval, second call yields 400.

- Test partial plan execution: if plan had multiple legs and one fails (maybe one asset price missing), either treat all as fail (and no trade inserted) to maintain atomicity. Using transactions, if one insert fails, we roll back. Simulate price missing by patching `price_source.get_price` to throw for one asset. Expect endpoint returns 500 and no trades saved.
- If everything is fine, daily update after that should pick up these trades and maybe produce an alert if something moved. We would have covered that in daily update test as long as trade was present.

## E5-Research Summary:

Paper trading allows safely testing strategies. The ExecutionAdapter design separates the concern of *what to trade* (TradePlan) from actually recording the trades (`execute_plan`), which aligns with a typical order execution flow. This modularity allows using the same plan generation logic to display to user for confirmation and then reuse for execution.

We operate under assumptions:

- No trading costs or slippage: execution at current price exactly for full quantity.
- Sufficient liquidity to fill orders (no partial fill issues).
- The entire plan executes at once (no time stagger).

These simplifications are fine for evaluation and mimic an *ideal execution* scenario.

A potential design choice: if the thesis's weights sum to less than 100, do we invest only that portion? If using `total_value`, we invest that amount proportionally. If `sum < 100`, effectively some cash remains uninvested. That's okay (maybe the user didn't allocate 100% intentionally). We'll respect given sizes.

We also want to consider if some positions already exist. E.g., user might approve additional capital to the same thesis later. Our current system doesn't explicitly handle adding to an active thesis (like scaling up). If they re-approve plan, we either should detect and prevent double execution, or treat it as adding more. We likely prevent for now (one thesis = one execution to avoid confusion). If user wants to add, they could create a new thesis or we'd need a different flow (like allow a second plan and mark it cumulative).

For P&L, we consider an approach:

- We can compute cost basis per asset by summing trade costs. Then P&L = current value - total cost (for all positions).
- Or track it per thesis: sum of trade costs for that thesis = invested capital, compare to current value of positions tied to that thesis.

We can store `trade.thesis_ref` already. So an approach:

```
thesis_pnl = sum((current_price(asset) - trade.price) * trade.quantity * (1 if trade.action == 'BUY' else -1) for each trade with thesis_ref = thesis)
```

But if multiple trades per asset for one thesis, simple formula gets tricky (should aggregate positions).

Better:

For each thesis, take all trades:

- Compute total cost: `sum(BUY trades value) - sum(SELL trades value)`.
- Compute current value: `sum(current_price * net_quantity)`.
- P&L = current value - total cost.

We could add a method `DataAccess.get_thesis_performance(thesis_id)` doing this. The UI can then display P&L%. Because they explicitly wanted P&L in thesis list, we likely need this.

We note that short positions yield negative cost (a cash inflow), and current value negative, so formula still works (e.g., short sold at \$100 (got \$100), now price \$120, net position -\$120, cost -\$100, P&L =  $-\$120 - (-\$100) = -\$20$  loss).

## E5-Design:

- **PaperExecutionAdapter Class:** (in `src/slice/execution/paper.py`)

```
class PaperExecutionAdapter:
 def __init__(self, trade_repo: TradeRepository, price_source):
 self.trade_repo = trade_repo
 self.price_source = price_source
 def create_plan_from_thesis(self, thesis: Thesis, total_value: float) -> TradePlan:
 legs = []
 for leg in thesis.expression_legs:
```

```

 legs.append({"asset": leg.asset, "direction": leg.direction,
"size_pct": leg.size_pct})
 return TradePlan(thesis_id=thesis.id, total_value=total_value,
legs=legs)
 def execute_plan(self, plan: TradePlan, as_of_date: date = None) ->
list[Trade]:
 trades = []
 exec_time = datetime.now() if as_of_date is None else
datetime.combine(as_of_date, datetime.min.time())
 for leg in plan.legs:
 price = self.price_source.get_current_price(leg["asset"]) # as
of execution time
 # Determine action and quantity
 action = "BUY" if leg["direction"] == "long" else "SELL"
 # size_pct is of total_value
 notional = plan.total_value * (leg["size_pct"] / 100.0)
 if action == "BUY":
 quantity = notional / price
 else:
 quantity = notional / price # selling short notional =
borrowed value
 trade = Trade(trade_id=str(uuid4()), timestamp=exec_time,
asset=leg["asset"],
 action=action, quantity=quantity, price=price,
type="PAPER", thesis_ref=plan.thesis_id)
 self.trade_repo.insert(trade)
 trades.append(trade)
 # After executing, ideally mark thesis as ACTIVE
 # (if we have ThesisRepository accessible, we could do that here. If
not, orchestrator will do it.)
 return trades

```

- We define TradePlan model:

```

class TradePlan(BaseModel):
 thesis_id: str
 total_value: float
 legs: list[dict] # each with asset, direction, size_pct

```

- Alternatively, define a TradeLeg(BaseModel): asset, direction, size\_pct and use legs: list[TradeLeg]. But dict is fine too in usage.

The create\_plan is straightforward: basically packaging the thesis expression with a chosen total\_value. It doesn't account for existing positions; that might be a future refinement (like net out what's already held). Right now, we assume starting from zero.

execute\_plan uses price\_source for each asset. We treat each leg independently. For long, we buy quantity = (target dollars)/price. For short, we "sell" the same formula (so if size\_pct=50, total\_value=100k, notional=50k, we sell short \$50k of asset, meaning we borrow shares worth 50k at current price). That yields a positive quantity in our representation, with action "SELL" indicating a short. The trade\_repo will store action so DataAccess knows to subtract quantity when aggregating.

After inserting trades, we mark thesis status. Possibly we should have a handle to ThesisRepository. We might modify PaperExecutionAdapter to accept a thesis\_repo or DataAccess so it can call something like thesis\_repo.update\_status(thesis\_id, ACTIVE). Or we handle that in orchestrator after execute\_plan returns (which is easier).

- **Approve Plan Endpoint:**

```
@router.post("/api/v1/thesis/{thesis_id}/approve-plan")
def approve_plan_endpoint(thesis_id: str,
data_access=Depends(DataAccessDep)):
 # We'll interpret no body or optional body for notional; or could be
 fixed
 thesis = data_access.get_thesis(thesis_id)
 if not thesis:
 raise HTTPException(status_code=404, detail="Thesis not found")
 if thesis.status == "ACTIVE":
 raise HTTPException(status_code=400, detail="Thesis already active /
plan already executed")
 # Use existing plan if we saved one in evaluation? For now, recalc a
 plan:
 adapter = PaperExecutionAdapter(trade_repo=data_access.trade_repo,
price_source=price_source)
 plan = adapter.create_plan_from_thesis(thesis, total_value=1000000.0) #
e.g., allocate $1M by default
 try:
 trades = adapter.execute_plan(plan)
 except Exception as e:
 raise HTTPException(status_code=500, detail="Execution failed: " +
str(e))
 # Mark thesis as active
 if hasattr(data_access, 'thesis_repo'):
 thesis.status = "ACTIVE"
 data_access.thesis_repo.update(thesis) # assume update writes status
 return {"status": "success", "trades": [t.dict() for t in trades]}
```

- 
- We decide on a default capital for execution (e.g., \$1M or could take from user input). We must communicate that to user maybe (UI could allow them to specify how much to deploy).

We also caution: if asset price very low or high, resulting quantity might be huge or small. That's fine for simulation.

After execution, portfolio now has positions for that thesis, and daily updates will consider it (since now active).

We test double approval as mentioned.

- **Integration with Sessions and UI:**

- The ThesisEvaluationSession included plan generation (if exec\_adapter passed in). That means when user sees evaluation results, they already have a TradePlan

proposed (with default notional, which UI might display as part of results, e.g., “Proposed trades: Buy 100 shares of X, short 50 shares of Y”). The user then presses “Approve Plan”.

- Approve plan endpoint will re-generate plan in our design (to avoid storing it or in case user changed notional). There’s slight redundancy and risk of plan mismatch. We might want to trust the plan we already have from evaluation to ensure consistency. But in our code, we didn’t store the plan globally; it was only in the returned JSON to UI.
- We could modify approve endpoint to accept the TradePlan in the request body (so UI sends back the plan it got). That would remove ambiguity and also allow user customizing notional or tweaking before execution. Or simpler: store the plan in a short-term cache on server (like DataAccess or an EvaluationRecord).
- Given time, we might skip that and just recompute (assuming same result).
- For evaluation stage, it’s fine either way, but a note: If market moved between evaluation and approval (maybe a day passed), re-generating plan uses new prices if not careful:

create\_plan doesn’t depend on price (just uses weights and notional), but execute\_plan will use current price at execution, which is appropriate.

If UI used an outdated plan with older prices, it might have shown slightly different quantities. However, UI likely doesn’t show specific quantity, just percent, so it’s okay.

We’ll document that slight difference could occur but negligible if time difference short.

- Marking thesis active is crucial so daily updates track it and UI shows it under portfolio. We included that.
- DataAccess or ThesisRepository needs an update method for status. If not present, we implement a simple one. Or we can do a direct SQL update in endpoint.
- After approve, user can see in UI: Thesis moves to “Active” status, P&L column starts showing something (initially 0 maybe). We ensure trade recorded cost is known so  $P\&L = 0$  at start.
- We may want to, after execution, compute initial portfolio value for that thesis so daily update can track if it grew or shrank. Actually, initial portfolio value for that thesis = plan.total\_value (the cash invested). We can store that somewhere (like Thesis.initial\_alloc or record an entry in a PortfolioHistory).
- But easier: when computing P&L for an active thesis, we use trades: at time of approval,  $P\&L = 0$  by definition if using trade prices as cost.

## E5-Testing Plan:

We’ve outlined tests earlier:



- Insert a thesis with e.g., legs: 50% A (long), 50% B (long). Simulate evaluation was done (or not needed strictly to test approve).
- PriceSource stub: asset A price \$100, B \$200 at execution.
- Call approve endpoint: expect two trades:
  - BUY A quantity  $\sim (0.5 * \$1M) / 100 = 5000$  shares at \$100.
  - BUY B quantity  $\sim (0.5 * \$1M) / 200 = 2500$  shares at \$200.
  - These in DB trade\_repo; test list\_by\_thesis returns them.
  - portfolio value now  $\sim \$1M$ . DataAccess.get\_current\_portfolio would sum to 1M as well (which it should by our logic, ignoring minor float error).
  - Thesis.status = ACTIVE in DB.
  - Response contains those trades (or at least status success).
- Then call daily-update or get\_current\_portfolio to see if everything picks up:
  - get\_current\_portfolio should show positions A and B with correct values (as we tested in E1).
  - daily\_update should not trigger alerts just from execution, unless some observation we set triggers (we can also test an alert post-approval to see how integrated flow works).
- Test if we call approve again:
  - It should return 400 and no new trades (check trade\_repo count remains same).
- Test short legs:
  - If thesis had short B 50%, plan yields SELL \$500k of B, price \$200 -> quantity 2500, action SELL.
  - Check that trade inserted action SELL quantity 2500, price 200.
  - get\_current\_portfolio should reflect a short position (quantity -2500 in B)? Actually, how do we represent short internally? We recorded as action SELL with positive quantity. DataAccess when summing:

It adds quantity if BUY, subtract if SELL. So net quantity = -2500 for B, value = -\$500k, which in positions might show negative value for B. Portfolio net value = \$500k from A - \$500k liability of B = \$0 net (if evenly hedged scenario).

That suggests if we invest equal dollars long and short, initial net equity = \$0? Actually, if we had \$1M notional, short B gave \$500k cash proceeds, which combined with original \$500k cash we had to fund long A, net we didn't use outside funds. The model might show portfolio\_value = 0 which is weird (should be \$1M? Actually, no external money, it's a fully hedged pair trade with no net investment: you used \$500k to buy A, financed by \$500k from short B, so net out-of-pocket \$0, portfolio net equity \$0 plus 500k long and -500k short).

This is a scenario our earlier portfolio calc would show portfolio\_value  $\sim 0$ . It means the strategy used leverage (long and short positions but no net capital).

If initial capital was needed, one might consider half capital for each, but in how we set up notional as the sum of abs legs, we effectively assumed margin usage.

It's a complexity but as an eval tool, it's okay if portfolio\_value small or zero for balanced long-short. Gross\_exposure would be \$1M though, net\_exposure 0.

We ensure our DataAccess at least reports net and gross exposures in such case to illustrate risk.

P&L tracking in such a scenario would be relative to that net? Actually, if net equity is near zero, percentages blow up. Realistically you'd allocate actual capital and accept a net \$0 starting equity meaning fully leveraged.

For evaluation, if we see such weirdness, we just note that a long-short strategy is being represented with net zero initial capital, which can confuse daily summary (which might see large fluctuations relative to net).

Perhaps for such, better to allocate total\_value meaning initial capital that covers long and short i.e., provide margin. If we had \$1M and did 50% long, 50% short with total\_value=1M, our code did \$500k long, \$500k short, and effectively didn't use the other \$500k that short provided.

Actually, in that case, initial actual cash used = \$500k (to buy A), the other \$500k came from short B. So initial equity \$500k. Actually, net position should be \$500k if done properly: we should treat total\_value as actual cash deployed, not sum of absolute. That means if doing 50% long, 50% short, we should only invest \$500k long and short \$500k (so gross \$1M, net \$500k initial equity).

Our create\_plan currently does legs = 50,50 of 1M = 500k each, as we did, which implies you needed \$500k cash (for long) and gained \$500k from short, leaving \$500k idle. Actually, in that scenario, after execution:

- long asset value = \$500k, short liability = \$500k, net = 0 + initial cash \$500k not used = still have \$500k cash aside. If we assumed total\_value=1M, we ended using only half?

This is tricky. A clearer method:

- If thesis legs sum to 100 and include shorts, the total gross exposure = 100% long + 100% short = 200 gross. If user gave 1M capital, possible they intend to use margin. Or they should only allocate half to longs and half to shorts to not use margin.
- Hard to guess. For simplicity, maybe restrict to long-only for now or accept levered scenario.

We'll proceed without adjusting logic (the model can handle net exposures maybe).

We'll mention in risk that short positions not fully accounted for net capital usage.

- For test, if such scenario yields `portfolio_value` smaller than total, we accept it and ensure system doesn't break (like division by net equity somewhere).
- Possibly adjust in `DataAccess`: we might treat `net_exposure` differently if `net < 0` or too low. But okay for now.
- *Risks and complexities from above analysis:*
  - Negative or zero net portfolio after executing balanced long-short with `total_value` might confuse daily update (it might compute portfolio change relative to `last_portfolio_value` 0, leading to inf or error).
  - We could mitigate by always adding an initial cash position equal to `total_value` to net out shorts. E.g., if short, keep the cash from short as part of portfolio. Actually, yes: when you short, you get cash that should stay in portfolio (to later cover buy-back). In our `DataAccess`, we did not track cash explicitly, so net equity was undervalued for short positions.
  - Proper fix: treat short proceeds as cash. Implementation: after executing trades, if any SELL trades (short opens), add a synthetic "Cash" trade or directly incorporate into portfolio calc.
  - One approach: in `get_current_portfolio`, after summing trade contributions, also compute `total cash = sum(sell proceeds) - sum(buy costs)` (since we never tracked initial cash injection, we could treat initial `total_value` as initial cash deposit). For net equity, add that cash. In a fully hedged scenario, initial cash deposit might equal half the plan notional if plan was levered, which would yield net equity equal to that deposit.
  - This is detailed but necessary for correct accounting. We'll note it but possibly out-of-scope to implement fully now. We'll caution that our simulation does not explicitly model cash, so the net equity for portfolios with shorts will be underrepresented, and so will `portfolio_value` (like in example).
  - For evaluation demonstration, typically examples might be long-only or not fully balanced short, to avoid confusion.
- *Integration timeline:* We design E5 now, but maybe the actual code integration can be done in Phase E5 after initial evaluation. We included it to show completeness and because UI and scenario mention "approve trade, see it flow into portfolio". That suggests E5 is indeed expected in the Use Stage script. We'll implement enough to support that.
- *Checkpoints:* After E5, the pipeline from thesis creation to daily monitoring is fully realized. We'll verify the Use Stage in next section covers all these transitions.

## B. Cross-Cutting Evaluation Strategy

Now that we have specified each phase’s design and tests, it’s crucial to ensure the entire system works as a cohesive whole and meets evaluation goals. Our testing strategy follows a **test pyramid** approach :

- We have detailed **unit tests** for components in isolation (repository, quant metrics, LLM parsing). These give quick feedback on specific logic.
- We wrote **integration tests** (in Phase E4) that cover how components interact through the defined APIs (simulate user flows end-to-end). These are fewer but cover broad paths .
- We will also consider a small number of **end-to-end scenario runs** as acceptance tests, effectively using the system as a user would. This is described in the Use Stage script (section C). These scenario runs are manual or semi-automated validations that the system produces the expected results in realistic cases.

To monitor and evaluate system performance and reliability:

- We track LLM usage metrics: after some runs, we can compute average latency for each tool and see if any are outliers (e.g., if `daily_summary` is much slower than `thesis_review`). We can also count how often `insufficient_context` is true or errors occur; if frequent, that indicates the input data might be insufficient or prompts need adjusting . We expose these via `/api/v1/diagnostics/llm` so the evaluator can inspect them easily. For example, after running 5 sessions, they might GET that endpoint and see:

```
{
 "llm_tool_metrics": {
 "thesis_review": {"calls": 5, "errors": 0, "avg_latency_ms": 1300},
 "cross_theses": {"calls": 1, "errors": 0, "avg_latency_ms": 1100},
 "intuition_qa": {"calls": 2, "errors": 0, "avg_latency_ms": 800},
 "daily_summary": {"calls": 5, "errors": 0, "avg_latency_ms": 1500}
 }
}
```

- This tells us that, say, `daily_summary` is averaging 1.5s, which is fine, and no errors so far. If an error had occurred, the “errors” count would be `>0`. This monitoring ensures our LLM integration is robust over multiple runs (no creeping failures or latency spikes).
- We maintain a unified test suite that can be run after each major change (maybe in CI). It includes the unit tests for E1–E3 and the integration tests for E4–E5. Passing all tests means the pipeline is intact.
- We use **canonical scenarios** (like the Gold vs Inflation thesis scenario provided) to do end-to-end checks:
  - Scenario A: *Gold vs Inflation* thesis – we know historically how gold performs with inflation. We will create that thesis, run evaluation (it should show maybe gold’s volatility and such), see LLM critique (likely flag “if inflation rises unexpectedly, risk to gold”), approve it, then simulate an inflation surprise observation and run daily update to see alert and summary. This scenario touches

all parts. We'll verify qualitatively that outputs make sense (for example, LLM flags indeed mention inflation risk).

- Scenario B: *USD vs EUR interest differential* thesis – maybe one thesis long USD, another long EUR, to test cross-thesis consistency. We run `cross_thesis` and see if LLM finds the contradiction (“both cannot be right about USD’s direction”). This scenario tests our `cross_thesis` tool in context.

We will manually run these and compare to expectations, adjusting prompts if needed to guide the LLM to highlight the intended insights.

- **Non-functional metrics:** We'll observe overall session latency. If a thesis evaluation session call (including quant + LLM) takes, say, 2 seconds on average, that's acceptable. If it was, say, 20s, we might consider optimizations (like simplifying prompts or using faster LLM for some tasks). We also consider memory usage: handling large price series or embedding a lot of text in prompts could be heavy. Our design limited inputs (we don't pass full time series to LLM, only summary stats), so memory overhead is fine. We can simulate a worst-case (like 5 theses, each with complex content, for `cross_thesis`) to ensure the prompt still within token limits.
- **Data integrity across components:** Our tests ensure that, for example, the thesis IDs and observation IDs are consistent between `DataAccess`, LLM outputs, and UI:

e.g., if LLM daily summary references “T1”, our UI or another layer can map “T1” to actual thesis. We ensure there's no mismatch (our tests feed actual IDs to LLM and validated references).

- **Security & privacy in evaluation:** All data here is fictional or sanitized (no actual PII). The LLM calls should not leak any internal data beyond what's given. We instruct them not to use outside knowledge, which also means not revealing internal environment info. We won't test adversarial prompts (like user asking the LLM about the system internals) as it's out of scope, but our isolation approach inherently prevents that (LLM is not exposed to conversation with a user directly, only via system-controlled prompts).
- **Robustness to bad inputs:** We test things like missing thesis id (we did 404), or question with no observations (we handle with `insufficient_context` flag).

Another robust design measure: use Pydantic for request bodies to auto-validate required fields. E.g., define `QAResponse(BaseModel): question: str, thesis_id: Optional[str]`. FastAPI will ensure 'question' is provided or else return 422. Similarly, for any endpoints expecting input, we rely on such validation to catch user errors early.

- **No ephemeral dependencies:** Because we rely on largely stateless operations (each session call fresh queries DB, calls services, returns), our pipeline can be run repeatedly without needing resets. One subtle dependency: `DataAccess.last_portfolio_value` which we used to compute daily change. That is ephemeral. If daily update was called after a restart, `last_portfolio_value` might be missing (we set `None` and skip percent change or compute from DB if we stored yesterday's value in an Alert or summary record). We might skip showing daily % change if cannot compute reliably across runs. It's more

important for continuous operation. Since evaluation likely runs sequentially in one process, it's fine. But we highlight that in a stateless environment, an alternative design is needed (like storing last portfolio value in a table).

- **Team evaluation tasks:** The evaluation harness will likely be used by others to test various scenarios. We've facilitated that by externalizing as much as possible:
  - PriceSource can be swapped (to test with different historical data or live data).
  - LLM prompts can be adjusted if needed (they're in code but could be parameterized).
  - All critical modules (repos, evaluation, LLM) are injectable, which means for testing we could plug in fake ones (which we did in tests) and for production we plug in real ones.
- **No usage of future phases:** We carefully avoided referencing UI or anything not implemented yet in our services. The sessions produce purely JSON-able results that UI can consume, fulfilling evaluation plan. Similarly, we didn't assume any Phase 7-9 advanced contexts beyond what we built (like "narrative coherence context" is replaced by our daily summary which covers narrative of the day, etc.). We thereby avoided building on undefined components. Where earlier docs were vague ("risk engine will provide exposures"), we concretely implemented exposure calculation. Where Phase 9 talked about "evaluation harness", that is literally what we built with contract tests and structured outputs (closing that gap).

In summary, our cross-phase evaluation strategy ensures each piece is individually reliable and the whole pipeline achieves the intended output for key use cases, with telemetry to catch performance or reliability issues early. We believe by following this plan and running the outlined tests and scenario validations, we will achieve a **"no surprises"** integration where each phase's outputs cleanly feed the next, and any mismatches would have been caught in our testing process.

## C. "Use Stage" Script

**Scenario:** *Macro Portfolio Manager using Slice for a new thesis and daily monitoring*

**Starting Point:** The Slice system is up with an empty database (or default demo data). The PM (user) will create a thesis about gold and inflation, evaluate it, approve the trade plan, and see how it's reflected in the portfolio and daily reports. We'll walk through each step as the user would, including expected system responses.

## Step 1: Create a New Thesis

**User Action:** In the Slice UI, navigate to the *Thesis* section and click “Add Thesis”. Fill in the details for the new idea:

- **Title:** “Gold vs Inflation”
- **Hypothesis:** “If inflation expectations rise, gold will appreciate as a hedge.”
- **Drivers:** “Real interest rates decreasing; USD weakening.”
- **Disconfirmers:** “Fed tightening aggressively (higher real yields).”
- **Expression (allocation legs):** 100% long Gold (GLD) – meaning the thesis will allocate entirely to gold. (For simplicity, one asset.)

*(In UI, the user might enter Asset: GLD, Direction: Long, Size: 100% in an expression table.)*

**System Processing:** When the user saves, the UI calls POST /api/v1/thesis (assuming an endpoint exists to create theses). The backend creates a Thesis record in the database with the provided fields and an ID (say “T1”). It returns a success response with the new thesis data.

**Expectation:** The new thesis appears in the *Thesis list* on UI with status “Draft” (since not evaluated yet), and key fields displayed (title “Gold vs Inflation”, etc.). Initially, P&L is blank or 0 (no trades yet), and no risk flags yet.

*(Our design didn’t explicitly define an API response for creation, but presumably it’s similar to listing – anyway, the UI now has the thesis available.)*

## Step 2: Run Thesis Evaluation

**User Action:** Select the “Gold vs Inflation” thesis in the UI (maybe click it to view detail, or directly click an “Evaluate” button on the list). Click “**Run Evaluation**”.

**System Processing:** This triggers POST /api/v1/thesis/T1/evaluate.

- The backend (ThesisEvaluationSession) loads thesis T1 from DB.
- It calls ThesisEvaluationService:
  - Fetches historical prices for GLD. (System uses PriceSource; for demo, assume we have data from 2018–2023.)
  - Runs a backtest: perhaps GLD had ~+50% total return over 5 years with volatility ~15%, Sharpe ~0.5, max drawdown ~-20%. (These numbers are hypothetical but plausible.)
  - It constructs ThesisEvaluationResult with those metrics and the equity curve. For example:
    - total\_return ~ +50%, CAGR ~8.45%, volatility ~15%, sharpe ~0.56, max\_drawdown ~20%.
    - A couple of scenario impacts might be: “All -10%” scenario -> portfolio -10% (since fully in gold, it drops ~10%); “All +10%” -> +10%.
- Then it calls the LLM tool for review:
  - Provides the thesis context (text fields) and key evaluation metrics.
  - The LLM (GPT-4) returns a JSON like:

```
{
 "critique": "The thesis is reasonable: gold often rises with inflation, but it assumes the Fed won't counteract inflation (risk to thesis). Performance data shows moderate volatility (15% annually) and a Sharpe of 0.56, which is decent but not exceptional.",
 "questions": ["What if real interest rates increase instead of decrease?"],
 "risk_flags": ["Fed policy risk", "Single-asset concentration"],
 "insufficient_context": false
}
```

- - *(This content is fictitious but plausible: the AI notes Fed policy as a risk, and concentration risk because it's 100% gold.)*
- The session prepares a result combining quant and LLM outputs. It might also generate a TradePlan proposal:
  - e.g., TradePlan for \$1,000,000 total: “BUY 100% of capital in GLD”. If GLD price is \$180, that's ~5555 shares. (We may not show quantity yet in UI, just percent allocation.)
- Saves the evaluation in DB (for record).

**System Response:** The API returns 200 with a ThesisEvaluationSessionResult JSON. The UI receives this and updates the Thesis detail view:

- It shows **Performance Metrics:** e.g. *Total Return: 50%, CAGR: ~8.5%, Volatility: 15%, Sharpe: 0.56, Max Drawdown: 20%.*
- It shows **LLM Critique:**



- *Critique*: “The thesis is reasonable: gold often rises with inflation, but it assumes the Fed won’t counteract inflation...” (from JSON).
  - *Questions*: “What if real interest rates increase instead of decrease?”
  - *Risk Flags*: “Fed policy risk”; “Single-asset concentration”.
- It likely highlights those risk flags (maybe a warning icon next to thesis on list, or in detail we can mark them in red). Now the PM knows that the AI flagged the possibility of Fed tightening as a risk.
- It shows **Proposed Trade Plan**:
  - Possibly as: “Proposed Allocation: 100% in GLD (approx \$1,000,000 worth).” If UI displays number of shares, it might say “~5,555 shares of GLD at current price \$180.” (This comes from TradePlan; we could present it as “BUY \$1,000,000 of GLD”).
  - The PM can adjust the notional amount if UI allows (maybe default \$1M provided).

At this point, the thesis status might update to “Evaluated” or similar (not just Draft). In the list, it might show something like P&L still blank (still no actual trades, just evaluation). But risk flags column could now show an icon since the LLM flagged some (if we design it to).

**User Verification:** The PM reviews the results. The metrics look acceptable and align with their expectations (Sharpe ~0.5 is okay, not stellar; max DD 20% is tolerable). The critique makes sense (they are indeed assuming the Fed doesn’t hike rates too much). They appreciate the AI’s question and might refine the thesis later to address it. For now, they decide to proceed with it.

### Step 3: Approve Trade Plan (Paper Trade)

**User Action:** In the UI Thesis detail, click “**Approve Plan**” to implement this thesis in the paper portfolio.

**System Processing:** Triggers POST /api/v1/thesis/T1/approve-plan.

- The backend checks T1 isn’t already active (it’s not).
- PaperExecutionAdapter creates a TradePlan for T1 (if not reusing existing): 100% GLD of whatever notional (let’s say default \$1M).
- Executes the plan: fetches current price of GLD (say still \$180). Inserts a Trade record:

- Trade: trade\_id=UUID, timestamp=now, asset="GLD", action="BUY", quantity ~ 5555.56, price=180, type="PAPER", thesis\_ref="T1".
- Marks thesis T1 status = "ACTIVE" in DB.
- Returns a success response with details of executed trades (maybe the trade\_id or summary "Executed BUY of \$1,000,000 GLD at \$180"). For simplicity, maybe just {"status":"success"}.

**System Response:** UI receives confirmation (perhaps not much detail beyond success). The UI updates:

- On the Thesis list, "Gold vs Inflation" now moves to an "Active Theses" section (or it could simply show status Active).
- It now can show a P&L column entry for it, initially 0.0% (just opened trades).
- In the Portfolio view, a new position appears: "GLD – Quantity ~5555, Value ~\$1,000,000, Weight ~100%, P&L 0.0%" (or \$0 profit). The total portfolio value is \$1,000,000 now (since previously portfolio was empty).
- If portfolio view has risk metrics, it will show 100% concentration in GLD (which aligns with the risk flag the AI gave).

*(Now the PM has effectively "invested" in the thesis on paper.)*

## Step 4: View Portfolio and Risk Dashboard

**User Action:** Navigate to the *Portfolio* tab in the UI.

**System Display:**

- Current Portfolio:** Shows one position:
  - Asset: GLD, Quantity: ~5555.56, Current Price: \$180, Market Value: ~\$1,000,000, P&L: \$0 (0.00%).
- Totals:** Portfolio Value: \$1,000,000, Gross Exposure: \$1,000,000, Net Exposure: \$1,000,000 (since all long, no debt).
- Maybe a pie-chart or list indicates 100% of portfolio in GLD.
- Risk metrics might highlight this:
  - Concentration: GLD 100% of portfolio (maybe flagged as high).

- If we had factor data, it might say e.g. *Beta to inflation*: etc., but we don't, so skip.
  - The “Risk Flags” panel might reiterate any active risk flags across portfolio: likely “Single-asset concentration” as the LLM flagged. (UI could compile risk\_flags from all active theses to show a summary risk dashboard).
- Perhaps the UI also lists *Active Theses* with their performance:
  - “Gold vs Inflation – P&L: +0.0%, Capital: \$1,000,000, Sharpe: 0.56, Risk Flags: [Fed policy risk]”.
  - (Sharpe here is from eval, not realized – UI might or might not show that on portfolio page. Possibly it just shows P&L and a link to evaluation details if needed.)

The PM notes that indeed the portfolio is all gold, which is risky by concentration. They are comfortable for now given this is just a test and they want to see how daily updates work.

## Step 5: Simulate a Daily Market Update (Observation and Daily Update Session)

Let's fast-forward to the next day (or simulate as if a day passed with new data):

Suppose news comes out: Inflation is higher than expected (a disconfirming event for our thesis), and as a result, gold price moves.

### System (Data) Update:

- Insert a new Observation in DB:
  - id: “O42”, timestamp today 9:00 AM, text: “US CPI comes in at 6% vs 5% expected, Fed signals possibility of faster tightening.”, categories: [“inflation”], actionable=True, thesis\_refs: [“T1”] (we link it as disconfirming evidence for thesis T1, because it's exactly what T1's disconfirmer was).
- Update price data: Gold (GLD) price falls from \$180 to \$170 (due to tightening fears). (We simulate PriceSource now would give \$170 as current price.)

*(We do this behind the scenes. In a real scenario, Slice might ingest price updates via an API and observations via a news feed or user input. For testing, we manually add them.)*

**User Action:** The next morning, the PM clicks “Run Daily Update” in UI.

**System Processing:** POST /api/v1/session/daily-update:

- DataAccess loads active theses (T1), current portfolio (GLD position).
- Recomputes risk snapshot (unchanged in structure; still GLD 100%, maybe VaR slightly changed due to volatility shift).
- Checks for alerts:
  - It finds Observation O42 referencing T1 and marked actionable. It matches a disconfirmer (inflation up). So it creates an Alert:

Alert(thesis\_id="T1", thesis\_title="Gold vs Inflation", message="Disconfirming evidence: US CPI comes in at 6% vs 5% exp...", timestamp=...).

- Builds DailyContext:
  - Portfolio change: from \$1,000,000 to \$944,444 (since GLD -5.56% roughly: 170/180 ~ -5.6%, so portfolio lost ~\$55,556). That's -5.56% daily.
  - Top alert: “Thesis Gold vs Inflation: Disconfirming evidence: US CPI comes in at 6% vs 5% expected...”
  - Notable observation: include the CPI observation text.
- LLM daily\_summary:
  - Likely output:

```
{
 "key_narratives": [
 "Higher-than-expected inflation triggered a drop in gold prices, hurting the Gold vs Inflation thesis."
],
 "risk_highlights": [
 "The Fed's hawkish response to inflation is a new risk for the portfolio."
],
 "thesis_references": ["T1"],
 "insufficient_context": false
}
```

- - (The AI summarizes that gold fell because inflation surprise leads to hawkish Fed, which is indeed T1's risk scenario. It flags that explicitly.)
- Save alert and summary in DB.

**System Response:** API returns DailyUpdateSessionResult JSON. UI updates the Dashboard:

- **Alerts section:** shows an alert for “Gold vs Inflation” thesis: “*Disconfirming evidence: US CPI comes in at 6% vs 5% exp...*” (with today’s date). This likely is highlighted (maybe a red flag icon). The PM sees clearly that one of their theses had a critical development.
- **Daily Summary section:** shows something like:
  - *Key Narratives:* “Higher-than-expected inflation triggered a drop in gold prices, hurting the Gold vs Inflation thesis.”
  - *Risk Highlights:* “Fed’s hawkish response to inflation is now a risk to the portfolio.”
  - This is exactly what happened: the summary connects the dots between the inflation news and the portfolio impact (gold down).
- Possibly, the UI also updates portfolio values:
  - Now GLD position value ~\$944k, P&L = -5.6%. The portfolio total value shows \$944k (down from \$1M). The P&L for thesis T1 on the list might now show - 5.6%.
  - Net exposure still \$944k (all gold, it shrunk).
- The risk flags on thesis T1 might get an update if we incorporate the new info: The AI’s risk\_highlights basically reiterate Fed policy risk. We might not change the original risk\_flags field, but the daily summary serves to highlight it.

**User Reaction:** The PM sees the alert and realizes the disconfirming scenario (higher inflation leading to Fed tightening) is unfolding. The daily summary confirms this narrative. They appreciate that Slice flagged it and summarized the impact on their portfolio (which lost ~5.6%). This prompts them to consider adjusting the thesis or hedging:

- They might decide to take profit or cut the position because the reason to hold gold (rising inflation with dovish Fed) might be invalidated by Fed’s reaction.
- Alternatively, they create a new thesis to hedge Fed policy (not detailed here).

For now, this demonstrates how Slice’s evaluation architecture guided them:

- The **LLM critique** earlier warned about Fed policy risk. Now that risk manifested,
- The **alert** system caught the relevant event,
- And the **LLM daily summary** succinctly explained what happened and why it matters.

## Step 6: Intuition Q&A (optional)

Perhaps the PM has a question: “What’s the outlook for inflation now?” They go to the Q&A panel.

**User Action:** In the *Q&A* or *Intuition* tab, they type: “How likely is inflation to keep rising according to recent observations?” and select relevant observations (maybe the CPI one, or just let it use all recent obs).

**System Processing:** POST /api/v1/intel/qa with the question and context.

- MemoryService (not explicitly separate now) retrieves the recent observations (the CPI one).
- LLM llm\_query\_intuition gets that: it’s basically one data point: CPI high, Fed hawkish. It responds:

```
{
 "answer": "The latest observation indicates inflation is running higher
than expected, and the Fed is turning more hawkish. This suggests inflation
could remain elevated short-term, but the Fed's response may eventually curb
it. However, there's no observation yet indicating inflation will drop
soon.",
 "references": ["O42"],
 "insufficient_context": false
}
```

- 
- The system returns that.

**System Response:** UI displays:

- *Answer:* “The latest observation indicates inflation is running higher than expected, and the Fed is more hawkish. This suggests inflation could stay elevated in the short term, though the Fed’s actions might later curb it. We have no evidence yet that inflation will decline soon.”
- And it cites “Source: [Obs42]” (and perhaps the UI can show the CPI observation content if user clicks it).

This gives the PM a quick, context-based answer using the data they’ve seen. It basically synthesizes: inflation is high now and Fed is reacting, so near-term inflation might not fall. This aligns with their own thinking but having it articulated and backed by the observation builds confidence in the reasoning.

**Wrap-Up:** This use-case showed how the Slice evaluation architecture works in practice:

- The PM created a thesis and got immediate **quant feedback** (performance metrics) and an **AI critique** highlighting risks.
- They executed the strategy in a simulated portfolio.
- The next day, the system **alerted** them to relevant news impacting the thesis and provided an **AI-generated summary** connecting the news to portfolio performance.
- The PM could also query the system for additional insights (Q&A) to inform their decision-making.

The entire pipeline functioned smoothly:

Data flowed from DB -> quant engine -> LLM and back to DB, then to UI, without manual intervention. Each component (data, quant, LLM) added value:

- Data provided raw inputs (prices, observations),
- Quant evaluation provided objective measures,
- LLM provided subjective analysis and narrative,
- The orchestrator tied it together and ensured results were stored and surfaced properly.

Throughout, the PM was able to make informed decisions:

- They saw the risk of Fed policy early (LLM risk flag),
- Then saw it materialize (alert),
- And can now take action (maybe exiting gold or adjusting thesis).

This script demonstrates **comprehensively** the capabilities enabled by phases E1–E6 working in concert. We validated:

- Repositories handled create/read of thesis and trades correctly.
- Quant backtest gave sensible metrics.
- LLM outputs were structured and relevant (no hallucinations – it stuck to inflation and Fed which were in context).
- The end-to-end latency was fine: evaluation took maybe ~2s (quant negligible, LLM ~2s), daily update similar.
- There were no errors; if any had occurred (say LLM failed), the system would have logged and alerted (in metrics or returned error code) so we could address it.

The PM can continue to use the system, adding more theses, etc., with confidence that the evaluation harness will catch issues and explain performance.

## D. Implementation Checklist

To ensure the Slice Evaluation & Operational Test Architecture is fully implemented, we provide this checklist of deliverables. Each item is a must-have component (code or test) that should exist and be verified:

- **Data & Repository Layer (E1):**
  - ThesisRepository with methods: insert, get\_by\_id, list\_all, list\_recent, and update (for status changes).
  - ObservationRepository with methods: insert, get\_by\_id, list\_for\_thesis, list\_recent.
  - TradeRepository with methods: insert, list\_all, list\_by\_thesis.
  - (Optional) EvaluationRepository for saving evaluation results (thesis\_id, evaluation JSON, review JSON, timestamp).
  - DataAccess class that aggregates the above repos and provides:
    - get\_thesis(thesis\_id), get\_all\_theses().
    - get\_observations\_for\_thesis(thesis\_id), get\_recent\_observations(limit).
    - get\_current\_portfolio() returning PortfolioSnapshot (positions list and totals with portfolio\_value).
    - get\_portfolio\_depth() returning PortfolioDepthSnapshot (with concentration dict, thesis\_exposures dict, etc.).
  - Pydantic models: Position, PortfolioTotals, PortfolioSnapshot, PortfolioDepthSnapshot as specified (with portfolio\_value field in totals).
  - Unit tests for: repository round-trips (insert/get) and DataAccess methods (empty vs populated, correct aggregation).
- **Thesis Evaluation Service (E2):**
  - ThesisEvaluationService.evaluate\_thesis(thesis) implemented to perform price fetch and backtest:
    - Computes daily returns and equity curve.
    - Calculates performance metrics: total\_return, CAGR, volatility, sharpe, max\_drawdown (all floats).
    - Populates risk\_metrics dict (e.g., “max\_weight\_pct”, “VaR\_95”, “max\_drawdown\_pct”).
    - Defines at least two scenario shocks and computes ScenarioImpact for each (name, pnl\_pct, pnl\_abs).
  - Pydantic models: EquityPoint(date, value), ScenarioImpact(name, pnl\_abs, pnl\_pct), ThesisEvaluationResult (with fields: performance (dict), timeseries (list of EquityPoint), risk\_metrics (dict), scenarios (list of ScenarioImpact)).
  - Unit tests for evaluate\_thesis:



- Synthetic data scenario (like two assets up/down) verifying equity\_curve and metric outputs as expected.
  - Edge cases: one asset constant (vol=0 -> sharpe infinite or handled), weight sum <100 (cash portion).
  - Scenario outputs make sense (fully invested -> scenario impact equals shock %, etc.).
  - That ValueError is raised for invalid theses (e.g., >100% allocation or missing data).
- **LLM Tools & Harness (E3):**
  - Pydantic schemas for LLM outputs:
    - ThesisReview (critique, questions[], risk\_flags[], insufficient\_context).
    - CrossThesisReport (overlaps[], contradictions[], gaps[], insufficient\_context).
    - IntuitionAnswer (answer, references[], insufficient\_context).
    - DailySummary (key\_narratives[], risk\_highlights[], thesis\_references[], insufficient\_context).
  - Implementation of LLM wrapper functions:
    - llm\_review\_thesis(thesis, evaluation) -> ThesisReview.
    - llm\_cross\_theses(list\_of\_theses) -> CrossThesisReport.
    - llm\_query\_intuition(question, observations) -> IntuitionAnswer.
    - llm\_daily\_summary(context: DailyContext) -> DailySummary.
    - These should construct prompts, call SessionOrchestrator.run\_session, parse JSON to Pydantic, and handle errors.
  - Post-processing:
    - Reference validation: Unknown observation IDs in IntuitionAnswer are filtered and flag set true.
    - If JSON parse fails, attempt regex extraction of JSON and re-parse. If still fails, raise error.
    - Metrics logging: each function records call count, latency, and increments error count on exception.
  - Unit tests using a stub LLM (no actual API):
    - llm\_review\_thesis returns a ThesisReview matching a canned JSON; parsed object matches expected content.
    - llm\_review\_thesis handles an LLM output with extra text (ensures extract\_json works).
    - llm\_cross\_theses similarly tested with stub output.
    - llm\_query\_intuition tests:
      - Normal scenario: given obs list and stub answer referencing them, returns IntuitionAnswer correctly.
      - Hallucinated ref: model output references an obs not in input -> function removes it and sets insufficient\_context True.
    - llm\_daily\_summary stub test (valid JSON -> DailySummary).
    - All tests assert no exceptions, and the returned Pydantic objects have correct values.
    - Metrics: after calling e.g. llm\_review\_thesis twice, check that llm\_stats["thesis\_review"]["calls"] == 2 and errors 0.

- **Session Orchestrators & API (E4):**

- ThesisEvaluationSession.run(thesis\_id) implemented:
  - Loads thesis from DataAccess, raises 404 error if not found.
  - Calls evaluate\_thesis, calls llm\_review\_thesis.
  - If both succeed, saves or updates evaluation record (if EvaluationRepository exists).
  - Prepares ThesisEvaluationSessionResult (with thesis\_id, ThesisEvaluationResult, ThesisReview, TradePlan or None).
  - (Optionally calls exec\_adapter.create\_plan to include TradePlan in result).
  - Error handling:
    - If evaluate\_thesis raises (e.g., missing data) -> propagate or convert to exception (will be HTTP 400/500).
    - If llm\_review raises -> currently we propagate (session fails). (We decided to not return partial in final plan to simplify API output. If partial desired, ensure code catches and sets review=None then returns result with review=null).
- DailyUpdateSession.run() implemented:
  - Loads all active theses, current PortfolioSnapshot, and portfolio\_depth.
  - Gathers recent observations and triggers alerts for any matching theses (by thesis\_refs & actionable or text match).
  - Saves alerts via AlertRepository if present.
  - Builds DailyContext (with at least date, maybe portfolio\_change %, list of alert messages, maybe top 1-3 obs text).
  - Calls llm\_daily\_summary and gets DailySummary.
  - Saves DailySummary via SummaryRepository if present.
  - Returns DailyUpdateSessionResult (date, summary, alerts list).
  - If no active theses or nothing notable, summary still generated (or could be insufficient\_context true, which is fine).
  - Error handling: if llm\_daily\_summary fails, propagate error (so API returns 500).
- Pydantic models:
  - ThesisEvaluationSessionResult (thesis\_id, evaluation: ThesisEvaluationResult, review: ThesisReview, trade\_plan: TradePlan|None).
  - Alert (thesis\_id, thesis\_title, message, timestamp).
  - DailyUpdateSessionResult (date, summary: DailySummary, alerts: list[Alert]).
  - DailyContext for LLM input (if used internally).
- FastAPI endpoints:
  - POST /api/v1/thesis/{id}/evaluate:
    - Returns 200 + ThesisEvaluationSessionResult JSON on success.
    - Returns 404 if thesis not found, 400 if evaluation invalid (we convert ValueError from eval\_service to 400), 500 if internal error (like LLM failure).
    - The endpoint uses dependencies to inject DataAccess, and pre-instantiated services (eval\_service, orchestrator for LLM).

- (Alternatively, we might set up global instances or instantiate in handler as we showed, which is fine for now).
- POST /api/v1/session/daily-update:
  - Returns 200 + DailyUpdateSessionResult JSON on success.
  - Returns 500 if something failed (we catch exceptions from session.run).
- POST /api/v1/intel/qa:
  - Accepts QARequest (with question and optional thesis\_id filter).
  - Calls data\_access.get\_recent\_observations (and filter by thesis if given).
  - Calls llm\_query\_intuition(question, obs\_list).
  - Returns 200 + IntuitionAnswer JSON.
  - If LLM fails, 500.
- (Optional) POST /api/v1/thesis/compare for cross\_thesis, and GET /api/v1/diagnostics/llm for metrics:
  - The diagnostics route should gather llm\_stats and return it (ensuring thread safety if needed – our usage is sequential so fine).
- Integration tests (with FastAPI TestClient and overriding dependencies):
  - Set up a test database with a thesis and required data:
    - Insert Thesis T1 as in the scenario.
    - Seed PriceSource with dummy data for GLD (via monkeypatch or use a special PriceSource stub that returns from a dict).
    - Patch orchestrator/LLM calls to use fakes (like in unit tests) to ensure deterministic output.
  - **Test /thesis/{id}/evaluate:**
    - Call API, expect 200.
    - Verify response JSON has thesis\_id “T1”; has evaluation.performance.total\_return etc., and review.critique matching our fake LLM output.
    - Verify that in DB, an evaluation record or at least that DataAccess can fetch that thesis (with maybe a status change if we decide to mark it evaluated). We may not change status though.
    - Simulate error: e.g., remove price data so evaluate\_thesis raises ValueError “No price data”. Ensure API returns 400 with that message.
    - Simulate LLM error: patch llm\_review\_thesis to throw; ensure API returns 500 with “Evaluation failed: ”.
  - **Test /thesis/{id}/approve-plan:**
    - (Implement in E5, but include here for completeness)
    - Insert test thesis with expression.
    - Patch PriceSource to give a price.
    - Call API, expect 200 + trades info.
    - Check DB: trade\_repo.list\_by\_thesis has new trades with correct fields.
    - Check that thesis.status became ACTIVE.
    - Second call returns 400 “already active”.

- **Test /session/daily-update:**
  - Ensure at least one active thesis with a trade and an observation that should trigger alert is in DB. (Use test data from above scenario: T1 active, an observation referencing T1 disconfirms).
  - Patch PriceSource current prices to simulate daily move (GLD from 180->170).
  - Patch llm\_daily\_summary to a fake output.
  - Call API, expect 200.
  - Verify response JSON alerts list length (should catch that observation). Check alert thesis\_id = T1 and message contains snippet of obs text.
  - Verify summary matches fake LLM output.
  - Verify internal state: alert\_repo count increased by 1, summary\_repo if any got entry.
  - (We could also allow llm\_daily\_summary to actually run on context with our stub LLM. That might be fine too as long as stub returns something plausible.)
  - Edge test: if no active theses and no alerts, summary might come back insufficient\_context or just say nothing major. Ensure it still returns 200 with maybe alerts: [] and some summary (or insufficient\_context true, which is okay).
- **Test /intel/qa:**
  - Insert a couple observations (with IDs).
  - Patch llm\_query\_intuition to return a known JSON (with references).
  - Call API with a question.
  - Expect 200 with IntuitionAnswer JSON.
  - References in answer should be subset of provided obs IDs (and match exactly the stub's if stub did it correctly).
  - If optional thesis\_id filter tested: include obs linked to two different theses, call with one thesis\_id, ensure only that thesis's obs passed and maybe answer shorter (if stub built to consider that).
- **Test diagnostics:**
  - After making some above calls, call GET /api/v1/diagnostics/llm (if implemented).
  - Expect JSON with call counts matching how many times we invoked each tool in tests, and error counts 0 if no errors in those calls.
  - This confirms metrics accumulation is working.
- **Paper Trading & Execution (E5):**
  - TradePlan Pydantic model (thesis\_id, total\_value, legs list).
  - PaperExecutionAdapter class with:
    - create\_plan\_from\_thesis implemented (translates thesis.expression\_legs -> TradePlan).

- `execute_plan(plan)` implemented (for each leg, fetch price, compute quantity, create Trade with correct action, insert via `trade_repo`).
  - approve-plan endpoint implemented as per design (with idempotence check).
  - Unit tests for `PaperExecutionAdapter`:
    - Plan generation yields legs equal to thesis legs (just packaged).
    - Execution with known prices produces expected Trade objects (check action and quantity calculations).
    - Ensure it inserts trades and returns list.
    - Test short scenario: e.g., 100% short asset yields a SELL trade and that `DataAccess` would interpret it as a short position.
  - Integration test for approve-plan (some combined with API tests above):
    - After /evaluate, take that thesis (or another draft), call /approve-plan.
    - Then call /session/daily-update to ensure the new trades are seen (like portfolio now has that position). (This overlaps with daily-update test, but nice to chain: evaluate -> approve -> daily-update in sequence, akin to use case).
  - Verify that after approval, `thesis.status` is ACTIVE (maybe check via `get_thesis` or `list_all`).
  - Ensure double-approval prevention: call endpoint twice, second returns HTTP 400.
- **UI & Dashboard (E6 readiness):**

*(Although not code deliverables, ensure the following outputs exist from backend for UI to use)*

- Thesis list data: API for listing theses (if not already existing, ensure that `get_all_theses` can be exposed via GET /api/v1/theses). The list should include status and maybe last evaluation date and P&L. If not, UI can gather those from other endpoints:
  - We might simply use `get_all_theses` (with perhaps an added field for last evaluation metrics if we want to include Sharpe or so, but could omit).
- After evaluation, the `ThesisEvaluationSessionResult` returned to UI provides all info needed for the Thesis detail page (metrics and critique JSON).
- After daily-update, `DailyUpdateSessionResult` provides summary text and alerts for the Dashboard page.
- Alert objects include `thesis_id` and ideally `thesis_title` so UI can show a friendly name and link (we included `thesis_title` in Alert).
- `IntuitionAnswer` from QA endpoint has references so UI can let user click and see those observation texts (UI can call another endpoint or have them already loaded).
- Verify via the Use Stage script in section C manually that each step's output is indeed present and correctly structured:
  - We essentially did in the script; we would go through it for real with the running system using test or demo inputs to see that UI (or at least API responses) match the described values.

This checklist ensures that every interface, class, and test we planned is implemented. By checking off all items, we confirm the evaluation architecture is fully built, integrated, and verified. If any item is missing, the architecture is incomplete and should not be considered done until addressed .

////////////////

### For a phase-specific instance (say E2)

#### Message 1 in that chat:

1. Paste **global constraints** (short):
  - Tech stack: Python + FastAPI + Streamlit + Postgres.
  - You are *Deep Research Architect – Evaluation & Ops* for Slice.
  - You must not change the meaning of E1–E6 or invent new phases.
  - You do not write business logic; you design interfaces, DTOs, and tests.
2. Paste the **full text of that phase spec only** (E2 block: Requirements, Research, Design, Testing Plan, Risks).
3. Add a very explicit scoping sentence:

In this chat, you operate **only within Phase E2**. If something seems to require E1/E3/etc., you may reference it as a dependency, but you must not redesign those phases.

That's enough to keep the model locked on that phase for the whole instance.